

C Pointers

C Pointers

The pointer in C language is a variable which stores the address of another variable. This variable can be of type int, char, array, function, or any other pointer. The size of the pointer depends on the architecture. However, in 32-bit architecture the size of a pointer is 4 byte.

Consider the following example to define a pointer which stores the address of an integer.

```
int n = 10;
```

```
int* p = &n; // Variable p of type pointer is pointing to the address of the variable n of type integer.
```

Declaring a pointer

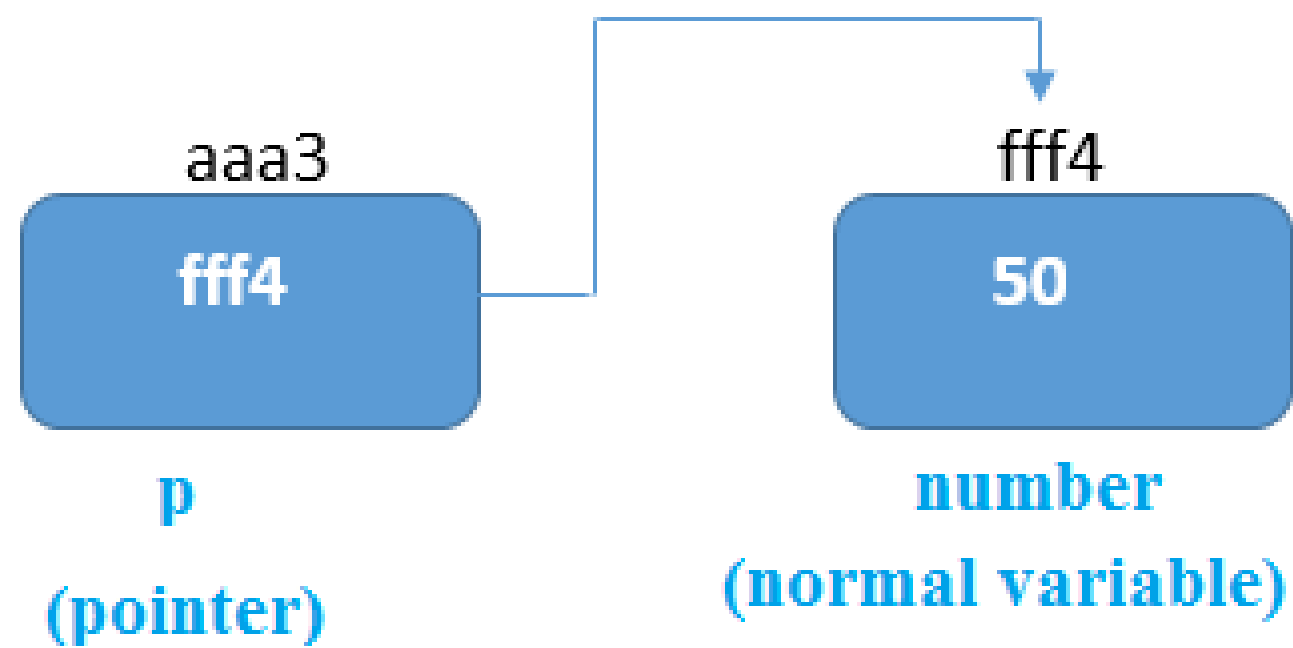
The pointer in c language can be declared using * (asterisk symbol). It is also known as indirection pointer used to dereference a pointer.

```
int *a; // pointer to int
```

```
char *c; // pointer to char
```

Pointer Example

An example of using pointers to print the address and value is given below.



javatpoint.com

As you can see in the above figure, pointer variable stores the address of number variable, i.e., fff4. The value of number variable is 50. But the address of pointer variable p is aaa3.

By the help of * (indirection operator), we can print the value of pointer variable p.

Let's see the pointer example as explained for the above figure.

```
#include<stdio.h>

int main(){

int number=50;

int *p;

p=&number;//stores the address of number variable

printf("Address of p variable is %x \n",p); // p contains the address of the number therefore
printing p gives the address of number.

printf("Value of p variable is %d \n",*p); // As we know that * is used to dereference a point
er therefore if we print *p, we will get the value stored at the address contained by p.

return 0;

}
```

Output

Address of number variable is fff4

Address of p variable is fff4

Value of p variable is 50

Pointer to array

```
int arr[10];
```

```
int *p[10]=&arr; // Variable p of type pointer is pointing to the address of an integer array a  
rr.
```

Pointer to a function

```
void show (int);
```

```
void(*p)(int) = &display; // Pointer p is pointing to the address of a function
```

Pointer to structure

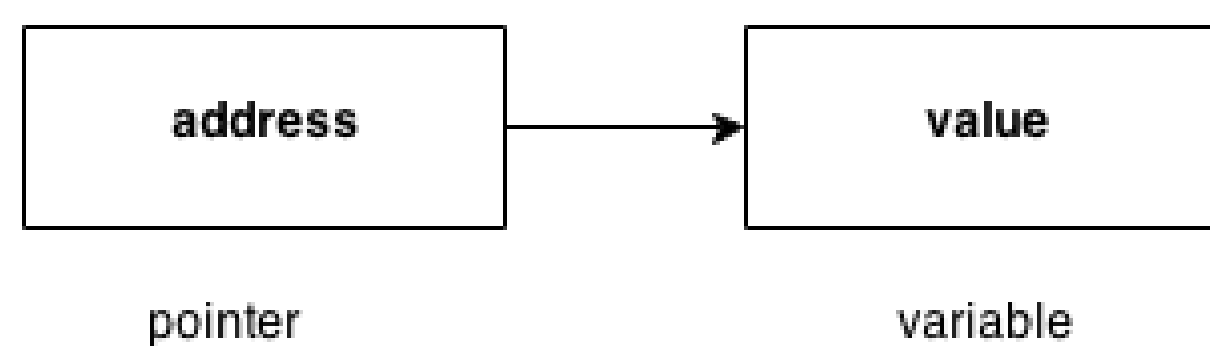
```
struct st {
```

```
    int i;
```

```
    float f;
```

```
}ref;
```

```
struct st *p = &ref;
```



Advantage of pointer

- 1) Pointer **reduces the code** and **improves the performance**, it is used to retrieving strings, trees, etc. and used with arrays, structures, and functions.
- 2) We can **return multiple values from a function** using the pointer.
- 3) It makes you able to **access any memory location** in the computer's memory.

Usage of pointer

There are many applications of pointers in c language.

1) Dynamic memory allocation

In c language, we can dynamically allocate memory using `malloc()` and `calloc()` functions where the pointer is used.

2) Arrays, Functions, and Structures

Pointers in c language are widely used in arrays, functions, and structures. It reduces the code and improves the performance.

Address Of (&) Operator

The address of operator '&' returns the address of a variable. But, we need to use `%u` to display the address of a variable.

```
#include<stdio.h>

int main(){

    int number=50;

    printf("value of number is %d, address of number is %u",number,&number);

    return 0;

}
```

Output

value of number is 50, address of number is fff4

NULL Pointer

A pointer that is not assigned any value but NULL is known as the NULL pointer. If you don't have any address to be specified in the pointer at the time of declaration, you can assign NULL value. It will provide a better approach.

```
int *p=NULL;
```

In the most libraries, the value of the pointer is 0 (zero).

Pointer Program to swap two numbers without using the 3rd variable.

```
#include<stdio.h>

int main(){

int a=10,b=20,*p1=&a,*p2=&b;

printf("Before swap: *p1=%d *p2=%d",*p1,*p2);

*p1=*p1+*p2;

*p2=*p1-*p2;

*p1=*p1-*p2;

printf("\nAfter swap: *p1=%d *p2=%d",*p1,*p2);

return 0;

}
```

Output

Before swap: *p1=10 *p2=20

After swap: *p1=20 *p2=10

Reading complex pointers

There are several things which must be taken into the consideration while reading the complex pointers in C. Lets see the precedence and associativity of the operators which are used regarding pointers.

Operator	Precedence	Associativity
<code>()</code> , <code>[]</code>	1	Left to right
<code>*</code> , identifier	2	Right to left
Data type	3	-

Here, we must notice that,

- `()`: This operator is a bracket operator used to declare and define the function.
- `[]`: This operator is an array subscript operator
- `*` : This operator is a pointer operator.
- Identifier: It is the name of the pointer. The priority will always be assigned to this.
- Data type: Data type is the type of the variable to which the pointer is intended to point. It also includes the modifier like signed int, long, etc).

How to read the pointer: `int (*p)[10]`.

To read the pointer, we must see that `()` and `[]` have the equal precedence. Therefore, their associativity must be considered here. The associativity is left to right, so the priority goes to `()`.

Inside the bracket `()`, pointer operator `*` and pointer name (identifier) `p` have the same precedence. Therefore, their associativity must be considered here which is right to left, so the priority goes to `p`, and the second priority goes to `*`.

Assign the 3rd priority to `[]` since the data type has the last precedence. Therefore the pointer will look like following.

- `char` -> 4
- `*` -> 2
- `p` -> 1
- `[10]` -> 3

The pointer will be read as `p` is a pointer to an array of integers of size 10.

Example

How to read the following pointer?

```
int (*p)(int (*)[2], int (*)void))
```

Explanation

This pointer will be read as `p` is a pointer to such function which accepts the first parameter as the pointer to a one-dimensional array of integers of size two and the second parameter as the pointer to a function which parameter is void and return type is the integer.

C Pointer to Pointer

As we know that, a pointer is used to store the address of a variable in C. Pointer reduces the access time of a variable. However, In C, we can also define a pointer to store the address of another pointer. Such pointer is known as a double pointer (pointer to pointer). The first pointer is used to store the address of a variable whereas the second pointer is used to store the address of the first pointer. Let's understand it by the diagram given below.



The syntax of declaring a double pointer is given below.

```
int **p; // pointer to a pointer which is pointing to an integer.
```

Consider the following example.

```
#include<stdio.h>

void main ( )
{
    int a = 10;

    int *p;

    int **pp;

    p = &a; // pointer p is pointing to the address of a

    pp = &p; // pointer pp is a double pointer pointing to the address of pointer p

    printf("address of a: %x\n",p); // Address of a will be printed
```

```

printf("address of p: %x\n",pp); // Address of p will be printed

printf("value stored at p: %d\n",*p); // value stoted at the address contained by p i.e. 10
will be printed

printf("value stored at pp: %d\n",**pp); // value stored at the address contained by the p
ointer stoyred at pp

}

```

Output

```

address of a: d26a8734

address of p: d26a8738

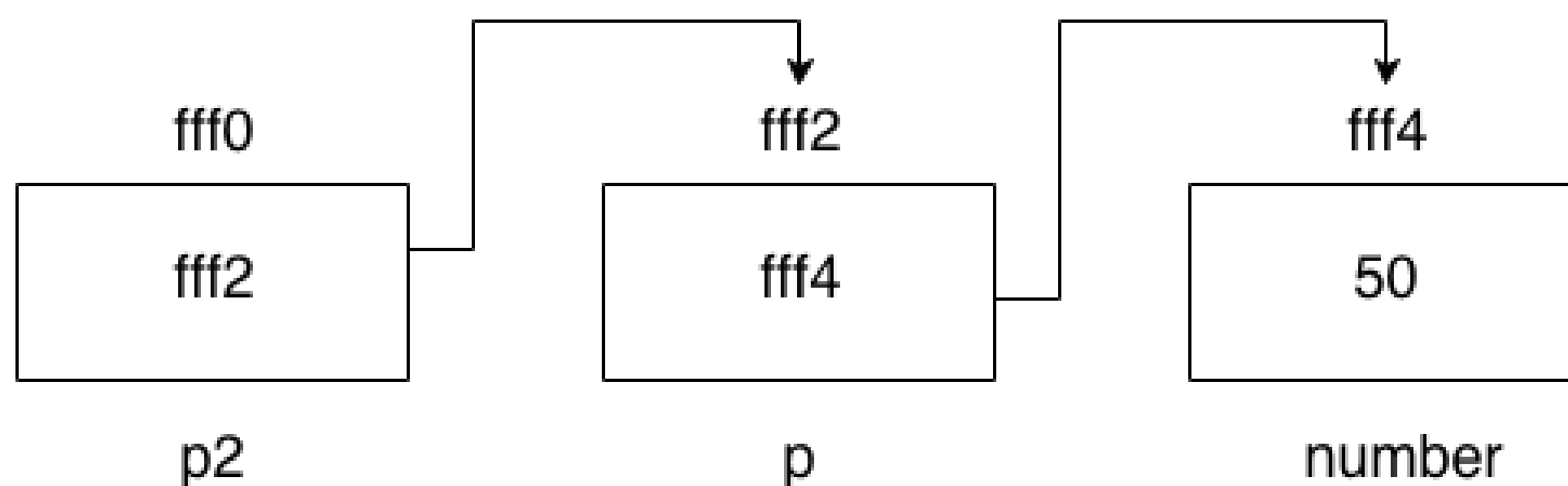
value stored at p: 10

value stored at pp: 10

```

C double pointer example

Let's see an example where one pointer points to the address of another pointer.



As you can see in the above figure, p2 contains the address of p (fff2), and p contains the address of number variable (fff4).

```

#include<stdio.h>

int main(){

```

```

int number=50;

int *p;//pointer to int

int **p2;//pointer to pointer

p=&number;//stores the address of number variable

p2=&p;

printf("Address of number variable is %x \n",&number);

printf("Address of p variable is %x \n",p);

printf("Value of *p variable is %d \n",*p);

printf("Address of p2 variable is %x \n",p2);

printf("Value of **p2 variable is %d \n",*p);

return 0;

}

```

Output

Address of number variable is fff4

Address of p variable is fff4

Value of *p variable is 50

Address of p2 variable is fff2

Value of **p variable is 50

Q. What will be the output of the following program?

```

#include<stdio.h>

void main ( )

{

```

```

int a[10] = {100, 206, 300, 409, 509, 601}; //Line 1

int *p[] = {a, a+1, a+2, a+3, a+4, a+5}; //Line 2

int **pp = p; //Line 3

pp++; // Line 4

printf("%d %d %d\n", pp-p, *pp - a, **pp); // Line 5

*pp++; // Line 6

printf("%d %d %d\n", pp-p, *pp - a, **pp); // Line 7

++*pp; // Line 8

printf("%d %d %d\n", pp-p, *pp - a, **pp); // Line 9

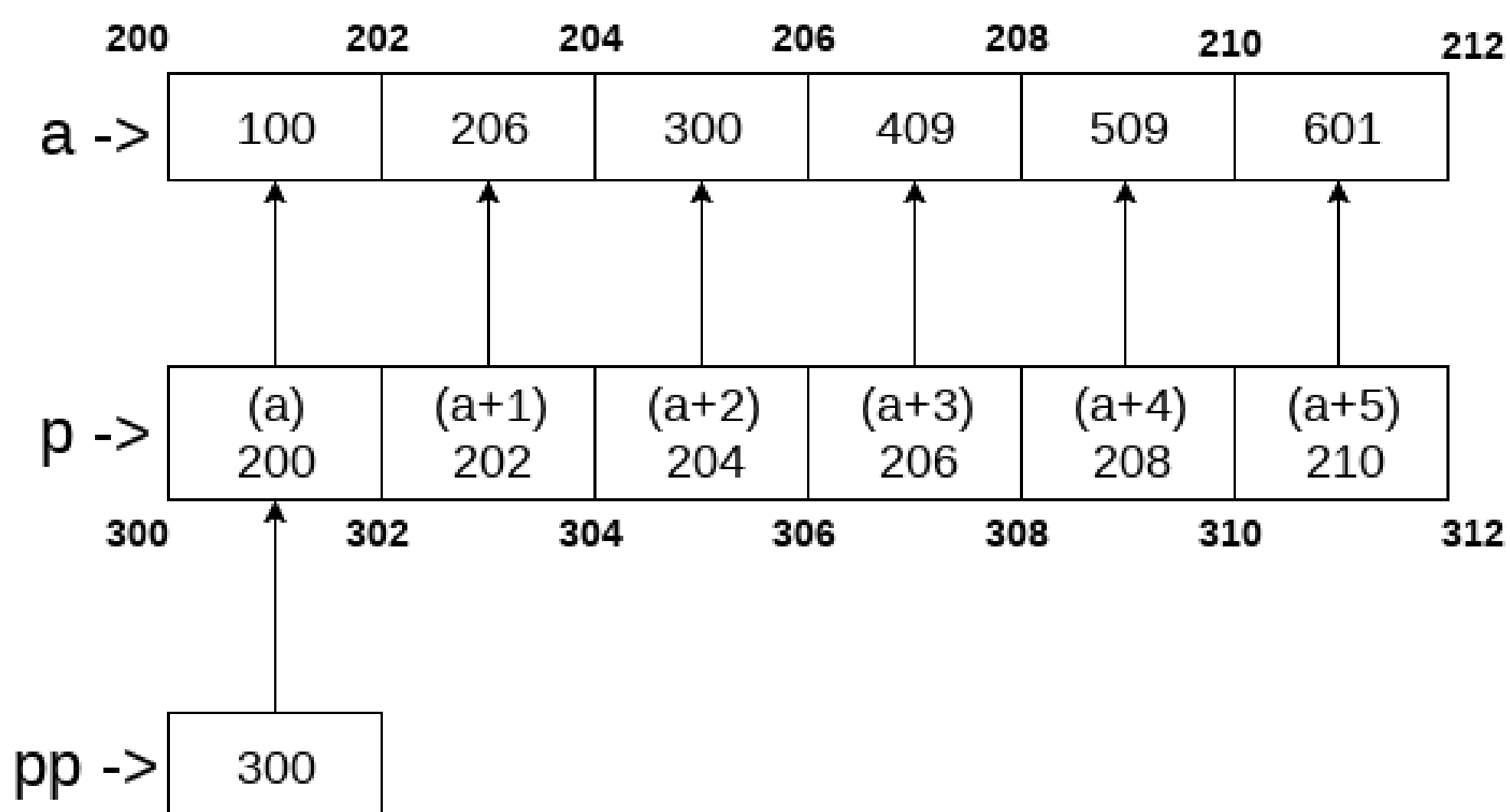
++**pp; // Line 10

printf("%d %d %d\n", pp-p, *pp - a, **pp); // Line 11

}

```

Explanation



To access $a[0]$ $\longrightarrow a[0] = * (a) = *p[0] = ** (p+0) = ** (pp+0) = 100$

In the above question, the pointer arithmetic is used with the double pointer. An array of 6 elements is defined which is pointed by an array of pointer p. The pointer array p is pointed by a double pointer pp. However, the above image gives you a brief idea about how the memory is being allocated to the array a and the pointer array p. The elements of p are the pointers that are pointing to every element of the array a. Since we know that the array name contains the base address of the array hence, it will work as a pointer and can the value can be traversed by using $*(a)$, $*(a+1)$, etc. As shown in the image, $a[0]$ can be accessed in the following ways.

- $a[0]$: it is the simplest way to access the first element of the array
- $*(a)$: since a store the address of the first element of the array, we can access its value by using indirection pointer on it.
- $*p[0]$: if $a[0]$ is to be accessed by using a pointer p to it, then we can use indirection operator ($*$) on the first element of the pointer array p, i.e., $*p[0]$.
- $**pp$: as pp stores the base address of the pointer array, $*pp$ will give the value of the first element of the pointer array that is the address of the first element of the integer array. $**p$ will give the actual value of the first element of the integer array.

Coming to the program, Line 1 and 2 declare the integer and pointer array relatively. Line 3 initializes the double pointer to the pointer array p. As shown in the image, if the address of the array starts from 200 and the size of the integer is 2, then the pointer array will contain the values as 200, 202, 204, 206, 208, 210. Let us consider that the base address of the pointer array is 300; the double pointer pp contains the address of pointer array, i.e., 300. Line number 4 increases the value of pp by 1, i.e., pp will now point to address 302.

Line number 5 contains an expression which prints three values, i.e., $pp - p$, $*pp - a$, $**pp$. Let's calculate them each one of them.

- $pp = 302, p = 300 \Rightarrow pp - p = (302 - 300) / 2 \Rightarrow pp - p = 1$, i.e., 1 will be printed.
- $pp = 302, *pp = 202, a = 200 \Rightarrow *pp - a = 202 - 200 = 2 / 2 = 1$, i.e., 1 will be printed.
- $pp = 302, *pp = 202, *(*pp) = 206$, i.e., 206 will be printed.

Therefore as the result of line 5, The output 1, 1, 206 will be printed on the console. On line 6, $*pp++$ is written. Here, we must notice that two unary operators $*$ and $++$ will have the same precedence. Therefore, by the rule of associativity, it will be evaluated from right to left. Therefore the expression $*pp++$ can be rewritten as $*(pp++)$. Since, $pp = 302$ which will now become, 304. $*pp$ will give 204.

On line 7, again the expression is written which prints three values, i.e., $pp - p$, $*pp - a$, $*pp$. Let's calculate each one of them.

- $pp = 304, p = 300 \Rightarrow pp - p = (304 - 300) / 2 \Rightarrow pp - p = 2$, i.e., 2 will be printed.
- $pp = 304, *pp = 204, a = 200 \Rightarrow *pp - a = (204 - 200) / 2 = 2$, i.e., 2 will be printed.
- $pp = 304, *pp = 204, *(*pp) = 300$, i.e., 300 will be printed.

Therefore, as the result of line 7, The output 2, 2, 300 will be printed on the console. On line 8, $++*pp$ is written. According to the rule of associativity, this can be rewritten as, $++(*pp)$. Since, $pp = 304$, $*pp = 204$, the value of $*pp = *(p[2]) = 206$ which will now point to $a[3]$.

On line 9, again the expression is written which prints three values, i.e., $pp - p$, $*pp - a$, $*pp$. Let's calculate each one of them.

- $pp = 304, p = 300 \Rightarrow pp - p = (304 - 300)/2 \Rightarrow pp - p = 2$, i.e., 2 will be printed.
- $pp = 304, *pp = 206, a = 200 \Rightarrow *pp - a = (206 - 200)/2 = 3$, i.e., 3 will be printed.
- $pp = 304, *pp = 206, *(*pp) = 409$, i.e., 409 will be printed.

Therefore, as the result of line 9, the output 2, 3, 409 will be printed on the console. On line 10, $++**pp$ is written. according to the rule of associativity, this can be rewritten as, $(++(*(*pp)))$. $pp = 304, *pp = 206, **pp = 409, ++**pp \Rightarrow *pp = *pp + 1 = 410$. In other words, $a[3] = 410$.

On line 11, again the expression is written which prints three values, i.e., $pp - p, *pp - a, *pp$. Let's calculate each one of them.

- $pp = 304, p = 300 \Rightarrow pp - p = (304 - 300)/2 \Rightarrow pp - p = 2$, i.e., 2 will be printed.
- $pp = 304, *pp = 206, a = 200 \Rightarrow *pp - a = (206 - 200)/2 = 3$, i.e., 3 will be printed.
- On line 8, $**pp = 410$.

Therefore as the result of line 9, the output 2, 3, 410 will be printed on the console.

At last, the output of the complete program will be given as:

Output

1 1 206

2 2 300

2 3 409

C Pointer Arithmetic

We can perform arithmetic operations on the pointers like addition, subtraction, etc. However, as we know that pointer contains the address, the result of an arithmetic operation performed on the pointer will also be a pointer if the other operand is of type integer. In pointer-from-pointer subtraction, the result will be an integer value. Following arithmetic operations are possible on the pointer in C language:

- Increment
- Decrement
- Addition
- Subtraction
- Comparison

Incrementing Pointer in C

If we increment a pointer by 1, the pointer will start pointing to the immediate next location. This is somewhat different from the general arithmetic since the value of the pointer will get increased by the size of the data type to which the pointer is pointing.

We can traverse an array by using the increment operation on a pointer which will keep pointing to every element of the array, perform some operation on that, and update itself in a loop.

The Rule to increment the pointer is given below:

$$\text{new_address} = \text{current_address} + i * \text{size_of}(\text{data type})$$

Where i is the number by which the pointer get increased.

32-bit

For 32-bit int variable, it will be incremented by 2 bytes.

64-bit

For 64-bit int variable, it will be incremented by 4 bytes.

Let's see the example of incrementing pointer variable on 64-bit architecture.

```
#include<stdio.h>

int main(){

int number=50;

int *p;//pointer to int

p=&number;//stores the address of number variable

printf("Address of p variable is %u \n",p);

p=p+1;

printf("After increment: Address of p variable is %u \n",p); // in our case, p will get incremented by 4 bytes.

return 0;

}
```

Output

Address of p variable is 3214864300

After increment: Address of p variable is 3214864304

Traversing an array by using pointer

```
#include<stdio.h>
```

```

void main ( )

{

    int arr[5] = {1, 2, 3, 4, 5};

    int *p = arr;

    int i;

    printf("printing array elements...\n");

    for(i = 0; i < 5; i++)

    {

        printf("%d ", *(p+i));

    }

}

```

Output

```

printing array elements...

1  2  3  4  5

```

Decrementing Pointer in C

Like increment, we can decrement a pointer variable. If we decrement a pointer, it will start pointing to the previous location. The formula of decrementing the pointer is given below:

$$\text{new_address} = \text{current_address} - i * \text{sizeof}(\text{data type})$$

32-bit

For 32-bit int variable, it will be decremented by 4 bytes.

64-bit

For 64-bit int variable, it will be decremented by 4 bytes.

Let's see the example of decrementing pointer variable on 64-bit OS.

```
#include <stdio.h>

void main(){

    int number=50;

    int *p;//pointer to int

    p=&number;//stores the address of number variable

    printf("Address of p variable is %u \n",p);

    p=p-1;

    printf("After decrement: Address of p variable is %u \n",p); // P will now point to the immediate previous location.

}
```

Output

Address of p variable is 3214864300

After decrement: Address of p variable is 3214864296

C Pointer Addition

We can add a value to the pointer variable. The formula of adding value to pointer is given below:

$$\text{new_address} = \text{current_address} + (\text{number} * \text{size_of}(\text{data type}))$$

32-bit

For 32-bit int variable, it will add $2 * \text{number}$.

64-bit

For 64-bit int variable, it will add $4 * \text{number}$.

Let's see the example of adding value to pointer variable on 64-bit architecture.

```
#include<stdio.h>

int main(){

int number=50;

int *p;//pointer to int

p=&number;//stores the address of number variable

printf("Address of p variable is %u \n",p);

p=p-3; //subtracting 3 from pointer variable

printf("After subtracting 3: Address of p variable is %u \n",p);

return 0;

}
```

Output

Address of p variable is 3214864300

After subtracting 3: Address of p variable is 3214864288

As you can see, the address of p is 3214864300. But after adding 3 with p variable, it is 3214864312, i.e., $4 * 3 = 12$ increment. Since we are using 64-bit architecture, it increments 12. But if we were using 32-bit architecture, it was incrementing to 6 only, i.e., $2 * 3 = 6$. As integer value occupies 2-byte memory in 32-bit OS.

C Pointer Subtraction

Like pointer addition, we can subtract a value from the pointer variable. Subtracting any number from a pointer will give an address. The formula of subtracting value from the pointer variable is given below:

```
new_address= current_address - (number * size_of(data type))
```

32-bit

For 32-bit int variable, it will subtract $2 * \text{number}$.

64-bit

For 64-bit int variable, it will subtract $4 * \text{number}$.

Let's see the example of subtracting value from the pointer variable on 64-bit architecture.

```
#include<stdio.h>

int main(){

    int number=50;

    int *p;//pointer to int

    p=&number;//stores the address of number variable

    printf("Address of p variable is %u \n",p);

    p=p-3; //subtracting 3 from pointer variable

    printf("After subtracting 3: Address of p variable is %u \n",p);

    return 0;

}
```

Output

```
Address of p variable is 3214864300
```

```
After subtracting 3: Address of p variable is 3214864288
```

You can see after subtracting 3 from the pointer variable, it is 12 ($4*3$) less than the previous address value.

However, instead of subtracting a number, we can also subtract an address from another address (pointer). This will result in a number. It will not be a simple arithmetic operation, but it will follow the following rule.

If two pointers are of the same type,

$$\text{Address2} - \text{Address1} = (\text{Subtraction of two addresses}) / \text{size of data type which pointer points}$$

Consider the following example to subtract one pointer from another.

```
#include<stdio.h>

void main ( )
{
    int i = 100;

    int *p = &i;

    int *temp;

    temp = p;

    p = p + 3;

    printf("Pointer Subtraction: %d - %d = %d",p, temp, p-temp);

}
```

Output

Pointer Subtraction: 1030585080 - 1030585068 = 3

Illegal arithmetic with pointers

There are various operations which can not be performed on pointers. Since, pointer stores address hence we must ignore the operations which may lead to an illegal

address, for example, addition, and multiplication. A list of such operations is given below.

- $\text{Address} + \text{Address} = \text{illegal}$
- $\text{Address} * \text{Address} = \text{illegal}$
- $\text{Address} \% \text{Address} = \text{illegal}$
- $\text{Address} / \text{Address} = \text{illegal}$
- $\text{Address} \& \text{Address} = \text{illegal}$
- $\text{Address} ^ \text{Address} = \text{illegal}$
- $\text{Address} | \text{Address} = \text{illegal}$
- $\sim \text{Address} = \text{illegal}$

Pointer to function in C

As we discussed in the previous chapter, a pointer can point to a function in C. However, the declaration of the pointer variable must be the same as the function. Consider the following example to make a pointer pointing to the function.

```
#include<stdio.h>
```

```
int addition ();
```

```
int main ()
```

```
{
```

```
    int result;
```

```
    int (*ptr)();
```

```

    ptr = &addition;

    result = ( *ptr)();

    printf("The sum is %d",result);

}

int addition()

{

    int a, b;

    printf("Enter two numbers?");

    scanf("%d %d",&a,&b);

    return a+b;

}

```

Output

Enter two numbers?10 15

The sum is 25

Pointer to Array of functions in C

To understand the concept of an array of functions, we must understand the array of function. Basically, an array of the function is an array which contains the addresses of functions. In other words, the pointer to an array of functions is a pointer pointing to an array which contains the pointers to the functions. Consider the following example.

```

#include<stdio.h>

int show();

int showadd(int);

```

```

int (*arr[3])();

int (*(*ptr)[3])();

int main ()

{

    int result1;

    arr[0] = show;

    arr[1] = showadd;

    ptr = &arr;

    result1 = (**ptr)();

    printf("printing the value returned by show : %d",result1);

    ((*ptr+1))(result1);

}

int show()

{

    int a = 65;

    return a++;

}

int showadd(int b)

{

    printf("\nAdding 90 to the value returned by show: %d",b+90);

}

```

Output

printing the value returned by show : 65

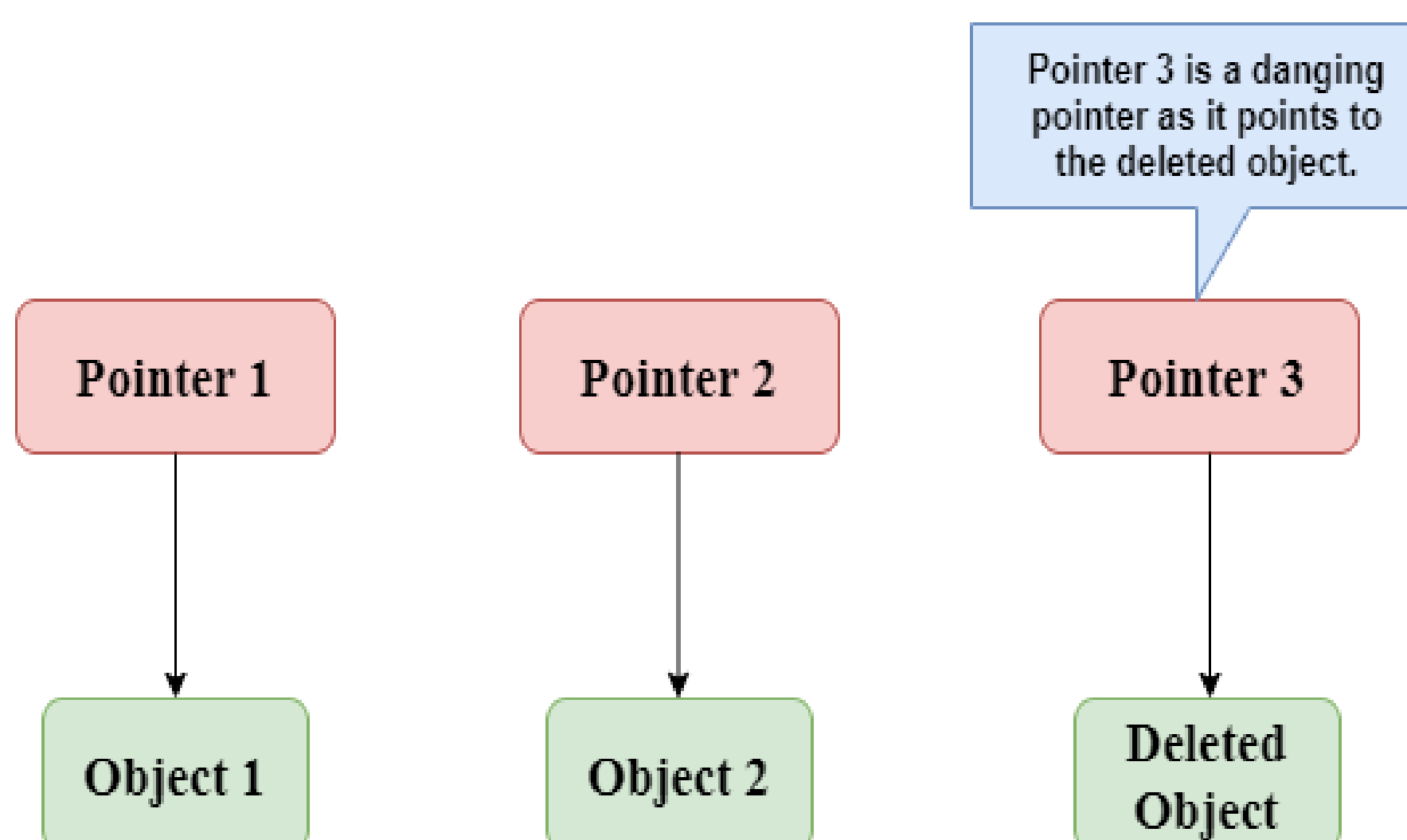
Adding 90 to the value returned by show: 155

Dangling Pointer in C

The most common bugs related to pointers and memory management is dangling/wild pointers. Sometimes the programmer fails to initialize the pointer with a valid address, then this type of initialized pointer is known as a dangling pointer in C.

Dangling pointer occurs at the time of the object destruction when the object is deleted or de-allocated from memory without modifying the value of the pointer. In this case, the pointer is pointing to the memory, which is de-allocated. The dangling pointer can point to the memory, which contains either the program code or the code of the operating system. If we assign the value to this pointer, then it overwrites the value of the program code or operating system instructions; in such cases, the program will show the undesirable result or may even crash. If the memory is re-allocated to some other process, then we dereference the dangling pointer will cause the segmentation faults.

Let's observe the following examples.



In the above figure, we can observe that the **Pointer 3** is a dangling pointer. **Pointer 1** and **Pointer 2** are the pointers that point to the allocated objects, i.e., Object 1 and Object 2, respectively. **Pointer 3** is a dangling pointer as it points to the de-allocated object.

Let's understand the dangling pointer through some C programs.

Using `free()` function to de-allocate the memory.

```
#include <stdio.h>

int main()
{
    int *ptr=(int *)malloc(sizeof(int));

    int a=560;

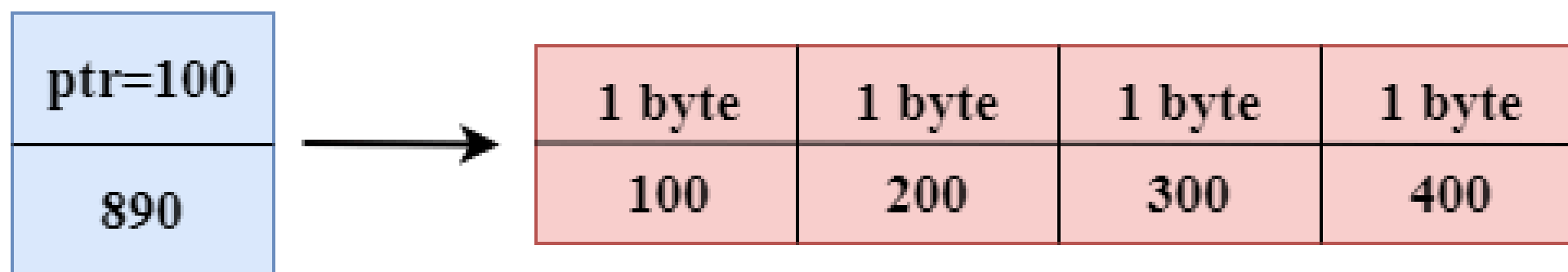
    ptr=&a;

    free(ptr);

    return 0;
}
```

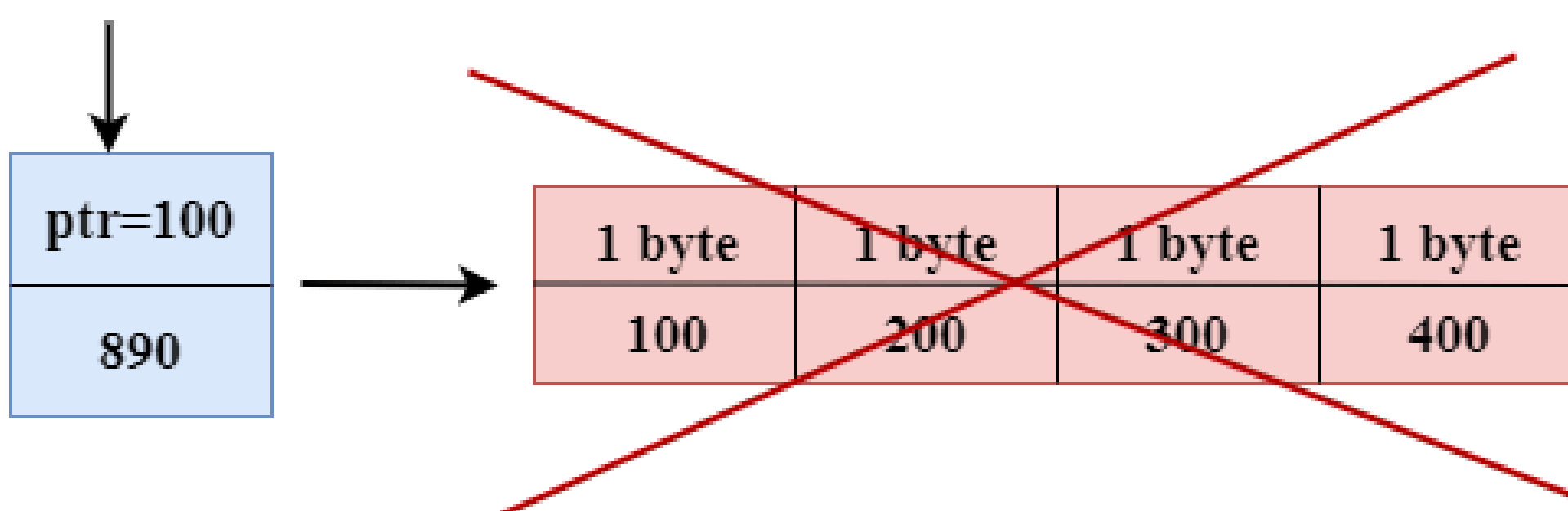
In the above code, we have created two variables, i.e., `*ptr` and `a` where '`ptr`' is a pointer and '`a`' is an integer variable. The `*ptr` is a pointer variable which is created with the help of `malloc()` function. As we know that `malloc()` function returns void, so we use `int *` to convert void pointer into int pointer.

The statement `int *ptr=(int *)malloc(sizeof(int));` will allocate the memory with 4 bytes shown in the below image:



The statement `free(ptr)` de-allocates the memory as shown in the below image with a cross sign, and 'ptr' pointer becomes dangling as it is pointing to the de-allocated memory.

Dangling pointer



If we assign the NULL value to the 'ptr', then 'ptr' will not point to the deleted memory. Therefore, we can say that ptr is not a dangling pointer, as shown in the below image:



Variable goes out of the scope

When the variable goes out of the scope then the pointer pointing to the variable becomes a **dangling pointer**.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    char *str;
```

```

{

    char a = 'A';

    str = &a;

}

// a falls out of scope

// str is now a dangling pointer

printf("%s", *str);

}

```

In the above code, we did the following steps:

- First, we declare the pointer variable named 'str'.
- In the inner scope, we declare a character variable. The str pointer contains the address of the variable 'a'.
- When the control comes out of the inner scope, 'a' variable will no longer be available, so str points to the de-allocated memory. It means that the str pointer becomes the dangling pointer.

Function call

Now, we will see how the pointer becomes dangling when we call the function.

Let's understand through an example.

```

#include <stdio.h>

int *fun(){

    int y=10;

```

```

    return &y;

}

int main()

{

    int *p=fun();

    printf("%d", *p);

    return 0;

}

```

In the above code, we did the following steps:

- First, we create the `main()` function in which we have declared 'p' pointer that contains the return value of the `fun()`.
- When the `fun()` is called, then the control moves to the context of the `int *fun()`, the `fun()` returns the address of the 'y' variable.
- When control comes back to the context of the `main()` function, it means the variable 'y' is no longer available. Therefore, we can say that the 'p' pointer is a dangling pointer as it points to the de-allocated memory.

Output

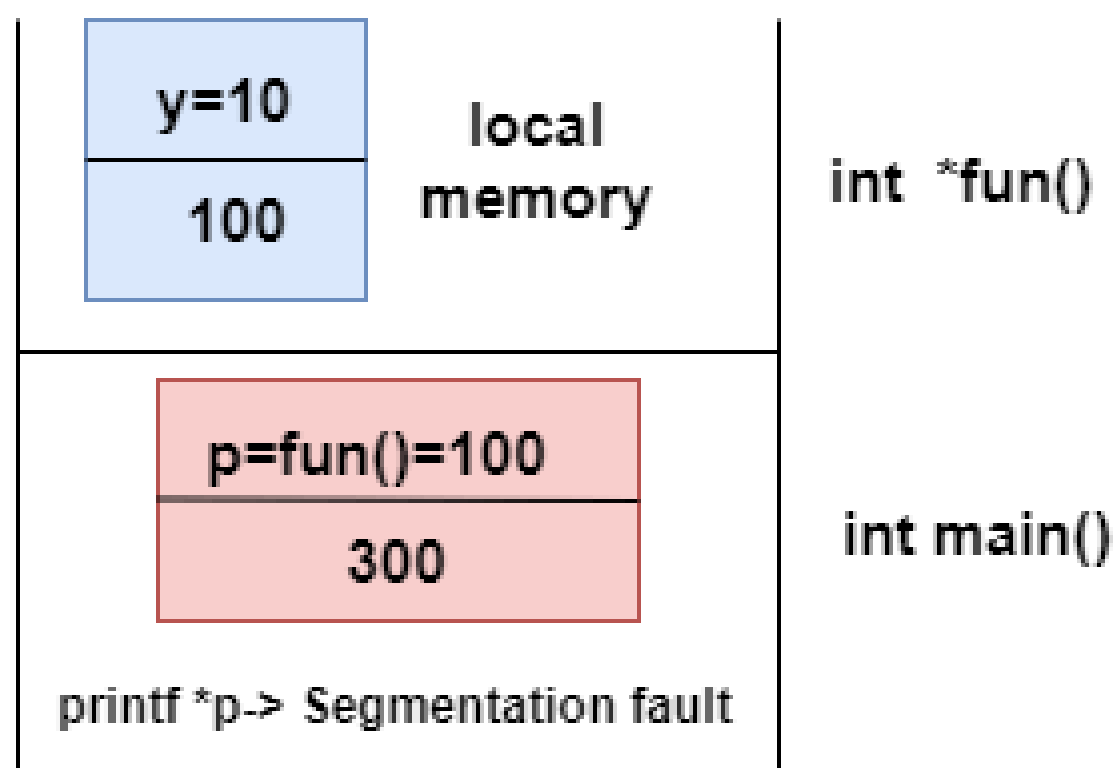
```

Segmentation fault

...Program finished with exit code 139
Press ENTER to exit console.

```

Let's represent the working of the above code diagrammatically.



Let's consider another example of a dangling pointer.

```
#include <stdio.h>

int *fun()
{
    static int y=10;

    return &y;
}

int main()
{
    int *p=fun();

    printf("%d", *p);

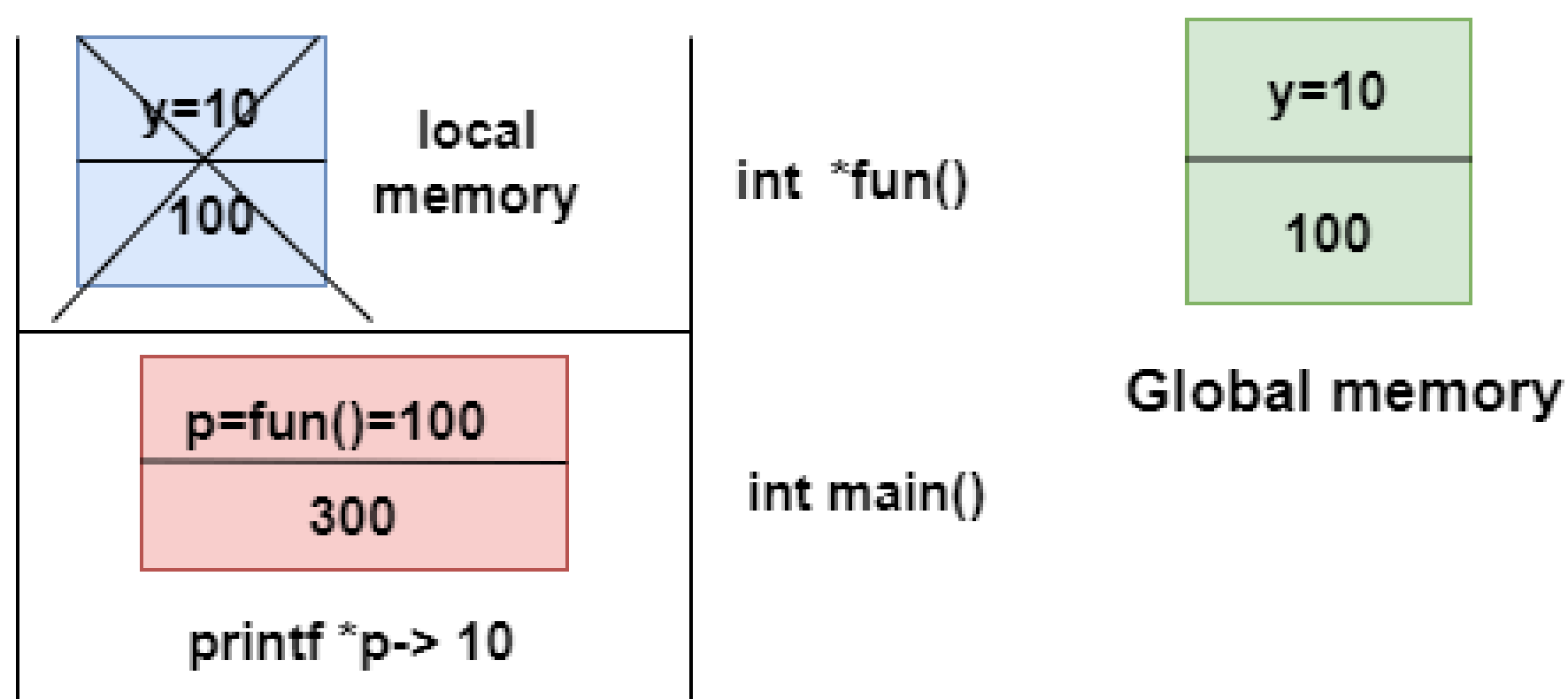
    return 0;
}
```

The above code is similar to the previous one but the only difference is that the variable 'y' is static. We know that static variable stores in the global memory.

Output

```
10
...Program finished with exit code 0
Press ENTER to exit console. □
```

Now, we represent the working of the above code diagrammatically.



The above diagram shows the stack memory. First, the `fun()` function is called, then the control moves to the context of the `int *fun()`. As `y` is a static variable, so it stores in the global memory; Its scope is available throughout the program. When the address value is returned, then the control comes back to the context of the `main()`. The pointer `p` contains the address of `y`, i.e., 100. When we print the value of `*p`, then it prints the value of `y`, i.e., 10. Therefore, we can say that the pointer `p` is not a dangling pointer as it contains the address of the variable which is stored in the global memory.

Avoiding Dangling Pointer Errors

The dangling pointer errors can be avoided by initializing the pointer to the **NULL** value.

If we assign the **NULL** value to the pointer, then the pointer will not point to the

de-allocated memory. Assigning **NULL** value to the pointer means that the pointer is not pointing to any memory location.

sizeof() operator in C

The **sizeof()** operator is commonly used in C. It determines the size of the expression or the data type specified in the number of char-sized storage units. The **sizeof()** operator contains a single operand which can be either an expression or a data typecast where the cast is data type enclosed within parenthesis. The data type cannot only be primitive data types such as integer or floating data types, but it can also be pointer data types and compound data types such as unions and structs.

Need of sizeof() operator

Mainly, programs know the storage size of the primitive data types. Though the storage size of the data type is constant, it varies when implemented in different platforms. For example, we dynamically allocate the array space by using **sizeof()** operator:

```
int *ptr=malloc(10*sizeof(int));
```

In the above example, we use the **sizeof()** operator, which is applied to the cast of type **int**. We use **malloc()** function to allocate the memory and returns the pointer which is pointing to this allocated memory. The memory space is equal to the number of bytes occupied by the **int** data type and multiplied by 10.

Note: The output can vary on different machines such as on 32-bit operating system will show different output, and the 64-bit operating system will show the different outputs of the same data types.

The **sizeof()** operator behaves differently according to the type of the operand.

- Operand is a data type

- Operand is an expression

When operand is a data type.

```
#include <stdio.h>

int main()

{

    int x=89;    // variable declaration.

    printf("size of the variable x is %d", sizeof(x)); // Displaying the size of ?x? variable.

    printf("\nsize of the integer data type is %d",sizeof(int)); //Displaying the size of integer
data type.

    printf("\nsize of the character data type is %d",sizeof(char)); //Displaying the size of cha
racter data type.

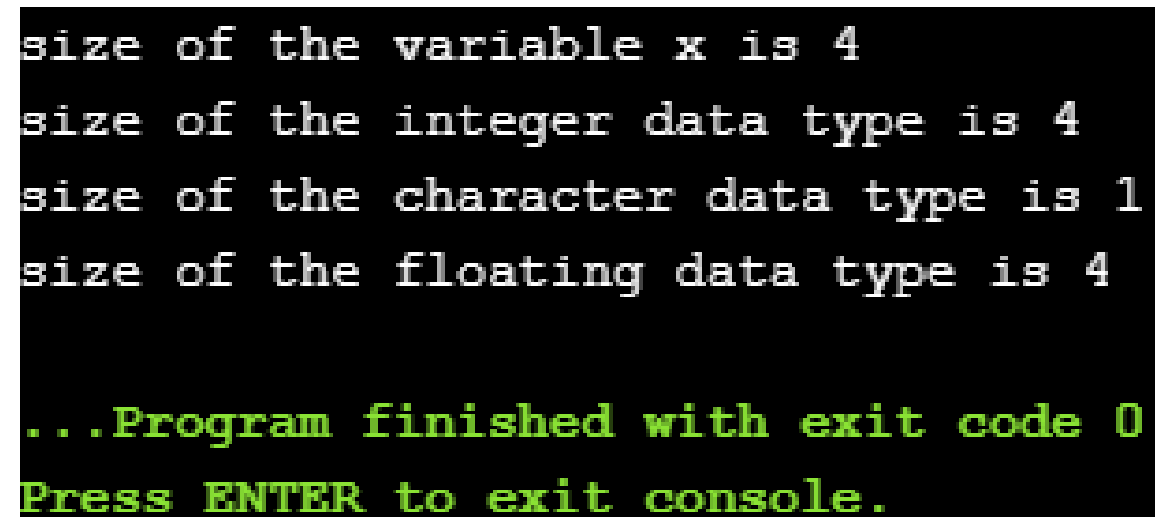
    printf("\nsize of the floating data type is %d",sizeof(float)); //Displaying the size of floati
ng data type.

    return 0;

}
```

In the above code, we are printing the size of different data types such as int, char, float with the help of **sizeof()** operator.

Output

A screenshot of a terminal window showing the output of the C program. The text is displayed in a monospaced font on a black background. The output shows the size of a variable 'x' (4), the size of an integer data type (4), the size of a character data type (1), and the size of a floating data type (4). It also shows the program finishing with exit code 0 and a prompt to press ENTER to exit the console.

```
size of the variable x is 4
size of the integer data type is 4
size of the character data type is 1
size of the floating data type is 4

...Program finished with exit code 0
Press ENTER to exit console.
```

When operand is an expression

```

#include <stdio.h>

int main()

{

    double i=78.0; //variable initialization.

    float j=6.78; //variable initialization.

    printf("size of (i+j) expression is : %d",sizeof(i+j)); //Displaying the size of the expression
    (i+j).

    return 0;

}

```

In the above code, we have created two variables 'i' and 'j' of type double and float respectively, and then we print the size of the expression by using **sizeof(i+j)** operator.

Output

```
size of (i+j) expression is : 8
```

Constant Pointer in C

Constant Pointers

A constant pointer in C cannot change the address of the variable to which it is pointing, i.e., the address will remain constant. Therefore, we can say that if a constant pointer is pointing to some variable, then it cannot point to any other variable.

Syntax of Constant Pointer

```
<type of pointer> *const <name of pointer>;
```

Declaration of a constant pointer is given below:

```
int *const ptr;
```

Let's understand the constant pointer through an example.

```
#include <stdio.h>

int main()

{

    int a=1;

    int b=2;

    int *const ptr;

    ptr=&a;

    ptr=&b;

    printf("Value of ptr is :%d", *ptr);

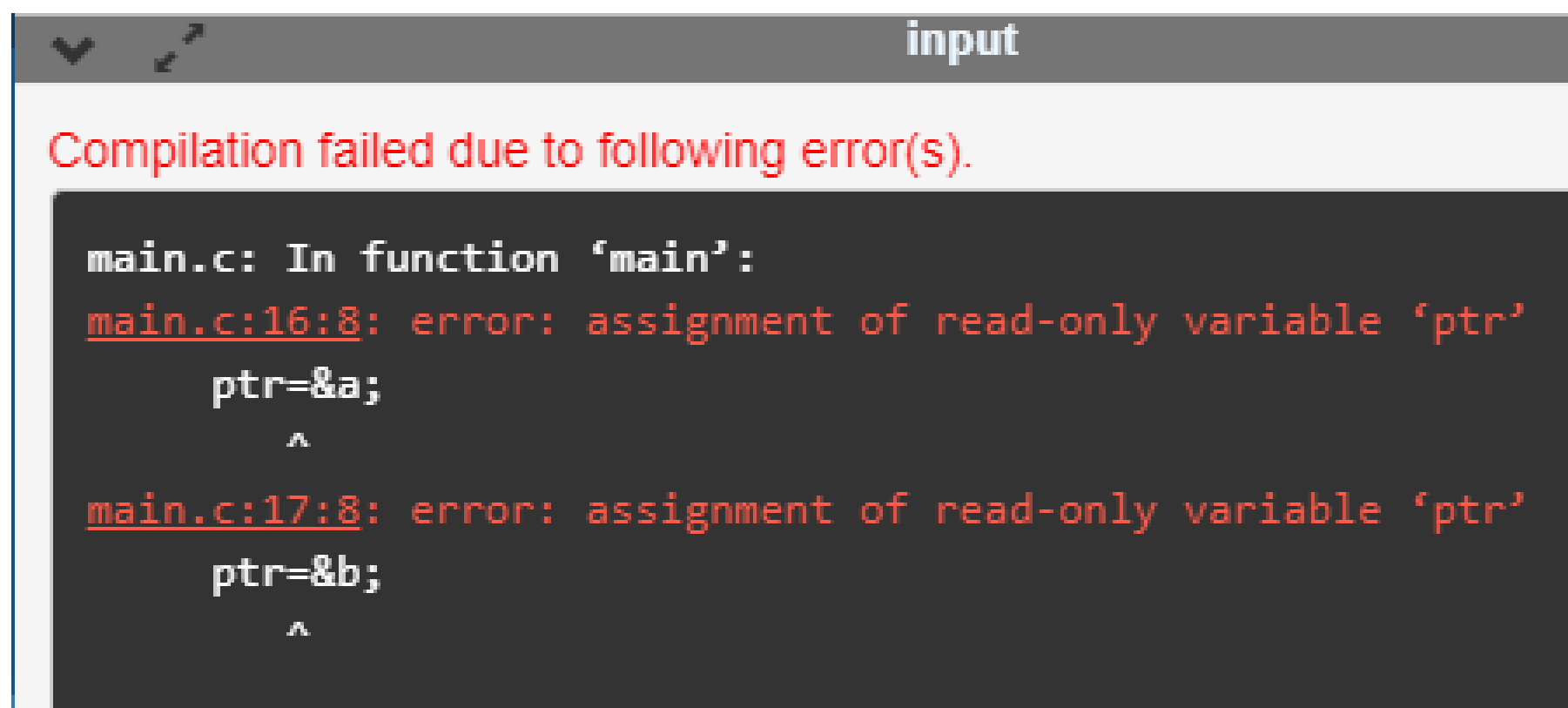
    return 0;

}
```

In the above code:

- We declare two variables, i.e., a and b with values 1 and 2, respectively.
- We declare a constant pointer.
- First, we assign the address of variable 'a' to the pointer 'ptr'.
- Then, we assign the address of variable 'b' to the pointer 'ptr'.
- Lastly, we try to print the value of the variable pointed by the 'ptr'.

Output

A screenshot of a compiler error message displayed in a window titled "input". The message is in red text on a light gray background, stating "Compilation failed due to following error(s)". Below this, on a dark gray background, the error details are shown in white and red text. It indicates the error occurs in "main.c: In function 'main':" at two locations: "main.c:16:8: error: assignment of read-only variable 'ptr'" and "main.c:17:8: error: assignment of read-only variable 'ptr'". The code snippets for these lines are "ptr=&a;" and "ptr=&b;" respectively, with a caret (^) pointing to the 'ptr' variable in each line.

```
main.c: In function 'main':
main.c:16:8: error: assignment of read-only variable 'ptr'
    ptr=&a;
    ^
main.c:17:8: error: assignment of read-only variable 'ptr'
    ptr=&b;
    ^
```

In the above output, we can observe that the above code produces the error "assignment of read-only variable 'ptr'". It means that the value of the variable 'ptr' which 'ptr' is holding cannot be changed. In the above code, we are changing the value of 'ptr' from &a to &b, which is not possible with constant pointers. Therefore, we can say that the constant pointer, which points to some variable, cannot point to another variable.

Pointer to Constant

A pointer to constant is a pointer through which the value of the variable that the pointer points cannot be changed. The address of these pointers can be changed, but the value of the variable that the pointer points cannot be changed.

Syntax of Pointer to Constant

`const` <type of pointer>* <name of pointer>

Declaration of a pointer to constant is given below:

`const int*` ptr;

Let's understand through an example.

First, we write the code where we are changing the value of a pointer

```
#include <stdio.h>
```

```
int main()  
  
{  
  
    int a=100;  
  
    int b=200;  
  
    const int* ptr;  
  
    ptr=&a;  
  
    ptr=&b;  
  
    printf("Value of ptr is :%u",ptr);  
  
    return 0;  
  
}
```

In the above code:

- We declare two variables, i.e., a and b with the values 100 and 200 respectively.
- We declare a pointer to constant.
- First, we assign the address of variable 'a' to the pointer 'ptr'.
- Then, we assign the address of variable 'b' to the pointer 'ptr'.
- Lastly, we try to print the value of 'ptr'.

Output

```
Value of ptr is :247760772
```

The above code runs successfully, and it shows the value of 'ptr' in the output.

Now, we write the code in which we are changing the value of the variable to which the pointer points.

```
#include <stdio.h>

int main()

{

    int a=100;

    int b=200;

    const int* ptr;

    ptr=&b;

    *ptr=300;

    printf("Value of ptr is :%d", *ptr);

    return 0;

}
```

In the above code:

- We declare two variables, i.e., 'a' and 'b' with the values 100 and 200 respectively.
- We declare a pointer to constant.
- We assign the address of the variable 'b' to the pointer 'ptr'.
- Then, we try to modify the value of the variable 'b' through the pointer 'ptr'.

- Lastly, we try to print the value of the variable which is pointed by the pointer 'ptr'.

Output

```
main.c: In function 'main':
main.c:17:9: error: assignment of read-only location '*ptr'
    *ptr=300;
        ^
```

The above code shows the error "assignment of read-only location '*ptr'". This error means that we cannot change the value of the variable to which the pointer is pointing.

Constant Pointer to a Constant

A constant pointer to a constant is a pointer, which is a combination of the above two pointers. It can neither change the address of the variable to which it is pointing nor it can change the value placed at this address.

Syntax

```
const <type of pointer> * const <name of the pointer>;
```

Declaration for a constant pointer to a constant is given below:

```
const int* const ptr;
```

Let's understand through an example.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a=10;
```

```

int b=90;

const int* const ptr=&a;

*ptr=12;

ptr=&b;

printf("Value of ptr is :%d",*ptr);

return 0;

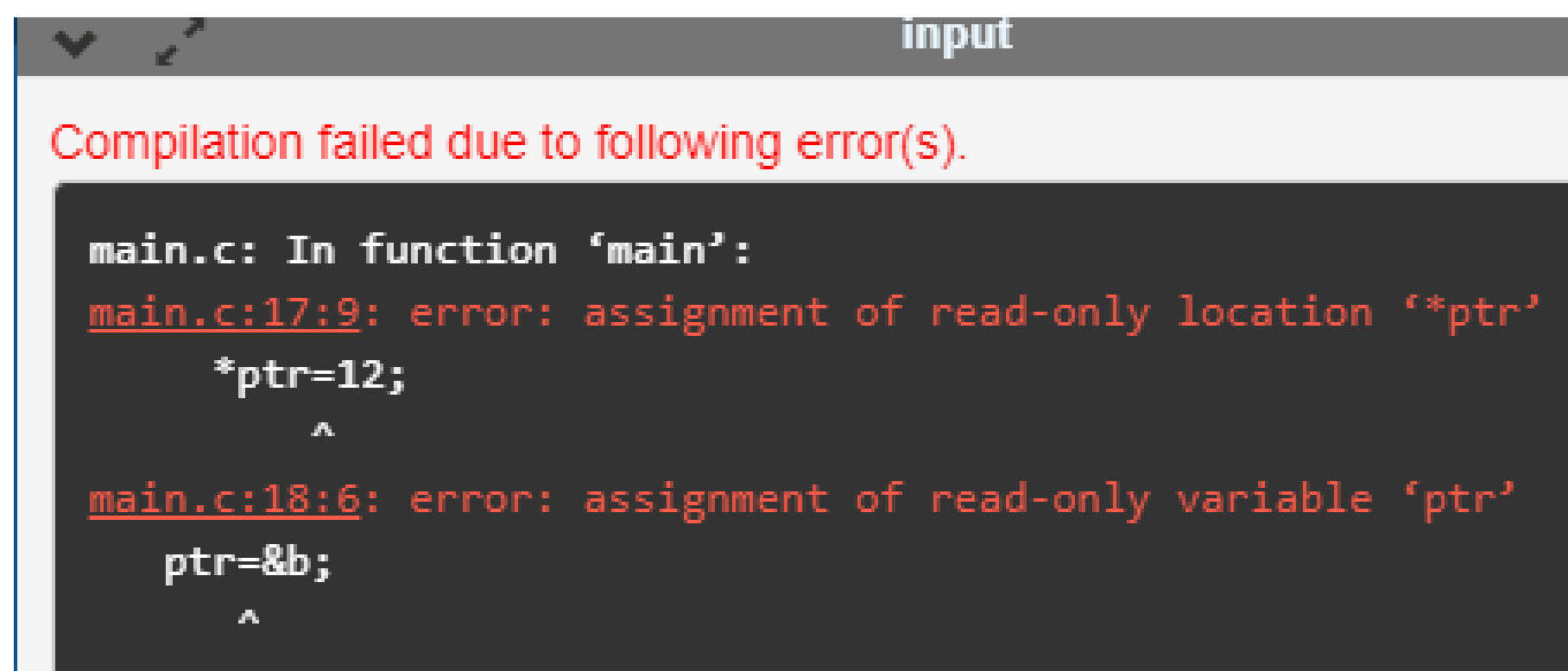
}

```

In the above code:

- We declare two variables, i.e., 'a' and 'b' with the values 10 and 90, respectively.
- We declare a constant pointer to a constant and then assign the address of 'a'.
- We try to change the value of the variable 'a' through the pointer 'ptr'.
- Then we try to assign the address of variable 'b' to the pointer 'ptr'.
- Lastly, we print the value of the variable, which is pointed by the pointer 'ptr'.

Output



```

input
Compilation failed due to following error(s).

main.c: In function 'main':
main.c:17:9: error: assignment of read-only location '*ptr'
    *ptr=12;
    ^
main.c:18:6: error: assignment of read-only variable 'ptr'
    ptr=&b;
    ^

```

The above code shows the error "assignment of read-only location '*ptr'" and "assignment of read-only variable 'ptr'". Therefore, we conclude that the constant pointer to a constant can change neither address nor value, which is pointing by this pointer.

void pointer in C

Till now, we have studied that the address assigned to a pointer should be of the same type as specified in the pointer declaration. For example, if we declare the int pointer, then this int pointer cannot point to the float variable or some other type of variable, i.e., it can point to only int type variable. To overcome this problem, we use a pointer to void. A pointer to void means a generic pointer that can point to any data type. We can assign the address of any data type to the void pointer, and a void pointer can be assigned to any type of the pointer without performing any explicit typecasting.

Syntax of void pointer

```
void *pointer name;
```

Declaration of the void pointer is given below:

```
void *ptr;
```

In the above declaration, the void is the type of the pointer, and 'ptr' is the name of the pointer.

Let us consider some examples:

```
int i=9;    // integer variable initialization.
```

```
int *p;     // integer pointer declaration.
```

```
float *fp;  // floating pointer declaration.
```

```
void *ptr;  // void pointer declaration.
```

```
p=fp;      // incorrect.
```

```
fp=&i;    // incorrect
```

```
ptr=p;    // correct
```

```
ptr=fp;    // correct
```

```
ptr=&i;    // correct
```

Size of the void pointer in C

The size of the void pointer in C is the same as the size of the pointer of character type. According to C perception, the representation of a pointer to void is the same as the pointer of character type. The size of the pointer will vary depending on the platform that you are using.

Let's look at the below example:

```
#include <stdio.h>

int main()

{

    void *ptr = NULL; //void pointer

    int *p = NULL; // integer pointer

    char *cp = NULL; //character pointer

    float *fp = NULL; //float pointer

    //size of void pointer

    printf("size of void pointer = %d\n\n",sizeof(ptr));

    //size of integer pointer

    printf("size of integer pointer = %d\n\n",sizeof(p));

    //size of character pointer
```

```

printf("size of character pointer = %d\n\n",sizeof(cp));

//size of float pointer

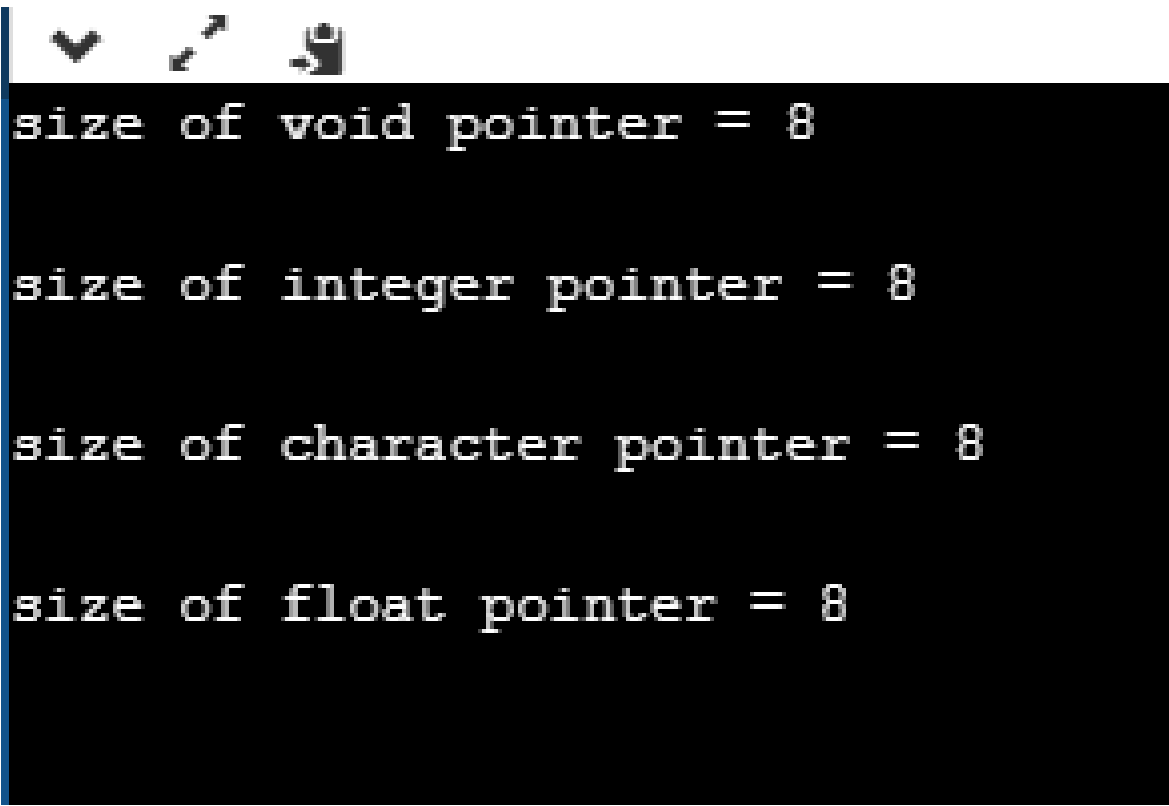
printf("size of float pointer = %d\n\n",sizeof(fp));

return 0;

}

```

Output



```

size of void pointer = 8

size of integer pointer = 8

size of character pointer = 8

size of float pointer = 8

```

Advantages of void pointer

Following are the advantages of a void pointer:

- The malloc() and calloc() function return the void pointer, so these functions can be used to allocate the memory of any data type.

```
#include <stdio.h>
```

```
#include<malloc.h>
```

```
int main()
```

```
{
```

```
int a=90;
```

```

int *x = (int *)malloc(sizeof(int)) ;

x=&a;

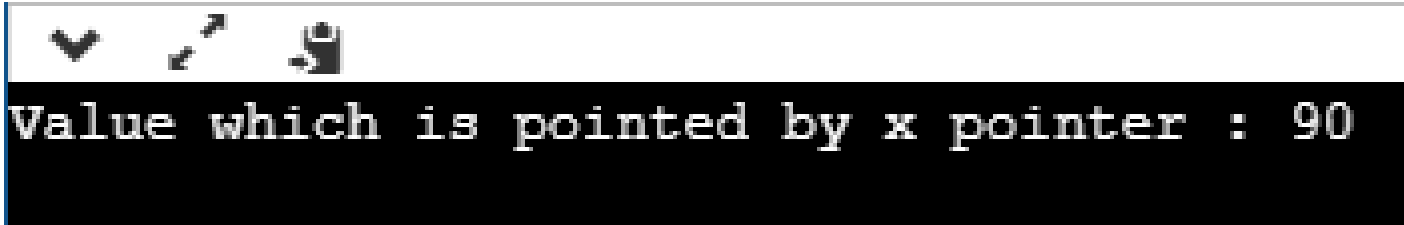
printf("Value which is pointed by x pointer : %d",*x);

return 0;

}

```

Output



```

Value which is pointed by x pointer : 90

```

- The void pointer in C can also be used to implement the generic functions in C.

Some important points related to void pointer are:

- **Dereferencing a void pointer in C**

The void pointer in C cannot be dereferenced directly. Let's see the below example.

```

#include <stdio.h>

int main()

{

    int a=90;

    void *ptr;

    ptr=&a;

    printf("Value which is pointed by ptr pointer : %d",*ptr);

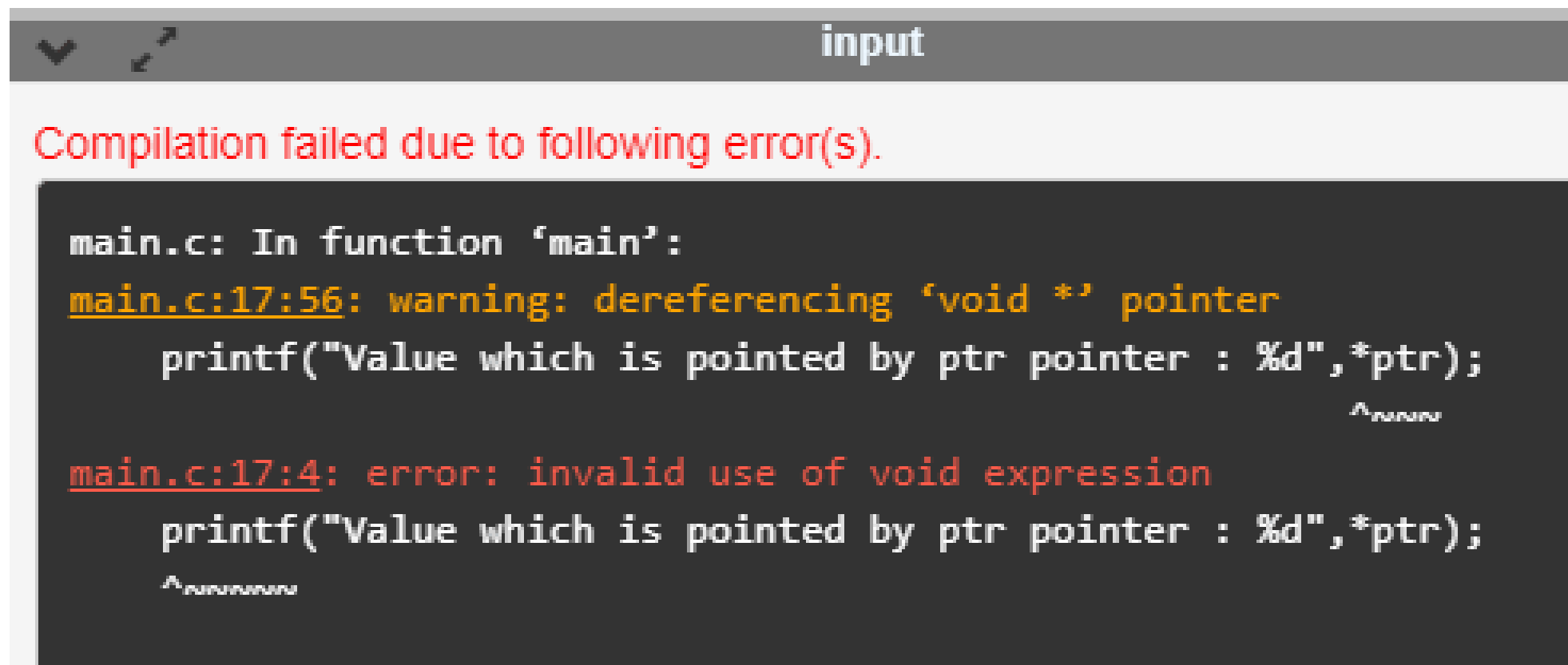
    return 0;

}

```

In the above code, *ptr is a void pointer which is pointing to the integer variable 'a'. As we already know that the void pointer cannot be dereferenced, so the above code will give the compile-time error because we are printing the value of the variable pointed by the pointer 'ptr' directly.

Output



```
input
Compilation failed due to following error(s).
main.c: In function 'main':
main.c:17:56: warning: dereferencing 'void *' pointer
    printf("Value which is pointed by ptr pointer : %d",*ptr);
                                                    ^~~~~~
main.c:17:4: error: invalid use of void expression
    printf("Value which is pointed by ptr pointer : %d",*ptr);
    ^~~~~~
```

Now, we rewrite the above code to remove the error.

```
#include <stdio.h>

int main()
{
    int a=90;

    void *ptr;

    ptr=&a;

    printf("Value which is pointed by ptr pointer : %d",*(int*)ptr);

    return 0;
}
```

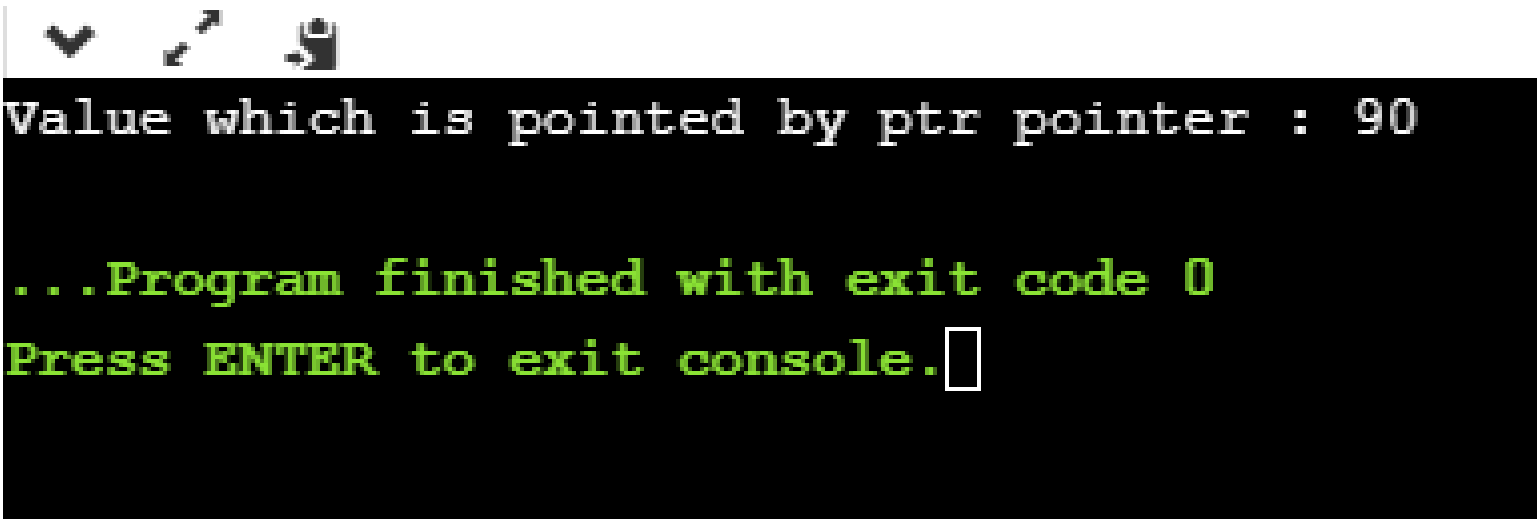
In the above code, we typecast the void pointer to the integer pointer by using the statement given below:

```
(int*)ptr;
```

Then, we print the value of the variable which is pointed by the void pointer 'ptr' by using the statement given below:

```
*(int*)ptr;
```

Output

A screenshot of a terminal window with a black background and white and green text. The text shows the output of a program: 'Value which is pointed by ptr pointer : 90', followed by '...Program finished with exit code 0' and 'Press ENTER to exit console.' with a cursor. There are some small icons in the top left corner of the terminal window.

```
Value which is pointed by ptr pointer : 90
...Program finished with exit code 0
Press ENTER to exit console.
```

○ Arithmetic operation on void pointers

We cannot apply the arithmetic operations on void pointers in C directly. We need to apply the proper typecasting so that we can perform the arithmetic operations on the void pointers.

Let's see the below example:

```
#include<stdio.h>

int main()
{
    float a[4]={6.1,2.3,7.8,9.0};

    void *ptr;

    ptr=a;

    for(int i=0;i<4;i++)
    {
```

```

printf("%f", *ptr);

ptr=ptr+1;    // Incorrect.

}}

```

The above code shows the compile-time error that "invalid use of void expression" as we cannot apply the arithmetic operations on void pointer directly, i.e., ptr=ptr+1.

Let's rewrite the above code to remove the error.

```

#include<stdio.h>

int main()

{

    float a[4]={6.1,2.3,7.8,9.0};

    void *ptr;

    ptr=a;

    for(int i=0;i<4;i++)

    {

        printf("%f",*((float*)ptr+i));

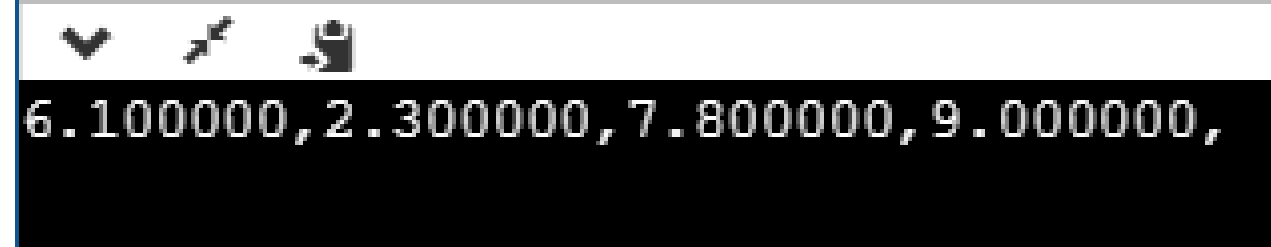
    }

}

```

The above code runs successfully as we applied the proper casting to the void pointer, i.e., (float*)ptr and then we apply the arithmetic operation, i.e., *((float*)ptr+i).

Output



```
6.100000,2.300000,7.800000,9.000000,
```

Why we use void pointers?

We use void pointers because of its reusability. Void pointers can store the object of any type, and we can retrieve the object of any type by using the indirection operator with proper typecasting.

Let's understand through an example.

```
#include<stdio.h>

int main()

{

    int a=56; // initialization of a integer variable 'a'.

    float b=4.5; // initialization of a float variable 'b'.

    char c='k'; // initialization of a char variable 'c'.

    void *ptr; // declaration of void pointer.

    // assigning the address of variable 'a'.

    ptr=&a;

    printf("value of 'a' is : %d",*((int*)ptr));

    // assigning the address of variable 'b'.

    ptr=&b;

    printf("\nvalue of 'b' is : %f",*((float*)ptr));

    // assigning the address of variable 'c'.
```

```

ptr=&c;

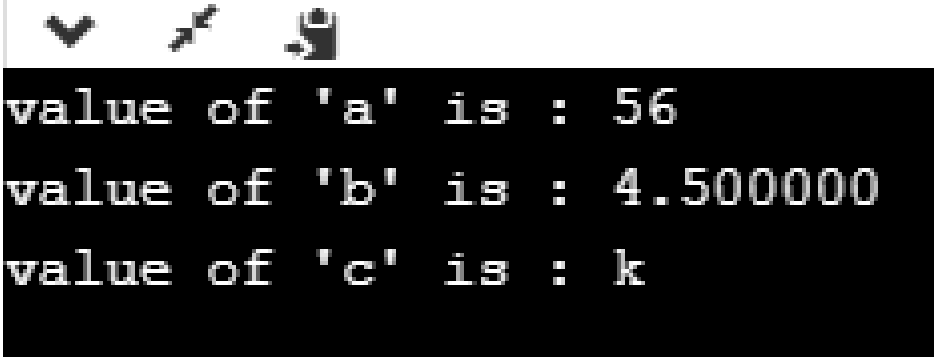
printf("\nvalue of 'c' is : %c",*((char*)ptr));

return 0;

}

```

Output



```

value of 'a' is : 56
value of 'b' is : 4.500000
value of 'c' is : k

```

Null Pointer in C

A Null Pointer is a pointer that does not point to any memory location. It stores the base address of the segment. The null pointer basically stores the Null value while void is the type of the pointer.

A null pointer is a special reserved value which is defined in a **stddef** header file. Here, Null means that the pointer is referring to the 0th memory location.

If we do not have any address which is to be assigned to the pointer, then it is known as a null pointer. When a NULL value is assigned to the pointer, then it is considered as a Null pointer.

Applications of Null Pointer

Following are the applications of a Null pointer:

- It is used to initialize a pointer variable when the pointer does not point to a valid memory address.

- It is used to perform error handling with pointers before dereferencing the pointers.
- It is passed as a function argument and to return from a function when we do not want to pass the actual memory address.

Examples of Null Pointer

- `int *ptr=(int *)0;`
- `float *ptr=(float *)0;`
- `char *ptr=(char *)0;`
- `double *ptr=(double *)0;`
- `char *ptr='\0';`
- `int *ptr=NULL;`

Let's look at the situations where we need to use the null pointer.

- **When we do not assign any memory address to the pointer variable.**

```
#include <stdio.h>

int main()
{
    int *ptr;

    printf("Address: %d", ptr); // printing the value of ptr.

    printf("Value: %d", *ptr); // dereferencing the illegal pointer

    return 0;
}
```

In the above code, we declare the pointer variable `*ptr`, but it does not contain the address of any variable. The dereferencing of the uninitialized pointer variable will show the compile-time error as it does not point any variable. According to the stack memory concept, the local variables of a function are stored in the stack, and if the variable does not contain any value, then it shows the garbage value. The above program shows some unpredictable results and causes the program to crash. Therefore, we can say that keeping an uninitialized pointer in a program can cause serious harm to the computer.

How to avoid the above situation?

We can avoid the above situation by using the Null pointer. A null pointer is a pointer pointing to the 0th memory location, which is a reserved memory and cannot be dereferenced.

```
#include <stdio.h>

int main()
{
    int *ptr=NULL;

    if(ptr!=NULL)
    {
        printf("value of ptr is : %d",*ptr);
    }

    else
    {
        printf("Invalid pointer");
    }
}
```

```

return 0;

}

```

In the above code, we create a pointer `*ptr` and assigns a **NULL** value to the pointer, which means that it does not point any variable. After creating a pointer variable, we add the condition in which we check whether the value of a pointer is null or not.

- **When we use the `malloc()` function.**

```

#include <stdio.h>

int main()
{

    int *ptr;

    ptr=(int*)malloc(4*sizeof(int));

    if(ptr==NULL)

    {

        printf("Memory is not allocated");

    }

    else

    {

        printf("Memory is allocated");

    }

    return 0;

}

```

In the above code, we use the library function, i.e., `malloc()`. As we know, that `malloc()` function allocates the memory; if `malloc()` function is not able to allocate the

memory, then it returns the **NULL** pointer. Therefore, it is necessary to add the condition which will check whether the value of a pointer is null or not, if the value of a pointer is not null means that the **memory is allocated**.

```
main.c:14:15: note: in  
Memory is allocated
```

Note: It is always a good practice that we assign a Null value to the pointer when we do not know the exact address of memory.

C dereference pointer

As we already know that "what is a pointer", a pointer is a variable that stores the address of another variable. The dereference operator is also known as an indirection operator, which is represented by (*). When indirection operator (*) is used with the pointer variable, then it is known as **dereferencing a pointer**. When we dereference a pointer, then the value of the variable pointed by this pointer will be returned.

Why we use dereferencing pointer?

Dereference a pointer is used because of the following reasons:

- It can be used to access or manipulate the data stored at the memory location, which is pointed by the pointer.
- Any operation applied to the dereferenced pointer will directly affect the value of the variable that it points to.

Let's observe the following steps to dereference a pointer.

- First, we declare the integer variable to which the pointer points.

```
int x = 9;
```

- Now, we declare the integer pointer variable.

```
int *ptr;
```

- After the declaration of an integer pointer variable, we store the address of 'x' variable to the pointer variable 'ptr'.

```
ptr = &x;
```

- We can change the value of 'x' variable by dereferencing a pointer 'ptr' as given below:

```
*ptr = 8;
```

The above line changes the value of 'x' variable from 9 to 8 because 'ptr' points to the 'x' location and dereferencing of 'ptr', i.e., *ptr=8 will update the value of x.

Let's combine all the above steps:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int x = 9;
```

```
    int *ptr;
```

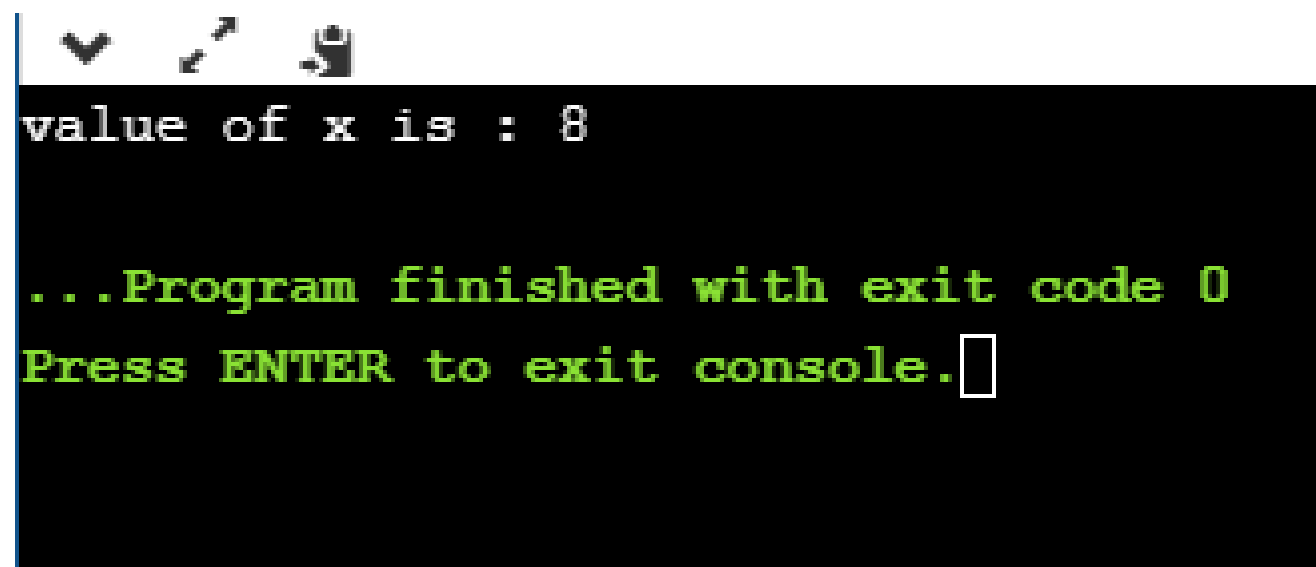
```
    ptr = &x;
```

```
    *ptr = 8;
```

```
    printf("value of x is : %d", x);
```

```
    return 0;}
```

Output

A screenshot of a console window with a black background and white and green text. At the top, there are three small icons: a downward arrow, a magnifying glass, and a document. The text in the console reads: "value of x is : 8" in white, followed by "...Program finished with exit code 0" and "Press ENTER to exit console." in green. A white cursor is visible at the end of the last line.

```
value of x is : 8

...Program finished with exit code 0
Press ENTER to exit console.
```

Let's consider another example.

```
#include <stdio.h>

int main()
{
    int x=4;

    int y;

    int *ptr;

    ptr=&x;

    y=*ptr;

    *ptr=5;

    printf("The value of x is : %d",x);

    printf("\n The value of y is : %d",y);

    return 0;
}
```

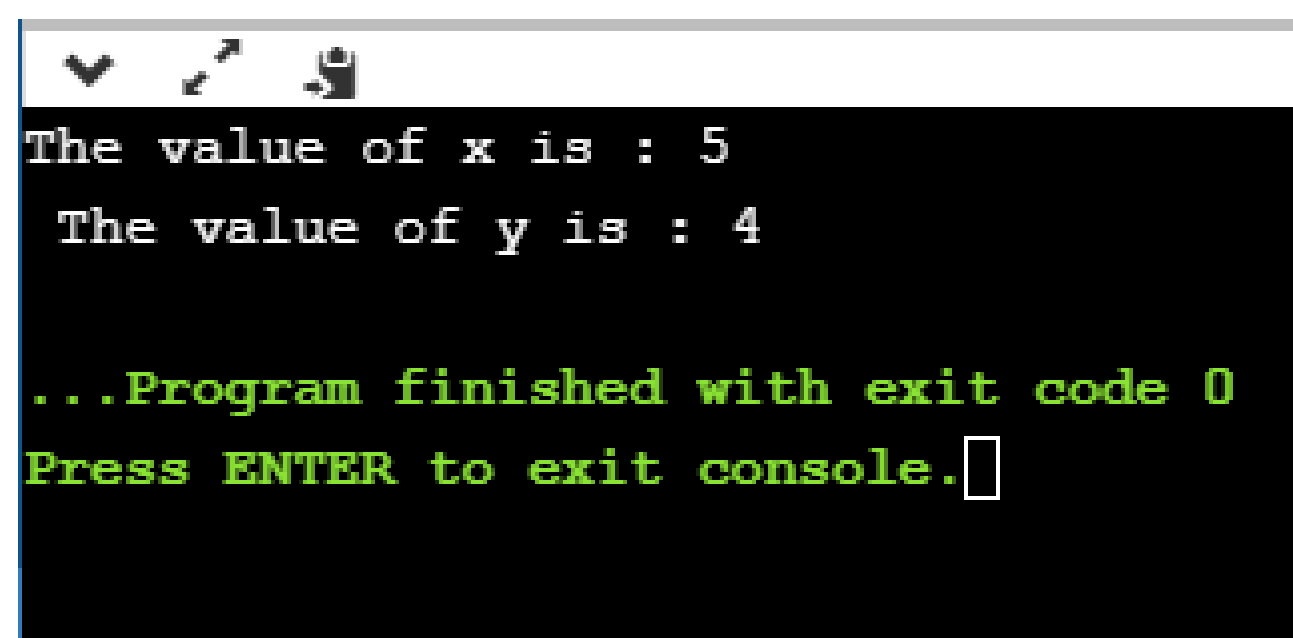
In the above code:

- We declare two variables 'x' and 'y' where 'x' is holding a '4' value.
- We declare a pointer variable 'ptr'.

- After the declaration of a pointer variable, we assign the address of the 'x' variable to the pointer 'ptr'.
- As we know that the 'ptr' contains the address of 'x' variable, so '*ptr' is the same as 'x'.
- We assign the value of 'x' to 'y' with the help of 'ptr' variable, i.e., `y=*ptr` instead of using the 'x' variable.

Note: According to us, if we change the value of 'x', then the value of 'y' will also get changed as the pointer 'ptr' holds the address of the 'x' variable. But this does not happen, as 'y' is storing the local copy of value '5'.

Output



```

The value of x is : 5
The value of y is : 4

...Program finished with exit code 0
Press ENTER to exit console.

```

Let's consider another scenario.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int a=90;
```

```
int *ptr1,*ptr2;
```

```
ptr1=&a;
```

```
ptr2=&a;

*ptr1=7;

*ptr2=6;

printf("The value of a is : %d",a);

return 0;

}
```

In the above code:

- First, we declare an 'a' variable.
- Then we declare two pointers, i.e., ptr1 and ptr2.
- Both the pointers contain the address of 'a' variable.
- We assign the '7' value to the *ptr1 and '6' to the *ptr2. The final value of 'a' would be '6'.

Note: If we have more than one pointer pointing to the same location, then the change made by one pointer will be the same as another pointer.

Output

```
The value of a is : 6
```

C Dynamic Memory

Dynamic memory allocation in C

The concept of **dynamic memory allocation in c language** *enables the C programmer to allocate memory at runtime*. Dynamic memory allocation in c language is possible by 4 functions of stdlib.h header file.

- 1. malloc()
- 2. calloc()
- 3. realloc()
- 4. free()

Before learning above functions, let's understand the difference between static memory allocation and dynamic memory allocation.

static memory allocation	dynamic memory allocation
memory is allocated at compile time.	memory is allocated at run time.
memory can't be increased while executing program.	memory can be increased while executing program.
used in array.	used in linked list.

Now let's have a quick look at the methods used for dynamic memory allocation.

malloc()	allocates single block of requested memory.
calloc()	allocates multiple block of requested memory.

<code>realloc()</code>	reallocates the memory occupied by <code>malloc()</code> or <code>calloc()</code> functions.
<code>free()</code>	frees the dynamically allocated memory.

malloc() function in C

The `malloc()` function allocates single block of requested memory.

It doesn't initialize memory at execution time, so it has garbage value initially.

It returns `NULL` if memory is not sufficient.

The syntax of `malloc()` function is given below:

```
ptr=(cast-type*)malloc(byte-size)
```

Let's see the example of `malloc()` function.

```
#include<stdio.h>

#include<stdlib.h>

int main(){

    int n,i,* ptr,sum=0;

    printf("Enter number of elements: ");

    scanf("%d",&n);

    ptr=(int*)malloc(n*sizeof(int)); //memory allocated using malloc

    if(ptr==NULL)

    {

        printf("Sorry! unable to allocate memory");
```

```

        exit(0);

    }

    printf("Enter elements of array: ");

    for(i=0;i<n;++i)

    {

        scanf("%d",&ptr[i]);

        sum+=*(ptr+i);

    }

    printf("Sum=%d",sum);

    free(ptr);

    return 0;

}

```

Output

```

Enter elements of array: 3

Enter elements of array: 10

10

10

Sum=30

```

calloc() function in C

The calloc() function allocates multiple block of requested memory.

It initially initialize all bytes to zero.

It returns NULL if memory is not sufficient.

The syntax of `calloc()` function is given below:

```
ptr=(cast-type*)calloc(number, byte-size)
```

Let's see the example of `calloc()` function.

```
#include<stdio.h>

#include<stdlib.h>

int main(){

    int n,i,*ptr,sum=0;

    printf("Enter number of elements: ");

    scanf("%d",&n);

    ptr=(int*)calloc(n,sizeof(int)); //memory allocated using calloc

    if(ptr==NULL)

    {

        printf("Sorry! unable to allocate memory");

        exit(0);

    }

    printf("Enter elements of array: ");

    for(i=0;i<n;i++)

    {

        scanf("%d",ptr+i);

        sum+=*(ptr+i);

    }

    printf("Sum=%d",sum);
```

```
    free(ptr);  
  
    return 0;  
  
}
```

Output

```
Enter elements of array: 3  
  
Enter elements of array: 10  
  
10  
  
10  
  
Sum=30
```

realloc() function in C

If memory is not sufficient for malloc() or calloc(), you can reallocate the memory by realloc() function. In short, it changes the memory size.

Let's see the syntax of realloc() function.

```
ptr=realloc(ptr, new-size)
```

free() function in C

The memory occupied by malloc() or calloc() functions must be released by calling free() function. Otherwise, it will consume memory until program exit.

Let's see the syntax of free() function.

```
free(ptr)
```

C Strings

Strings in C

The string can be defined as the one-dimensional array of characters terminated by a null ('\0'). The character array or the string is used to manipulate text such as word or sentences. Each character in the array occupies one byte of memory, and the last character must always be 0. The termination character ('\0') is important in a string since it is the only way to identify where the string ends. When we define a string as `char s[10]`, the character `s[10]` is implicitly initialized with the null in the memory.

There are two ways to declare a string in c language.

1. By char array
2. By string literal

Let's see the example of declaring **string by char array** in C language.

```
char ch[10]={'j','a','v','a','t','p','o','i','n','t','\0'};
```

As we know, array index starts from 0, so it will be represented as in the figure given below.

0	1	2	3	4	5	6	7	8	9	10
j	a	v	a	t	p	o	i	n	t	\0

While declaring string, size is not mandatory. So we can write the above code as given below:

```
char ch[]={'j','a','v','a','t','p','o','i','n','t','\0'};
```

We can also define the **string by the string literal** in C language. For example:

```
char ch[]="javatpoint";
```

In such case, '\0' will be appended at the end of the string by the compiler.

Difference between char array and string literal

There are two main differences between char array and literal.

- We need to add the null character '\0' at the end of the array by ourself whereas, it is appended internally by the compiler in the case of the character array.
- The string literal cannot be reassigned to another set of characters whereas, we can reassign the characters of the array.

String Example in C

Let's see a simple example where a string is declared and being printed. The '%s' is used as a format specifier for the string in c language.

```
#include<stdio.h>

#include <string.h>

int main(){

    char ch[11]={ 'j', 'a', 'v', 'a', 't', 'p', 'o', 'i', 'n', 't', '\0' };

    char ch2[11]="javatpoint";


    printf("Char Array Value is: %s\n", ch);

    printf("String Literal Value is: %s\n", ch2);

    return 0;

}
```

Output

Char Array Value is: javatpoint

String Literal Value is: javatpoint

Traversing String

Traversing the string is one of the most important aspects in any of the programming languages. We may need to manipulate a very large text which can be done by traversing the text. Traversing string is somewhat different from the traversing an integer array. We need to know the length of the array to traverse an integer array, whereas we may use the null character in the case of string to identify the end the string and terminate the loop.

Hence, there are two ways to traverse a string.

- By using the length of string
- By using the null character.

Let's discuss each one of them.

Using the length of string

Let's see an example of counting the number of vowels in a string.

```
#include<stdio.h>

void main ( )

{

    char s[11] = "javatpoint";

    int i = 0;

    int count = 0;

    while(i<11)
```

```

{

    if(s[i]=='a' || s[i] == 'e' || s[i] == 'i' || s[i] == 'u' || s[i] == 'o')

    {

        count ++;

    }

    i++;

}

printf("The number of vowels %d",count);

}

```

Output

The number of vowels 4

Using the null character

Let's see the same example of counting the number of vowels by using the null character.

```

#include<stdio.h>

void main ()

{

    char s[11] = "javatpoint";

    int i = 0;

    int count = 0;

    while(s[i] != NULL)

    {

```

```

        if(s[i]=='a' || s[i] == 'e' || s[i] == 'i' || s[i] == 'u' || s[i] == 'o')
        {
            count ++;
        }

        i++;
    }

    printf("The number of vowels %d",count);
}

```

Output

The number of vowels 4

Accepting string as the input

Till now, we have used scanf to accept the input from the user. However, it can also be used in the case of strings but with a different scenario. Consider the below code which stores the string while space is encountered.

```

#include<stdio.h>

void main ( )
{
    char s[20];

    printf("Enter the string?");

    scanf("%s",s);

    printf("You entered %s",s);
}

```

Output

```
Enter the string?javatpoint is the best
```

```
You entered javatpoint
```

It is clear from the output that, the above code will not work for space separated strings. To make this code working for the space separated strings, the minor change required in the scanf function, i.e., instead of writing `scanf("%s",s)`, we must write: `scanf("%[^\n]s",s)` which instructs the compiler to store the string `s` while the new line (`\n`) is encountered. Let's consider the following example to store the space-separated strings.

```
#include<stdio.h>

void main ()
{
    char s[20];

    printf("Enter the string?");

    scanf("%[^\n]s",s);

    printf("You entered %s",s);
}
```

Output

```
Enter the string?javatpoint is the best
```

```
You entered javatpoint is the best
```

Here we must also notice that we do not need to use address of (`&`) operator in `scanf` to store a string since string `s` is an array of characters and the name of the array, i.e., `s` indicates the base address of the string (character array) therefore we need not use `&` with it.

Some important points

However, there are the following points which must be noticed while entering the strings by using scanf.

- The compiler doesn't perform bounds checking on the character array. Hence, there can be a case where the length of the string can exceed the dimension of the character array which may always overwrite some important data.
- Instead of using scanf, we may use gets() which is an inbuilt function defined in a header file string.h. The gets() is capable of receiving only one string at a time.

Pointers with strings

We have used pointers with the array, functions, and primitive data types so far. However, pointers can be used to point to the strings. There are various advantages of using pointers to point strings. Let us consider the following example to access the string via the pointer.

```
#include<stdio.h>

void main ( )

{

    char s[11] = "javatpoint";

    char *p = s; // pointer p is pointing to string s.

    printf("%s",p); // the string javatpoint is printed if we print p.

}
```

Output

javatpoint

char s[11] = "javatpoint"

Index	0	1	2	3	4	5	6	7	8	9	10
values	j	a	v	a	t	p	o	i	n	t	\0
Address	20	21	22	23	24	25	26	27	28	29	30
Variable	ptr	char *ptr = s									
Value	20										
Address	10										

As we know that string is an array of characters, the pointers can be used in the same way they were used with arrays. In the above example, p is declared as a pointer to the array of characters s. P affects similar to s since s is the base address of the string and treated as a pointer internally. However, we can not change the content of s or copy the content of s into another string directly. For this purpose, we need to use the pointers to store the strings. In the following example, we have shown the use of pointers to copy the content of a string into another.

```
#include<stdio.h>

void main ( )

{

    char *p = "hello javatpoint";

    printf("String p: %s\n",p);

    char *q;

    printf("copying the content of p into q...\n");
```

```
q = p;

printf("String q: %s\n",q);

}
```

Output

```
String p: hello javatpoint

copying the content of p into q...

String q: hello javatpoint
```

Once a string is defined, it cannot be reassigned to another set of characters. However, using pointers, we can assign the set of characters to the string. Consider the following example.

```
#include<stdio.h>

void main ( )

{

    char *p = "hello javatpoint";

    printf("Before assigning: %s\n",p);

    p = "hello";

    printf("After assigning: %s\n",p);

}
```

Output

```
Before assigning: hello javatpoint

After assigning: hello
```

C gets() and puts() Functions

The gets() and puts() are declared in the header file stdio.h. Both the functions are involved in the input/output operations of the strings.

C gets() function

The gets() function enables the user to enter some characters followed by the enter key. All the characters entered by the user get stored in a character array. The null character is added to the array to make it a string. The gets() allows the user to enter the space-separated strings. It returns the string entered by the user.

Declaration

```
char[] gets(char[]);
```

Reading string using gets()

```
#include<stdio.h>

void main ()
{
    char s[30];

    printf("Enter the string? ");

    gets(s);

    printf("You entered %s",s);
}
```

Output

```
Enter the string?
```

```
javatpoint is the best
```

You entered javatpoint is the best

The `gets()` function is risky to use since it doesn't perform any array bound checking and keep reading the characters until the new line (enter) is encountered. It suffers from buffer overflow, which can be avoided by using `fgets()`. The `fgets()` makes sure that not more than the maximum limit of characters are read. Consider the following example.

```
#include<stdio.h>

void main()

{

    char str[20];

    printf("Enter the string? ");

    fgets(str, 20, stdin);

    printf("%s", str);

}
```

Output

```
Enter the string? javatpoint is the best website

javatpoint is the b
```

C puts() function

The `puts()` function is very much similar to `printf()` function. The `puts()` function is used to print the string on the console which is previously read by using `gets()` or `scanf()` function. The `puts()` function returns an integer value representing the number of characters being printed on the console. Since, it prints an additional newline character with the string, which moves the cursor to the new line on the console, the

integer value returned by `puts()` will always be equal to the number of characters present in the string plus 1.

Declaration

```
int puts(char[])
```

Let's see an example to read a string using `gets()` and print it on the console using `puts()`.

```
#include<stdio.h>

#include <string.h>

int main(){

char name[50];

printf("Enter your name: ");

gets(name); //reads string from user

printf("Your name is: ");

puts(name); //displays string

return 0;

}
```

Output:

Enter your name: Sonoo Jaiswal

Your name is: Sonoo Jaiswal

C String Functions

There are many important string functions defined in "string.h" library.

No.	Function	Description
1)	strlen(string_name)	returns the length of string name.
2)	strcpy(destination, source)	copies the contents of source string to destination string.
3)	strcat(first_string, second_string)	concatenates or joins first string with second string. The result of the string is stored in first string.
4)	strcmp(first_string, second_string)	compares the first string with second string. If both strings are same, it returns 0.
5)	strrev(string)	returns reverse string.
6)	strlwr(string)	returns string characters in lowercase.
7)	strupr(string)	returns string characters in uppercase.

C String Length: strlen()

The strlen() function returns the length of the given string. It doesn't count null character '\0'.

```
#include<stdio.h>
```

```
#include <string.h>
```

```

int main(){

char ch[20]={'j','a','v','a','t','p','o','i','n','t','\0'};

printf("Length of string is: %d",strlen(ch));

return 0;

}

```

Output:

Length of string is: 10

C Copy String: strcpy()

The strcpy(destination, source) function copies the source string in destination.

```

#include<stdio.h>

#include <string.h>

int main(){

char ch[20]={'j','a','v','a','t','p','o','i','n','t','\0'};

char ch2[20];

strcpy(ch2,ch);

printf("Value of second string is: %s",ch2);

return 0;

}

```

Output:

Value of second string is: javatpoint

C String Concatenation: strcat()

The `strcat(first_string, second_string)` function concatenates two strings and result is returned to `first_string`.

```
#include<stdio.h>

#include <string.h>

int main(){

    char ch[10]={'h','e','l','l','o','\0'};

    char ch2[10]={'c','\0'};

    strcat(ch,ch2);

    printf("Value of first string is: %s",ch);

    return 0;

}
```

Output:

Value of first string is: helloc

C Compare String: strcmp()

The `strcmp(first_string, second_string)` function compares two string and returns 0 if both strings are equal.

Here, we are using `gets()` function which reads string from the console.

```
#include<stdio.h>

#include <string.h>

int main(){
```

```

char str1[20],str2[20];

printf("Enter 1st string: ");

gets(str1);//reads string from console

printf("Enter 2nd string: ");

gets(str2);

if(strcmp(str1,str2)==0)

    printf("Strings are equal");

else

    printf("Strings are not equal");

return 0;

}

```

Output:

```

Enter 1st string: hello

Enter 2nd string: hello

Strings are equal

```

C Reverse String: strrev()

The strrev(string) function returns reverse of the given string. Let's see a simple example of strrev() function.

```

#include<stdio.h>

#include <string.h>

int main(){

    char str[20];

```

```

printf("Enter string: ");

gets(str); // reads string from console

printf("String is: %s",str);

printf("\nReverse String is: %s",strrev(str));

return 0;

}

```

Output:

```

Enter string: javatpoint

String is: javatpoint

Reverse String is: tnioptavaj

```

C String Lowercase: strlwr()

The `strlwr(string)` function returns string characters in lowercase. Let's see a simple example of `strlwr()` function.

```

#include<stdio.h>

#include <string.h>

int main(){

    char str[20];

    printf("Enter string: ");

    gets(str); // reads string from console

    printf("String is: %s",str);

    printf("\nLower String is: %s",strlwr(str));

    return 0;
}

```

```
}
```

Output:

```
Enter string: JAVATpoint
String is: JAVATpoint
Lower String is: javatpoint
```

C String Uppercase:strupr()

The `strupr(string)` function returns string characters in uppercase. Let's see a simple example of `strupr()` function.

```
#include<stdio.h>

#include <string.h>

int main(){

    char str[20];

    printf("Enter string: ");

    gets(str);//reads string from console

    printf("String is: %s",str);

    printf("\nUpper String is: %s",strupr(str));

    return 0;

}
```

Output:

```
Enter string: javatpoint
String is: javatpoint
Upper String is: JAVATPOINT
```

C String: strstr()

The strstr() function returns pointer to the first occurrence of the matched string in the given string. It is used to return substring from first match till the last character.

Syntax:

```
char *strstr(const char *string, const char *match)
```

String strstr() parameters

string: It represents the full string from where substring will be searched.

match: It represents the substring to be searched in the full string.

String strstr() example

```
#include<stdio.h>

#include <string.h>

int main(){

    char str[100]="this is javatpoint with c and java";

    char *sub;

    sub=strstr(str,"java");

    printf("\nSubstring is: %s",sub);

    return 0;

}
```

Output:

```
javatpoint with c and java
```

C Math Functions

C Math Functions

C Programming allows us to perform mathematical operations through the functions defined in `<math.h>` header file. The `<math.h>` header file contains various methods for performing mathematical operations such as `sqrt()`, `pow()`, `ceil()`, `floor()` etc.

C Math Functions

There are various methods in `math.h` header file. The commonly used functions of `math.h` header file are given below.

No.	Function	Description
1)	<code>ceil(number)</code>	rounds up the given number. It returns the integer value which is greater than or equal to given number.
2)	<code>floor(number)</code>	rounds down the given number. It returns the integer value which is less than or equal to given number.
3)	<code>sqrt(number)</code>	returns the square root of given number.
4)	<code>pow(base, exponent)</code>	returns the power of given number.
5)	<code>abs(number)</code>	returns the absolute value of given number.

C Math Example

Let's see a simple example of math functions found in math.h header file.

```
#include<stdio.h>

#include <math.h>

int main(){

printf("\n%f",ceil(3.6));

printf("\n%f",ceil(3.3));

printf("\n%f",floor(3.6));

printf("\n%f",floor(3.2));

printf("\n%f",sqrt(16));

printf("\n%f",sqrt(7));

printf("\n%f",pow(2,4));

printf("\n%f",pow(3,3));

printf("\n%d",abs(-12));

return 0;

}
```

Output:

4.000000

4.000000

3.000000

3.000000

4.000000

2.645751

16.000000

27.000000

12

C Structure and Union

C Structure

Why use structure?

In C, there are cases where we need to store multiple attributes of an entity. It is not necessary that an entity has all the information of one type only. It can have different attributes of different data types. For example, an entity **Student** may have its name (string), roll number (int), marks (float). To store such type of information regarding an entity student, we have the following approaches:

- Construct individual arrays for storing names, roll numbers, and marks.
- Use a special data structure to store the collection of different data types.

Let's look at the first approach in detail.

```
#include<stdio.h>
```

```
void main ( )
```

```
{
```

```
    char names[2][10],dummy; // 2-dimensioanal character array names is used to store the  
    names of the students
```

```
    int roll_numbers[2],i;
```

```
    float marks[2];
```

```

for (i=0;i<3;i++)

{

printf("Enter the name, roll number, and marks of the student %d",i+1);

scanf("%s %d %f",&names[i],&roll_numbers[i],&marks[i]);

scanf("%c",&dummy); // enter will be stored into dummy character at each iteration

}

printf("Printing the Student details ...\n");

for (i=0;i<3;i++)

{

printf("%s %d %f\n",names[i],roll_numbers[i],marks[i]);

}

}

```

Output

Enter the name, roll number, and marks of the student 1Arun
90 91

Enter the name, roll number, and marks of the student 2Varun
91 56

Enter the name, roll number, and marks of the student 3Sham
89 69

Printing the Student details...

Arun 90 91.000000

Varun 91 56.000000

The above program may fulfill our requirement of storing the information of an entity student. However, the program is very complex, and the complexity increase with the amount of the input. The elements of each of the array are stored contiguously, but all the arrays may not be stored contiguously in the memory. C provides you with an additional and simpler approach where you can use a special data structure, i.e., structure, in which, you can group all the information of different data type regarding an entity.

What is Structure

Structure in c is a user-defined data type that enables us to store the collection of different data types. Each element of a structure is called a member. Structures can simulate the use of classes and templates as it can store various information

The **struct** keyword is used to define the structure. Let's see the syntax to define the structure in c.

```
struct structure_name

{

    data_type member1;

    data_type member2;

    .

    .

    data_type memberN;

};
```

Let's see the example to define a structure for an entity employee in c.

```
struct employee
```

```

{ int id;

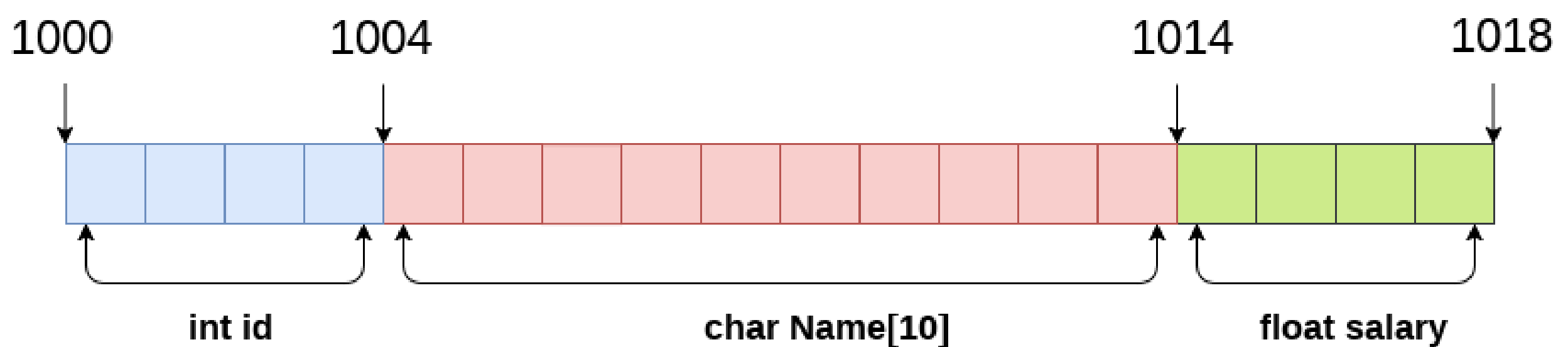
char name[20];

float salary;

};

```

The following image shows the memory allocation of the structure employee that is defined in the above example.



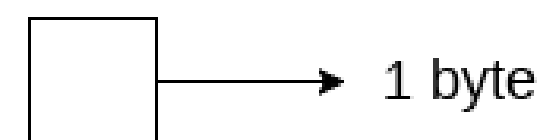
```

struct Employee
{
    int id;
    char Name[10];
    float salary;
} emp;

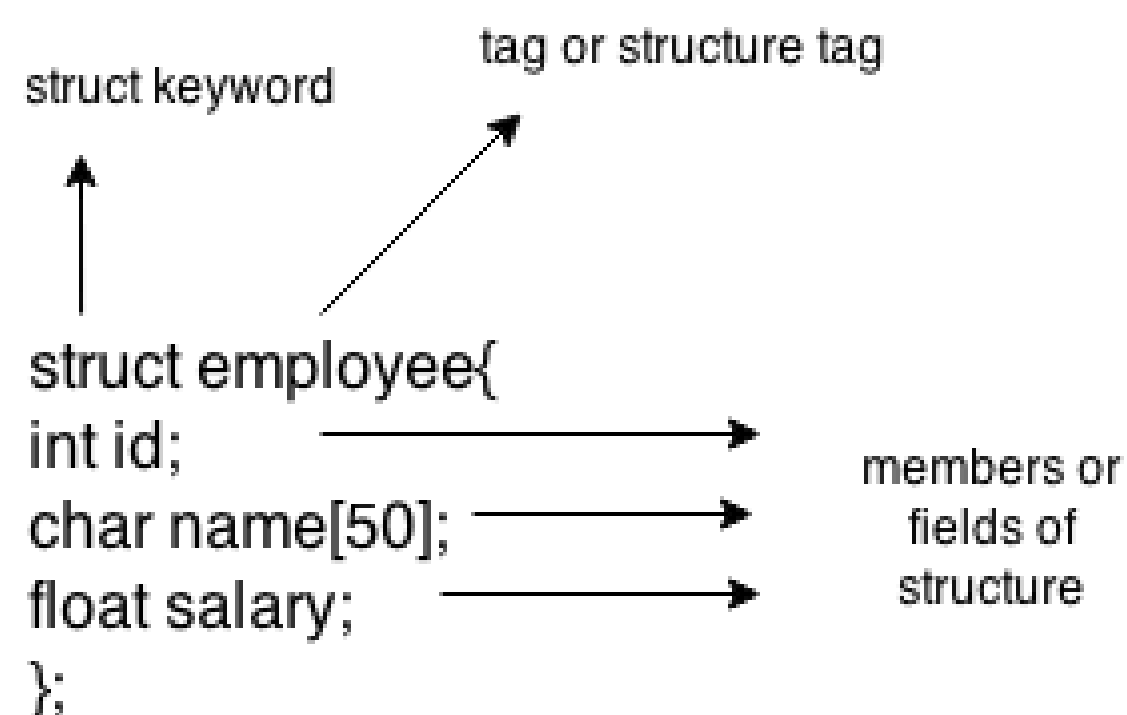
```

sizeof (emp) = 4 + 10 + 4 = 18 bytes

where;
sizeof (int) = 4 byte
sizeof (char) = 1 byte
sizeof (float) = 4 byte



Here, **struct** is the keyword; **employee** is the name of the structure; **id**, **name**, and **salary** are the members or fields of the structure. Let's understand it by the diagram given below:



JavaTpoint.com

Declaring structure variable

We can declare a variable for the structure so that we can access the member of the structure easily. There are two ways to declare structure variable:

1. By struct keyword within main() function
2. By declaring a variable at the time of defining the structure.

1st way:

Let's see the example to declare the structure variable by struct keyword. It should be declared within the main function.

```
struct employee  
  
{ int id;  
  
  char name[50];  
  
  float salary;  
  
};
```

Now write given code inside the main() function.

```
struct employee e1, e2;
```

The variables e1 and e2 can be used to access the values stored in the structure. Here, e1 and e2 can be treated in the same way as the objects in C++ and Java.

2nd way:

Let's see another way to declare variable at the time of defining the structure.

```
struct employee  
  
{ int id;  
  
  char name[50];
```

```
float salary;  
  
}e1,e2;
```

Which approach is good

If number of variables are not fixed, use the 1st approach. It provides you the flexibility to declare the structure variable many times.

If no. of variables are fixed, use 2nd approach. It saves your code to declare a variable in `main()` function.

Accessing members of the structure

There are two ways to access structure members:

1. By `.` (member or dot operator)
2. By `->` (structure pointer operator)

Let's see the code to access the *id* member of *p1* variable by `.` (member) operator.

```
p1.id
```

C Structure example

Let's see a simple example of structure in C language.

```
#include<stdio.h>  
  
#include <string.h>  
  
struct employee  
{ int id;  
  
char name[50];  
  
}e1; //declaring e1 variable for structure
```

```

int main( )

{

    //store first employee information

    e1.id=101;

    strcpy(e1.name, "Sonoo Jaiswal");//copying string into char array

    //printing first employee information

    printf( "employee 1 id : %d\n", e1.id);

    printf( "employee 1 name : %s\n", e1.name);

    return 0;

}

```

Output:

```
employee 1 id : 101
```

```
employee 1 name : Sonoo Jaiswal
```

Let's see another example of the structure in C language to store many employees information.

```

#include<stdio.h>

#include <string.h>

struct employee

{   int id;

    char name[50];

    float salary;

}e1,e2; //declaring e1 and e2 variables for structure

```

```

int main( )

{

    //store first employee information

    e1.id=101;

    strcpy(e1.name, "Sonoo Jaiswal");//copying string into char array

    e1.salary=56000;

    //store second employee information

    e2.id=102;

    strcpy(e2.name, "James Bond");

    e2.salary=126000;

    //printing first employee information

    printf( "employee 1 id : %d\n", e1.id);

    printf( "employee 1 name : %s\n", e1.name);

    printf( "employee 1 salary : %f\n", e1.salary);

    //printing second employee information

    printf( "employee 2 id : %d\n", e2.id);

    printf( "employee 2 name : %s\n", e2.name);

    printf( "employee 2 salary : %f\n", e2.salary);

    return 0;

}

```

Output:

```

employee 1 id : 101

```

employee 1 name : Sonoo Jaiswal

employee 1 salary : 56000.000000

employee 2 id : 102

employee 2 name : James Bond

employee 2 salary : 126000.000000

typedef in C

The **typedef** is a keyword used in C programming to provide some meaningful names to the already existing variable in the C program. It behaves similarly as we define the alias for the commands. In short, we can say that this keyword is used to redefine the name of an already existing variable.

Syntax of typedef

```
typedef <existing_name> <alias_name>
```

In the above syntax, '**existing_name**' is the name of an already existing variable while '**alias name**' is another name given to the existing variable.

For example, suppose we want to create a variable of type **unsigned int**, then it becomes a tedious task if we want to declare multiple variables of this type. To overcome the problem, we use a **typedef** keyword.

```
typedef unsigned int unit;
```

In the above statements, we have declared the **unit** variable of type unsigned int by using a **typedef** keyword.

Now, we can create the variables of type **unsigned int** by writing the following statement:

```
unit a, b;
```

instead of writing:

```
unsigned int a, b;
```

Till now, we have observed that the **typedef** keyword provides a nice shortcut by providing an alternative name for an already existing variable. This keyword is useful when we are dealing with the long data type especially, structure declarations.

Let's understand through a simple example.

```
#include <stdio.h>

int main()

{

    typedef unsigned int unit;

    unit i,j;

    i=10;

    j=20;

    printf("Value of i is :%d",i);

    printf("\nValue of j is :%d",j);

    return 0;

}
```

Output

```
Value of i is :10
```

```
Value of j is :20
```

Using typedef with structures

Consider the below structure declaration:

```
struct student
```

```

{

char name[20];

int age;

};

struct student s1;

```

In the above structure declaration, we have created the variable of **student** type by writing the following statement:

```

struct student s1;

```

The above statement shows the creation of a variable, i.e., s1, but the statement is quite big. To avoid such a big statement, we use the **typedef** keyword to create the variable of type **student**.

```

struct student

{

char name[20];

int age;

};

typedef struct student stud;

stud s1, s2;

```

In the above statement, we have declared the variable **stud** of type struct student. Now, we can use the **stud** variable in a program to create the variables of type **struct student**.

The above typedef can be written as:

```

typedef struct student

{

```

```
char name[20];
```

```
int age;
```

```
} stud;
```

```
stud s1,s2;
```

From the above declarations, we conclude that **typedef** keyword reduces the length of the code and complexity of data types. It also helps in understanding the program.

Let's see another example where we typedef the structure declaration.

```
#include <stdio.h>
```

```
typedef struct student
```

```
{
```

```
char name[20];
```

```
int age;
```

```
}stud;
```

```
int main()
```

```
{
```

```
stud s1;
```

```
printf("Enter the details of student s1: ");
```

```
printf("\nEnter the name of the student:");
```

```
scanf("%s",&s1.name);
```

```
printf("\nEnter the age of student:");
```

```
scanf("%d",&s1.age);
```

```
printf("\n Name of the student is : %s", s1.name);
```

```
printf("\n Age of the student is : %d", s1.age);

return 0;

}
```

Output

```
Enter the details of student s1:

Enter the name of the student: Peter

Enter the age of student: 28

Name of the student is : Peter

Age of the student is : 28
```

Using typedef with pointers

We can also provide another name or alias name to the pointer variables with the help of the typedef.

For example, we normally declare a pointer, as shown below:

```
int* ptr;
```

We can rename the above pointer variable as given below:

```
typedef int* ptr;
```

In the above statement, we have declared the variable of type `int*`. Now, we can create the variable of type `int*` by simply using the '`ptr`' variable as shown in the below statement:

```
ptr p1, p2 ;
```

In the above statement, `p1` and `p2` are the variables of type '`ptr`'.

C Array of Structures

Why use an array of structures?

Consider a case, where we need to store the data of 5 students. We can store it by using the structure as given below.

```
#include<stdio.h>

struct student

{

    char name[20];

    int id;

    float marks;

};

void main()

{

    struct student s1,s2,s3;

    int dummy;

    printf("Enter the name, id, and marks of student 1 ");

    scanf("%s %d %f",s1.name,&s1.id,&s1.marks);

    scanf("%c",&dummy);

    printf("Enter the name, id, and marks of student 2 ");

    scanf("%s %d %f",s2.name,&s2.id,&s2.marks);

    scanf("%c",&dummy);

    printf("Enter the name, id, and marks of student 3 ");
```

```

scanf("%s %d %f",s3.name,&s3.id,&s3.marks);

scanf("%c",&dummy);

printf("Printing the details....\n");

printf("%s %d %f\n",s1.name,s1.id,s1.marks);

printf("%s %d %f\n",s2.name,s2.id,s2.marks);

printf("%s %d %f\n",s3.name,s3.id,s3.marks);

}

```

Output

```

Enter the name, id, and marks of student 1 James 90 90

Enter the name, id, and marks of student 2 Adoms 90 90

Enter the name, id, and marks of student 3 Nick 90 90

Printing the details....

James 90 90.000000

Adoms 90 90.000000

Nick 90 90.000000

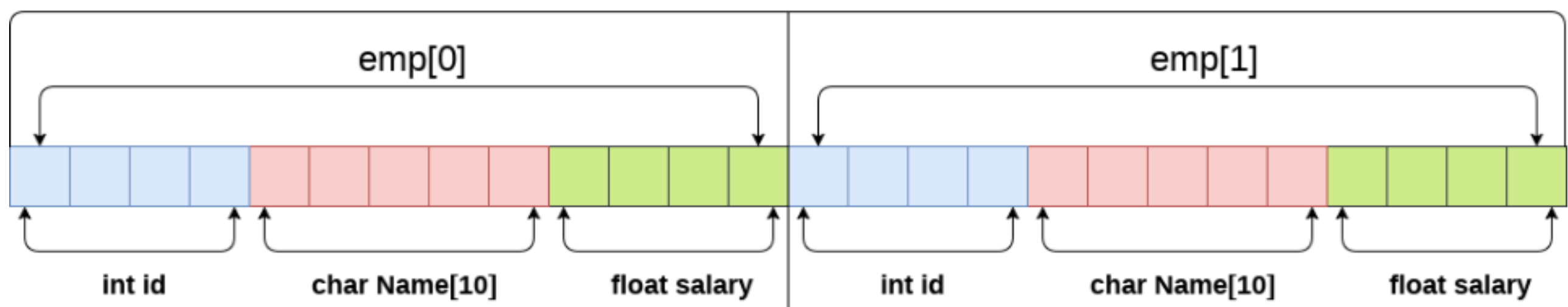
```

In the above program, we have stored data of 3 students in the structure. However, the complexity of the program will be increased if there are 20 students. In that case, we will have to declare 20 different structure variables and store them one by one. This will always be tough since we will have to declare a variable every time we add a student. Remembering the name of all the variables is also a very tricky task. However, C enables us to declare an array of structures by using which, we can avoid declaring the different structure variables; instead we can make a collection containing all the structures that store the information of different entities.

Array of Structures in C

An array of structures in C can be defined as the collection of multiple structures variables where each variable contains information about different entities. The array of structures in C are used to store information about multiple entities of different data types. The array of structures is also known as the collection of structures.

Array of structures



```
struct employee
{
    int id;
    char name[5];
    float salary;
};
struct employee emp[2];
```

`sizeof (emp) = 4 + 5 + 4 = 13 bytes`

`sizeof (emp[2]) = 26 bytes`

Let's see an example of an array of structures that stores information of 5 students and prints it.

```
#include<stdio.h>
```

```
#include <string.h>
```

```
struct student{
```

```
int rollno;
```

```
char name[10];
```

```
};
```

```
int main(){
```

```
int i;
```

```
struct student st[5];

printf("Enter Records of 5 students");

for(i=0;i<5;i++){

printf("\nEnter Rollno:");

scanf("%d",&st[i].rollno);

printf("\nEnter Name:");

scanf("%s",&st[i].name);

}

printf("\nStudent Information List:");

for(i=0;i<5;i++){

printf("\nRollno:%d, Name:%s",st[i].rollno,st[i].name);

}

return 0;

}
```

Output:

Enter Records of 5 students

Enter Rollno:1

Enter Name:Sonoo

Enter Rollno:2

Enter Name:Ratan

Enter Rollno:3

Enter Name:Vimal

Enter Rollno:4

Enter Name:James

Enter Rollno:5

Enter Name:Sarfraz

Student Information List:

Rollno:1, Name:Sonoo

Rollno:2, Name:Ratan

Rollno:3, Name:Vimal

Rollno:4, Name:James

Rollno:5, Name:Sarfraz

Nested Structure in C

C provides us the feature of nesting one structure within another structure by using which, complex data types are created. For example, we may need to store the address of an entity employee in a structure. The attribute address may also have the subparts as street number, city, state, and pin code. Hence, to store the address of the employee, we need to store the address of the employee into a separate structure and nest the structure address into the structure employee. Consider the following program.

```
#include<stdio.h>
```

```
struct address
```

```
{
```

```
    char city[20];
```

```

    int pin;

    char phone[14];

};

struct employee

{

    char name[20];

    struct address add;

};

void main ( )

{

    struct employee emp;

    printf("Enter employee information?\n");

    scanf("%s %s %d %s",emp.name,emp.add.city, &emp.add.pin, emp.add.phone);

    printf("Printing the employee information....\n");

    printf("name: %s\nCity: %s\nPincode: %d\nPhone: %s",emp.name,emp.add.city,emp.a
dd.pin,emp.add.phone);

}

```

Output

Enter employee information?

Arun

Delhi

110001

1234567890

Printing the employee information....

name: Arun

City: Delhi

Pincode: 110001

Phone: 1234567890

The structure can be nested in the following ways.

1. By separate structure
2. By Embedded structure

1) Separate structure

Here, we create two structures, but the dependent structure should be used inside the main structure as a member. Consider the following example.

```
struct Date
```

```
{
```

```
    int dd;
```

```
    int mm;
```

```
    int yyyy;
```

```
};
```

```
struct Employee
```

```
{
```

```
    int id;
```

```
char name[20];

struct Date doj;

}emp1;
```

As you can see, doj (date of joining) is the variable of type Date. Here doj is used as a member in Employee structure. In this way, we can use Date structure in many structures.

2) Embedded structure

The embedded structure enables us to declare the structure inside the structure. Hence, it requires less line of codes but it can not be used in multiple data structures. Consider the following example.

```
struct Employee

{

    int id;

    char name[20];

    struct Date

    {

        int dd;

        int mm;

        int yyyy;

    }doj;

}emp1;
```

Accessing Nested Structure

We can access the member of the nested structure by Outer_Structure.Nested_Structure.member as given below:

```
e1.doj.dd
```

```
e1.doj.mm
```

```
e1.doj.yyyy
```

C Nested Structure example

Let's see a simple example of the nested structure in C language.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
struct Employee
```

```
{
```

```
    int id;
```

```
    char name[20];
```

```
    struct Date
```

```
    {
```

```
        int dd;
```

```
        int mm;
```

```
        int yyyy;
```

```
    }doj;
```

```
}e1;
```

```
int main( )
```

```
{
```

```

//storing employee information

e1.id=101;

strcpy(e1.name, "Sonoo Jaiswal");//copying string into char array

e1.doj.dd=10;

e1.doj.mm=11;

e1.doj.yyyy=2014;

//printing first employee information

printf( "employee id : %d\n", e1.id);

printf( "employee name : %s\n", e1.name);

printf( "employee date of joining (dd/mm/yyyy) : %d/%d/%d\n", e1.doj.dd,e1.doj.mm,e1.
doj.yyyy);

return 0;

}

```

Output:

```

employee id : 101

employee name : Sonoo Jaiswal

employee date of joining (dd/mm/yyyy) : 10/11/2014

```

Passing structure to function

Just like other variables, a structure can also be passed to a function. We may pass the structure members into the function or pass the structure variable at once. Consider the following example to pass the structure variable employee to a function display() which is used to display the details of an employee.

```
#include<stdio.h>
```

```

struct address

{

    char city[20];

    int pin;

    char phone[14];

};

struct employee

{

    char name[20];

    struct address add;

};

void display(struct employee);

void main ( )

{

    struct employee emp;

    printf("Enter employee information?\n");

    scanf("%s %s %d %s",emp.name,emp.add.city, &emp.add.pin, emp.add.phone);

    display(emp);

}

void display(struct employee emp)

{

    printf("Printing the details....\n");

```

```
printf("%s %s %d %s",emp.name,emp.add.city,emp.add.pin,emp.add.phone);  
  
}
```

Structure Padding in C

Structure padding is a concept in C that adds the one or more empty bytes between the memory addresses to align the data in memory.

Let's first understand the structure padding in C through a simple scenario which is given below:

Suppose we create a user-defined structure. When we create an object of this structure, then the contiguous memory will be allocated to the structure members.

```
struct student  
  
{  
  
    char a;  
  
    char b;  
  
    int c;  
  
} stud1;
```

In the above example, we have created a structure of type **student**. We have declared the object of this structure named as '**stud1**'. After the creation of an object, a contiguous block of memory is allocated to its structure members. First, the memory will be allocated to the 'a' variable, then 'b' variable, and then 'c' variable.

What is the size of the struct student?

Now, we calculate the size of the **struct student**. We assume that the size of the int is 4 bytes, and the size of the char is 1 byte.

```
struct student
```

```
{  
  
    char a; // 1 byte  
  
    char b; // 1 byte  
  
    int c; // 4 bytes  
  
}
```

In the above case, when we calculate the size of the **struct student**, size comes to be 6 bytes. But this answer is wrong. Now, we will understand why this answer is wrong? We need to understand the concept of structure padding.

Structure Padding

The processor does not read 1 byte at a time. It reads 1 word at a time.

What does the 1 word mean?

If we have a 32-bit processor, then the processor reads 4 bytes at a time, which means that 1 word is equal to 4 bytes.

1 word = 4 bytes

If we have a 64-bit processor, then the processor reads 8 bytes at a time, which means that 1 word is equal to 8 bytes.

1 word = 8 bytes

Therefore, we can say that a 32-bit processor is capable of accessing 4 bytes at a time, whereas a 64-bit processor is capable of accessing 8 bytes at a time. It depends upon the architecture that what would be the size of the word.

Why structure padding?

struct student

```

{

char a; // 1 byte

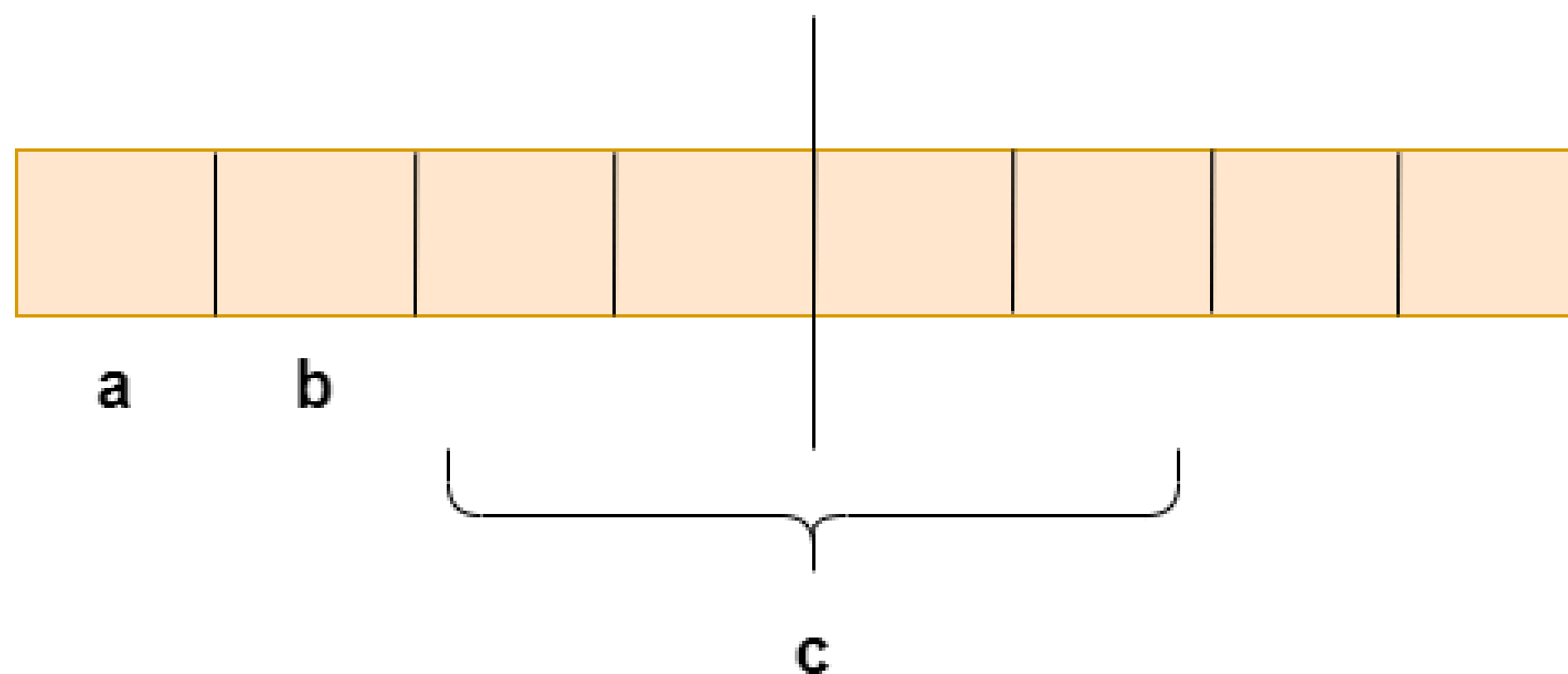
char b; // 1 byte

int c; // 4 bytes

}

```

If we have a 32-bit processor (4 bytes at a time), then the pictorial representation of the memory for the above structure would be:



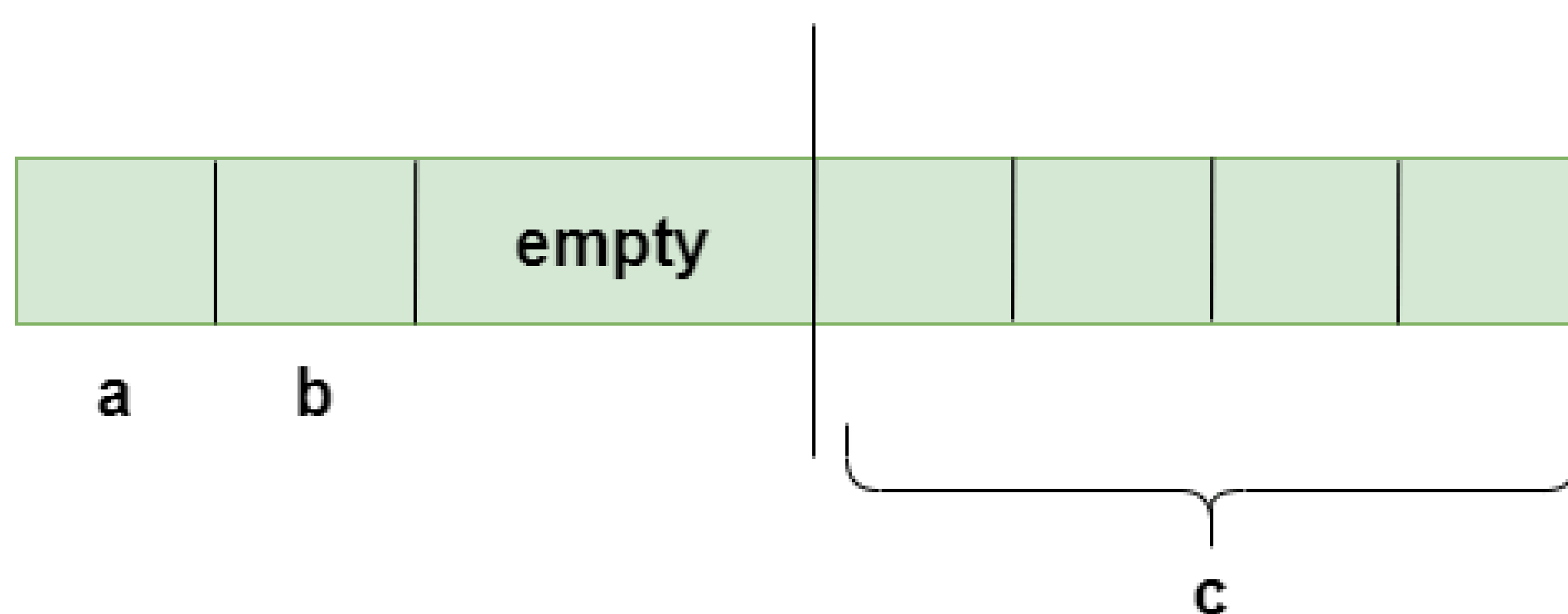
As we know that structure occupies the contiguous block of memory as shown in the above diagram, i.e., 1 byte for char a, 1 byte for char b, and 4 bytes for int c, then what problem do we face in this case.

What's the problem?

The 4-bytes can be accessed at a time as we are considering the 32-bit architecture. The problem is that in one CPU cycle, one byte of **char a**, one byte of **char b**, and 2 bytes of **int c** can be accessed. We will not face any problem while accessing the **char a** and **char b** as both the variables can be accessed in one CPU cycle, but we will face the problem when we access the **int c** variable as 2 CPU cycles are required to access the value of the 'c' variable. In the first CPU cycle, the first two bytes are accessed, and in the second cycle, the other two bytes are accessed.

Suppose we do not want to access the 'a' and 'b' variable, we only want to access the variable 'c', which requires two cycles. The variable 'c' is of 4 bytes, so it can be accessed in one cycle also, but in this scenario, it is utilizing 2 cycles. This is an unnecessary wastage of CPU cycles. Due to this reason, the structure padding concept was introduced to save the number of CPU cycles. The structure padding is done automatically by the compiler. Now, we will see how structure padding is done.

How is structure padding done?



In order to achieve the structure padding, an empty row is created on the left, as shown in the above diagram, and the two bytes which are occupied by the 'c' variable on the left are shifted to the right. So, all the four bytes of 'c' variable are on the right. Now, the 'c' variable can be accessed in a single CPU cycle. After structure padding, the total memory occupied by the structure is 8 bytes (1 byte+1 byte+2 bytes+4 bytes), which is greater than the previous one. Although the memory is wasted in this case, the variable can be accessed within a single cycle.

Let's create a simple program of structures.

```
#include <stdio.h>
```

```
struct student
```

```
{
```

```
    char a;
```

```

    char b;

    int c;

};

int main()

{

    struct student stud1; // variable declaration of the student type..

    // Displaying the size of the structure student.

    printf("The size of the student structure is %d", sizeof(stud1));

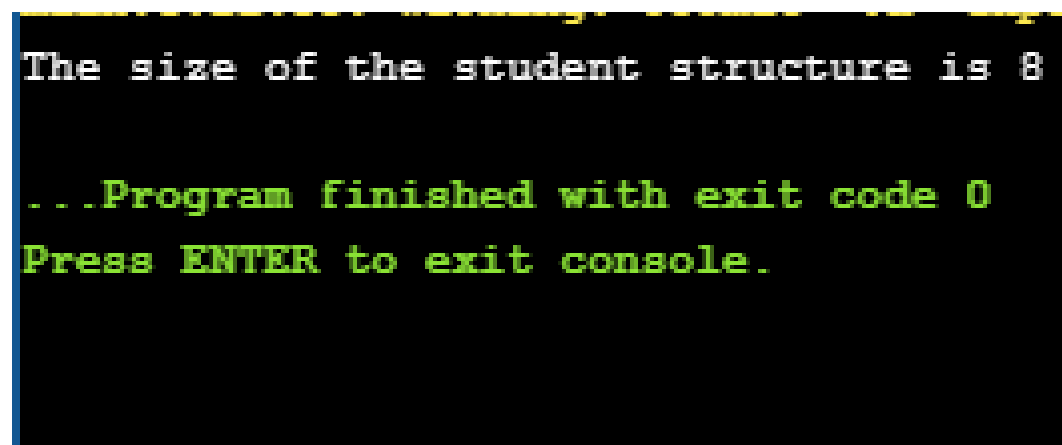
    return 0;

}

```

In the above code, we have created a structure named '**student**'. Inside the **main()** method, we declare a variable of student type, i.e., **stud1**, and then we calculate the size of the student by using the **sizeof()** operator. The output would be **8 bytes** due to the concept of structure padding, which we have already discussed in the above.

Output



```

The size of the student structure is 8

...Program finished with exit code 0
Press ENTER to exit console.

```

Changing order of the variables

Now, we will see what happens when we change the order of the variables, does it affect the output of the program. Let's consider the same program.

```

#include <stdio.h>

struct student
{
    char a;

    int b;

    char c;
};

int main()
{
    struct student stud1; // variable declaration of the student type..

    // Displaying the size of the structure student.

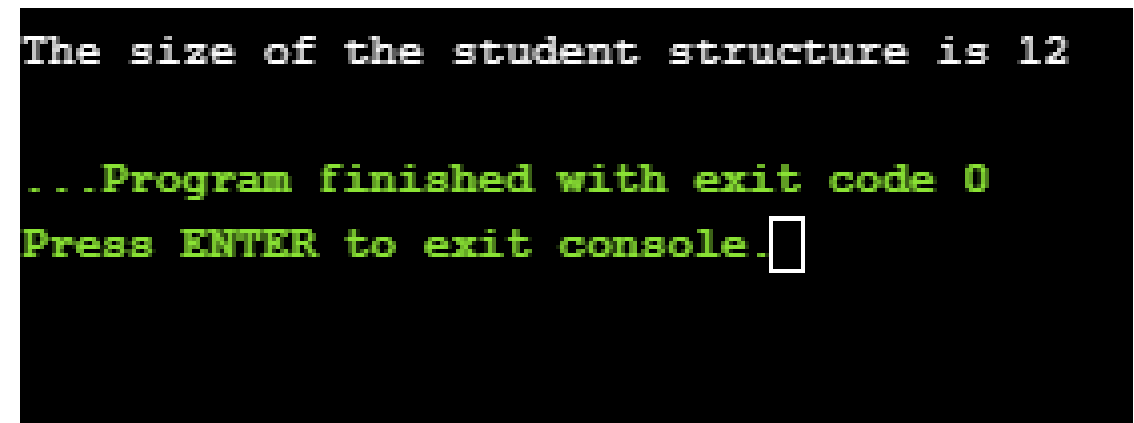
    printf("The size of the student structure is %d", sizeof(stud1));

    return 0;
}

```

The above code is similar to the previous code; the only thing we change is the order of the variables inside the **structure student**. Due to the change in the order, the output would be different in both the cases. In the previous case, the output was 8 bytes, but in this case, the output is 12 bytes, as we can observe in the below screenshot.

Output



```

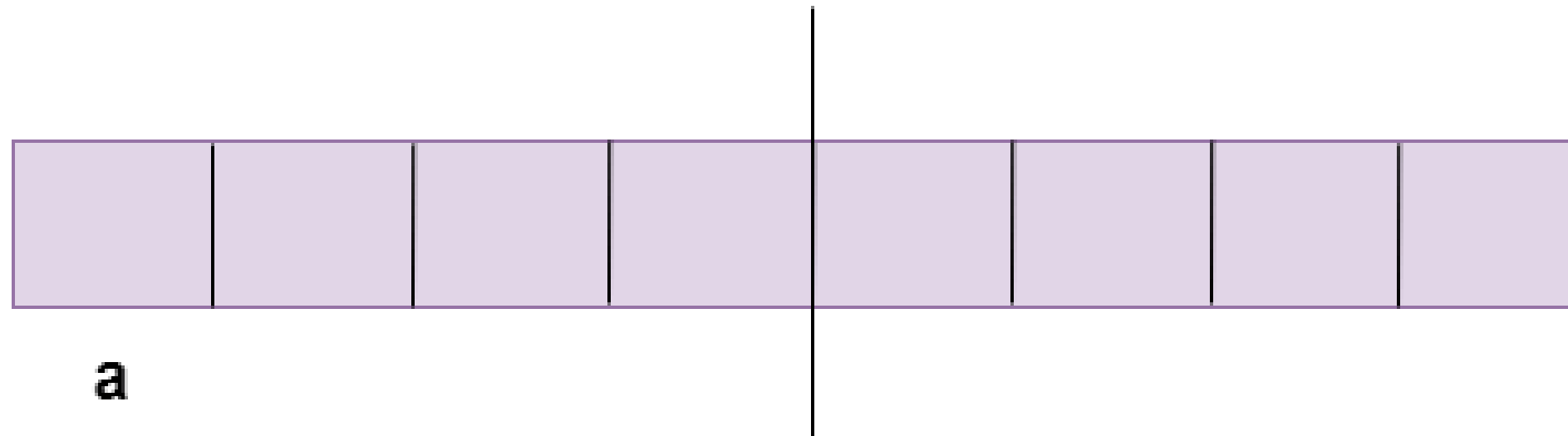
The size of the student structure is 12

...Program finished with exit code 0
Press ENTER to exit console.

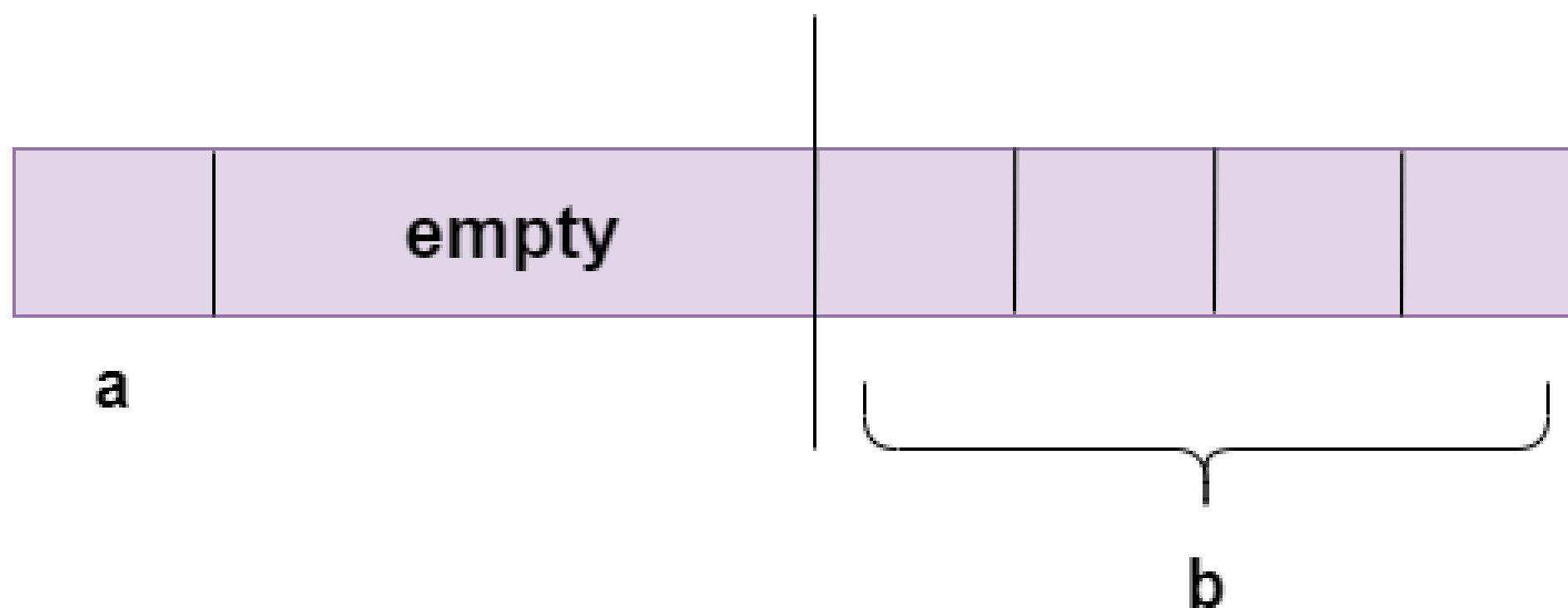
```

Now, we need to understand "why the output is different in this case".

- First, memory is allocated to the **char a** variable, i.e., 1 byte.

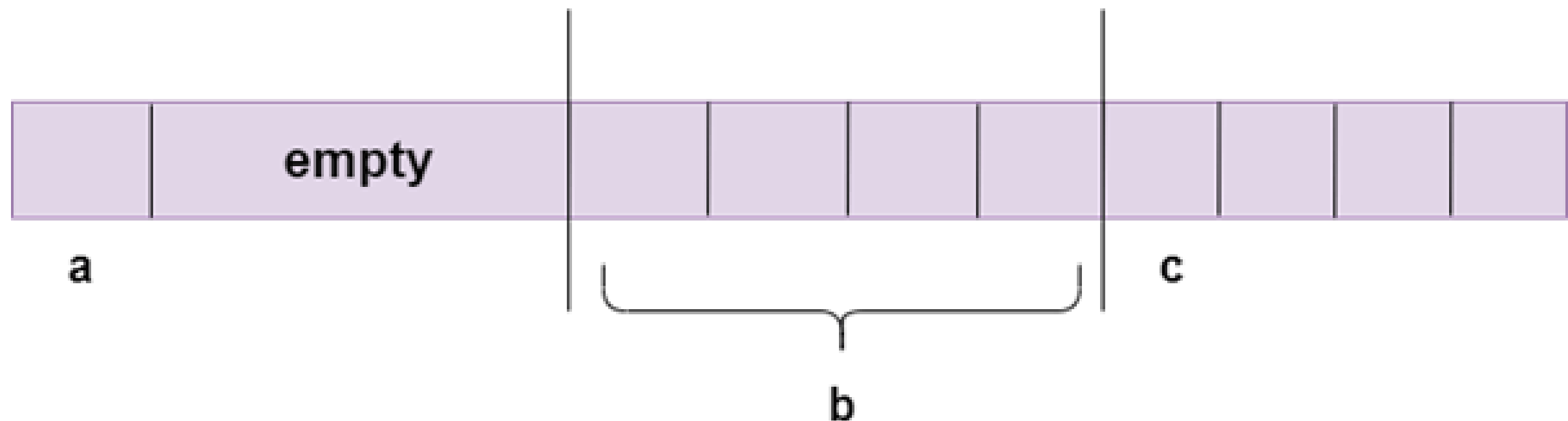


- Now, the memory will be allocated to the **int b**. Since the **int** variable occupies 4 bytes, but on the left, only 3 bytes are available. The empty row will be created on these 3 bytes, and the **int** variable would occupy the other 4 bytes so that the integer variable can be accessed in a single CPU cycle.



- Now, the memory will be given to the **char c**. At a time, CPU can access 1 word, which is equal to 4 bytes, so CPU will use 4 bytes to access a 'c' variable. Therefore, the total memory required is 12 bytes (4 bytes + 4 bytes + 4 bytes), i.

e., 4 bytes required to access **char** **a** variable, 4 bytes required to access **int** **b** variable, and other 4 bytes required to access a single character of 'c' variable.



How to avoid the structure padding in C?

The structural padding is an in-built process that is automatically done by the compiler. Sometimes it required to avoid the structure padding in C as it makes the size of the structure greater than the size of the structure members.

We can avoid the structure padding in C in two ways:

- Using `#pragma pack(1)` directive
- Using attribute

Using `#pragma pack(1)` directive

```
#include <stdio.h>
```

```
#pragma pack(1)
```

```
struct base
```

```
{
```

```
    int a;
```

```
    char b;
```

```

    double c;

};

int main()

{

    struct base var; // variable declaration of type base

    // Displaying the size of the structure base

    printf("The size of the var is : %d", sizeof(var));

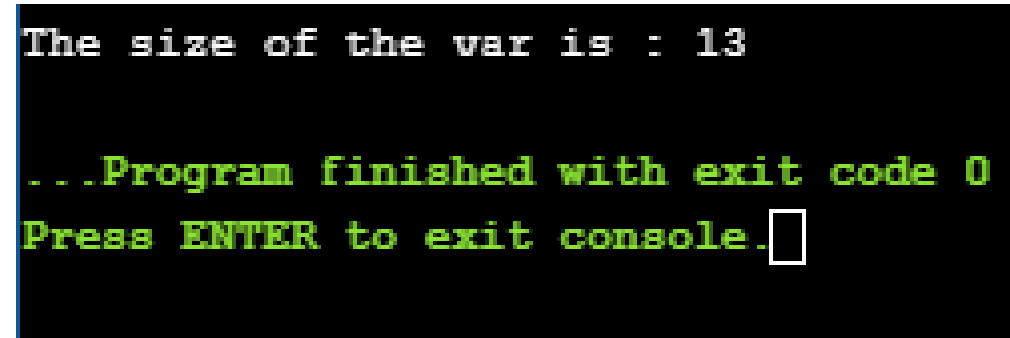
    return 0;

}

```

In the above code, we have used the **#pragma pack(1)** directive to avoid the structure padding. If we do not use this directive, then the output of the above program would be 16 bytes. But the actual size of the structure members is 13 bytes, so 3 bytes are wasted. To avoid the wastage of memory, we use the **#pragma pack(1)** directive to provide the 1-byte packaging.

Output



```

The size of the var is : 13

...Program finished with exit code 0
Press ENTER to exit console.

```

By using attribute

```

#include <stdio.h>

struct base

{

    int a;

```

```

    char b;

    double c;

}__attribute__((packed)); ;

int main()

{

    struct base var; // variable declaration of type base

    // Displaying the size of the structure base

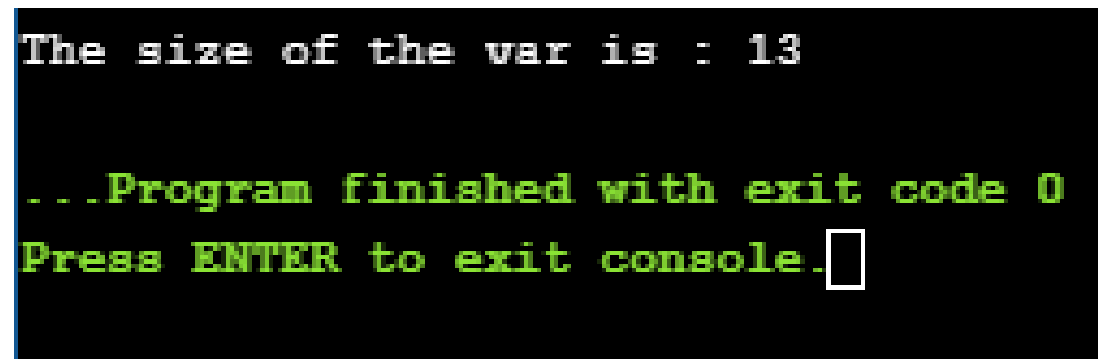
    printf("The size of the var is : %d", sizeof(var));

    return 0;

}

```

Output



```

The size of the var is : 13

...Program finished with exit code 0
Press ENTER to exit console.

```

Union in C

Union can be defined as a user-defined data type which is a collection of different variables of different data types in the same memory location. The union can also be defined as many members, but only one member can contain a value at a particular point in time.

Union is a user-defined data type, but unlike structures, they share the same memory location.

Let's understand this through an example.

```
struct abc  
  
{  
  
    int a;  
  
    char b;  
  
}
```

The above code is the user-defined structure that consists of two members, i.e., 'a' of type **int** and 'b' of type **character**. When we check the addresses of 'a' and 'b', we found that their addresses are different. Therefore, we conclude that the members in the structure do not share the same memory location.

When we define the union, then we found that union is defined in the same way as the structure is defined but the difference is that union keyword is used for defining the union data type, whereas the struct keyword is used for defining the structure. The union contains the data members, i.e., 'a' and 'b', when we check the addresses of both the variables then we found that both have the same addresses. It means that the union members share the same memory location.

Let's have a look at the pictorial representation of the memory allocation.

The below figure shows the pictorial representation of the structure. The structure has two members; i.e., one is of integer type, and the another one is of character type. Since 1 block is equal to 1 byte; therefore, 'a' variable will be allocated 4 blocks of memory while 'b' variable will be allocated 1 block of memory.

The below figure shows the pictorial representation of union members. Both the variables are sharing the same memory location and having the same initial address.

In union, members will share the memory location. If we try to make changes in any of the member then it will be reflected to the other member as well. Let's understand this concept through an example.

```

union abc

{

    int a;

    char b;

}var;

int main()

{

    var.a = 66;

    printf("\n a = %d", var.a);

    printf("\n b = %d", var.b);

}

```

In the above code, union has two members, i.e., 'a' and 'b'. The 'var' is a variable of union abc type. In the `main()` method, we assign the 66 to 'a' variable, so var.a will print 66 on the screen. Since both 'a' and 'b' share the memory location, `var.b` will print 'B' (ascii code of 66).

Deciding the size of the union

The size of the union is based on the size of the largest member of the union.

Let's understand through an example.

```

union abc{

    int a;

    char b;

    float c;
}

```

```

double d;

};

int main()

{

    printf("Size of union abc is %d", sizeof(union abc));

    return 0;

}

```

As we know, the size of int is 4 bytes, size of char is 1 byte, size of float is 4 bytes, and the size of double is 8 bytes. Since the double variable occupies the largest memory among all the four variables, so total 8 bytes will be allocated in the memory. Therefore, the output of the above program would be 8 bytes.

Accessing members of union using pointers

We can access the members of the union through pointers by using the (->) arrow operator.

Let's understand through an example.

```

#include <stdio.h>

union abc

{

    int a;

    char b;

};

int main()

{

```

```

union abc *ptr; // pointer variable declaration

union abc var;

var.a= 90;

ptr = &var;

printf("The value of a is : %d", ptr->a);

return 0;

}

```

In the above code, we have created a pointer variable, i.e., *ptr, that stores the address of var variable. Now, ptr can access the variable 'a' by using the (->) operator. Hence the output of the above code would be 90.

Why do we need C unions?

Consider one example to understand the need for C unions. Let's consider a store that has two items:

- Books
- Shirts

Store owners want to store the records of the above-mentioned two items along with the relevant information. For example, Books include Title, Author, no of pages, price, and Shirts include Color, design, size, and price. The 'price' property is common in both items. The Store owner wants to store the properties, then how he/she will store the records.

Initially, they decided to store the records in a structure as shown below:

```

struct store

{

```

```

double price;

char *title;

char *author;

int number_pages;

int color;

int size;

char *design;

};

```

The above structure consists of all the items that store owner wants to store. The above structure is completely usable but the price is common property in both the items and the rest of the items are individual. The properties like price, *title, *author, and number_pages belong to Books while color, size, *design belongs to Shirt.

Let's see how can we access the members of the structure.

```

int main()

{

    struct store book;

    book.title = "C programming";

    book.author = "Paulo Cohelo";

    book.number_pages = 190;

    book.price = 205;

    printf("Size is : %ld bytes", sizeof(book));

    return 0;

}

```

In the above code, we have created a variable of type **store**. We have assigned the values to the variables, title, author, number_pages, price but the book variable does not possess the properties such as size, color, and design. Hence, it's a wastage of memory. The size of the above structure would be 44 bytes.

We can save lots of space if we use unions.

```
#include <stdio.h>

struct store

{

    double price;

    union

    {

        struct{

            char *title;

            char *author;

            int number_pages;

        } book;

        struct {

            int color;

            int size;

            char *design;

        } shirt;

    } item;

};
```

```

int main()

{

    struct store s;

    s.item.book.title = "C programming";

    s.item.book.author = "John";

    s.item.book.number_pages = 189;

    printf("Size is %ld", sizeof(s));

    return 0;

}

```

In the above code, we have created a variable of type store. Since we used the unions in the above code, so the largest memory occupied by the variable would be considered for the memory allocation. The output of the above program is 32 bytes. In the case of structures, we obtained 44 bytes, while in the case of unions, the size obtained is 44 bytes. Hence, 44 bytes is greater than 32 bytes saving lots of memory space.

C File Handling

File Handling in C

In programming, we may require some specific input data to be generated several numbers of times. Sometimes, it is not enough to only display the data on the console. The data to be displayed may be very large, and only a limited amount of data can be displayed on the console, and since the memory is volatile, it is impossible to recover the programmatically generated data again and again. However, if we need to do so, we may store it onto the local file system which is volatile and can be accessed every time. Here, comes the need of file handling in C.

File handling in C enables us to create, update, read, and delete the files stored on the local file system through our C program. The following operations can be performed on a file.

- Creation of the new file
- Opening an existing file
- Reading from the file
- Writing to the file
- Deleting the file

Functions for file handling

There are many functions in the C library to open, read, write, search and close the file. A list of file functions are given below:

No.	Function	Description
1	fopen()	opens new or existing file
2	fprintf()	write data into the file
3	fscanf()	reads data from the file
4	fputc()	writes a character into the file
5	fgetc()	reads a character from file

6	<code>fclose()</code>	closes the file
7	<code>fseek()</code>	sets the file pointer to given position
8	<code>fputw()</code>	writes an integer to file
9	<code>fgetw()</code>	reads an integer from file
10	<code>ftell()</code>	returns current position
11	<code>rewind()</code>	sets the file pointer to the beginning of the file

Opening File: `fopen()`

We must open a file before it can be read, write, or update. The `fopen()` function is used to open a file. The syntax of the `fopen()` is given below.

```
FILE *fopen( const char * filename, const char * mode
```

The `fopen()` function accepts two parameters:

- The file name (string). If the file is stored at some specific location, then we must mention the path at which the file is stored. For example, a file name can be like `"c://some_folder/some_file.ext"`.
- The mode in which the file is to be opened. It is a string.

We can use one of the following modes in the `fopen()` function.

Mode	Description
------	-------------

r	opens a text file in read mode
w	opens a text file in write mode
a	opens a text file in append mode
r+	opens a text file in read and write mode
w+	opens a text file in read and write mode
a+	opens a text file in read and write mode
rb	opens a binary file in read mode
wb	opens a binary file in write mode
ab	opens a binary file in append mode
rb+	opens a binary file in read and write mode
wb+	opens a binary file in read and write mode
ab+	opens a binary file in read and write mode

The fopen function works in the following way.

- Firstly, It searches the file to be opened.

- Then, it loads the file from the disk and place it into the buffer. The buffer is used to provide efficiency for the read operations.
- It sets up a character pointer which points to the first character of the file.

Consider the following example which opens a file in write mode.

```
#include<stdio.h>

void main( )

{

FILE *fp ;

char ch ;

fp = fopen("file_handle.c","r") ;

while ( 1 )

{

ch = fgetc ( fp ) ;

if ( ch == EOF )

break ;

printf("%c",ch) ;

}

fclose (fp ) ;

}
```

Output

The content of the file will be printed.

```
#include;
```

```

void main( )

{

FILE *fp; // file pointer

char ch;

fp = fopen("file_handle.c","r");

while ( 1 )

{

ch = fgetc ( fp ); //Each character of the file is read and
stored in the character file.

if ( ch == EOF )

break;

printf("%c",ch);

}

fclose (fp );

}

```

Closing File: fclose()

The fclose() function is used to close a file. The file must be closed after performing all the operations on it. The syntax of fclose() function is given below:

```
int fclose( FILE *fp );
```

C fprintf() and fscanf()

Writing File : fprintf() function

The `fprintf()` function is used to write set of characters into file. It sends formatted output to a stream.

Syntax:

```
int fprintf(FILE *stream, const char *format [, argument, ...])
```

Example:

```
#include <stdio.h>

main(){

    FILE *fp;

    fp = fopen("file.txt", "w");//opening file

    fprintf(fp, "Hello file by fprintf...\n");//writing data into file

    fclose(fp);//closing file

}
```

Reading File : fscanf() function

The `fscanf()` function is used to read set of characters from file. It reads a word from the file and returns EOF at the end of file.

Syntax:

```
int fscanf(FILE *stream, const char *format [, argument, ...])
```

Example:

```
#include <stdio.h>

main(){

    FILE *fp;
```

```

char buff[255]; //creating char array to store data of file

fp = fopen("file.txt", "r");

while(fscanf(fp, "%s", buff)!=EOF){

printf("%s ", buff );

}

fclose(fp);

}

```

Output:

Hello file by fprintf...

C File Example: Storing employee information

Let's see a file handling example to store employee information as entered by user from console. We are going to store id, name and salary of the employee.

```

#include <stdio.h>

void main()

{

    FILE *fptr;

    int id;

    char name[30];

    float salary;

    fptr = fopen("emp.txt", "w+"); /* open for writing */

    if (fptr == NULL)

    {

```

```

        printf("File does not exists \n");

        return;

    }

    printf("Enter the id\n");

    scanf("%d", &id);

    fprintf(fp, "Id= %d\n", id);

    printf("Enter the name \n");

    scanf("%s", name);

    fprintf(fp, "Name= %s\n", name);

    printf("Enter the salary\n");

    scanf("%f", &salary);

    fprintf(fp, "Salary= %.2f\n", salary);

    fclose(fp);

}

```

Output:

Enter the id

1

Enter the name

sonoo

Enter the salary

120000

Now open file from current directory. For windows operating system, go to TC\bin directory, you will see emp.txt file. It will have following information.

emp.txt

Id= 1

Name= sonoo

Salary= 120000

C fputc() and fgetc()

Writing File : fputc() function

The fputc() function is used to write a single character into file. It outputs a character to a stream.

Syntax:

```
int fputc(int c, FILE *stream)
```

Example:

```
#include <stdio.h>

main(){

    FILE *fp;

    fp = fopen("file1.txt", "w");//opening file

    fputc('a',fp);//writing single character into file

    fclose(fp);//closing file

}
```

file1.txt

a

Reading File : fgetc() function

The fgetc() function returns a single character from the file. It gets a character from the stream. It returns EOF at the end of file.

Syntax:

```
int fgetc(FILE *stream)
```

Example:

```
#include<stdio.h>

#include<conio.h>

void main(){

FILE *fp;

char c;

clrscr( );

fp=fopen( "myfile.txt","r" );

while( (c=fgetc(fp))!=EOF){

printf( "%c",c);

}

fclose(fp);

getch();

}
```

myfile.txt

```
this is simple text message
```

C fputs() and fgets()

The fputs() and fgets() in C programming are used to write and read string from stream. Let's see examples of writing and reading file using fgets() and fputs() functions.

Writing File : fputs() function

The fputs() function writes a line of characters into file. It outputs string to a stream.

Syntax:

```
int fputs(const char *s, FILE *stream)
```

Example:

```
#include<stdio.h>

#include<conio.h>

void main(){

    FILE *fp;

    clrscr();

    fp=fopen("myfile2.txt","w");

    fputs("hello c programming",fp);

    fclose(fp);

    getch();

}
```

myfile2.txt

```
hello c programming
```

Reading File : fgets() function

The fgets() function reads a line of characters from file. It gets string from a stream.

Syntax:

```
char* fgets(char *s, int n, FILE *stream)
```

Example:

```
#include<stdio.h>

#include<conio.h>

void main(){

FILE *fp;

char text[300];

clrscr();

fp=fopen("myfile2.txt","r");

printf("%s",fgets(text,200,fp));

fclose(fp);

getch();

}
```

Output:

```
hello c programming
```

C fseek()

The fseek() function is used to set the file pointer to the specified offset. It is used to write data into file at desired location.

Syntax:

```
int fseek( FILE *stream, long int offset, int whence)
```

There are 3 constants used in the `fseek()` function for whence: `SEEK_SET`, `SEEK_CUR` and `SEEK_END`.

Example:

```
#include <stdio.h>

void main(){

    FILE *fp;

    fp = fopen("myfile.txt","w+");

    fputs("This is javatpoint", fp);

    fseek( fp, 7, SEEK_SET );

    fputs("sonoo jaiswal", fp);

    fclose(fp);

}
```

myfile.txt

```
This is sonoo jaiswal
```

C rewind()

The `rewind()` function sets the file pointer at the beginning of the stream. It is useful if you have to use stream many times.

Syntax:

```
void rewind(FILE *stream)
```

Example:

File: file.txt

this is a simple text

File: rewind.c

```
#include<stdio.h>

#include<conio.h>

void main(){

FILE *fp;

char c;

clrscr();

fp=fopen("file.txt","r");

while((c=fgetc(fp))!=EOF){

printf("%c",c);

}

rewind(fp); //moves the file pointer at beginning of the file

while((c=fgetc(fp))!=EOF){

printf("%c",c);

}

fclose(fp);

getch();

}
```

Output:

```
this is a simple text this is a simple text
```

As you can see, `rewind()` function moves the file pointer at beginning of the file that is why "this is simple text" is printed 2 times. If you don't call `rewind()` function, "this is simple text" will be printed only once.

C `ftell()`

The `ftell()` function returns the current file position of the specified stream. We can use `ftell()` function to get the total size of a file after moving file pointer at the end of file. We can use `SEEK_END` constant to move the file pointer at the end of file.

Syntax:

```
long int ftell(FILE *stream)
```

Example:

File: `ftell.c`

```
#include <stdio.h>

#include <conio.h>

void main () {

    FILE *fp;

    int length;

    clrscr();

    fp = fopen("file.txt", "r");

    fseek(fp, 0, SEEK_END);

    length = ftell(fp);

    fclose(fp);
```

```
printf("Size of file: %d bytes", length);  
  
getch();  
  
}
```

Output:

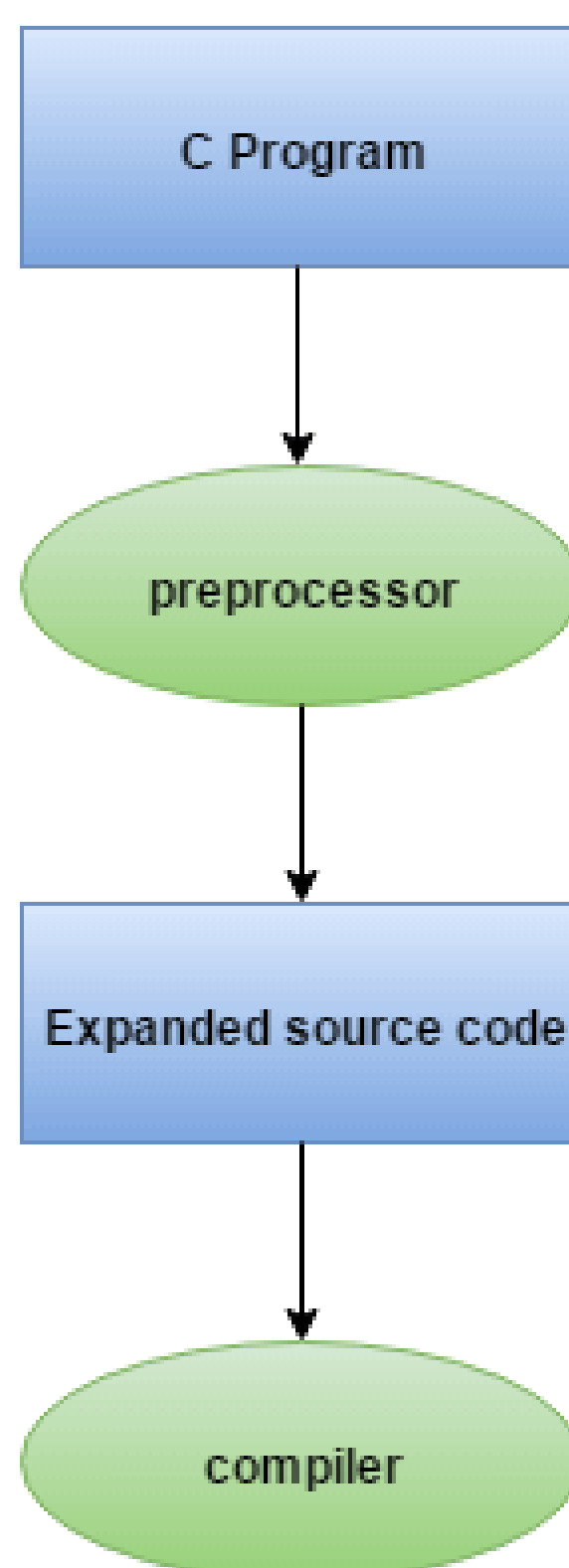
Size of file: 21 bytes

C Preprocessor

C Preprocessor Directives

The C preprocessor is a micro processor that is used by compiler to transform your code before compilation. It is called micro preprocessor because it allows us to add macros.

Note: Preprocessor directives are executed before compilation.



All preprocessor directives starts with hash # symbol.

Let's see a list of preprocessor directives.

- #include
- #define
- #undef
- #ifdef
- #ifndef
- #if
- #else
- #elif
- #endif
- #error
- #pragma

C Macros

A macro is a segment of code which is replaced by the value of macro. Macro is defined by #define directive. There are two types of macros:

1. Object-like Macros
2. Function-like Macros

Object-like Macros

The object-like macro is an identifier that is replaced by value. It is widely used to represent numeric constants. For example:

```
#define PI 3.14
```

Here, PI is the macro name which will be replaced by the value 3.14.

Function-like Macros

The function-like macro looks like function call. For example:

```
#define MIN(a,b) ((a)<(b)?(a):(b))
```

Here, MIN is the macro name.

C Predefined Macros

ANSI C defines many predefined macros that can be used in c program.

No.	Macro	Description
1	_DATE_	represents current date in "MMM DD YYYY" format.
2	_TIME_	represents current time in "HH:MM:SS" format.
3	_FILE_	represents current file name.
4	_LINE_	represents current line number.
5	_STDC_	It is defined as 1 when compiler complies with the ANSI standard.

C predefined macros example

File: simple.c

```
#include<stdio.h>

int main(){

    printf("File :%s\n", __FILE__ );

    printf("Date :%s\n", __DATE__ );

    printf("Time :%s\n", __TIME__ );

    printf("Line :%d\n", __LINE__ );

    printf("STDC :%d\n", __STDC__ );

    return 0;

}
```

Output:

```
File :simple.c

Date :Dec 6 2015

Time :12:28:46

Line :6

STDC :1
```

C #include

The #include preprocessor directive is used to paste code of given file into current file. It is used include system-defined and user-defined header files. If included file is not found, compiler renders error.

By the use of `#include` directive, we provide information to the preprocessor where to look for the header files. There are two variants to use `#include` directive.

1. `#include <filename>`

2. `#include "filename"`

The `#include <filename>` tells the compiler to look for the directory where system header files are held. In UNIX, it is `\usr\include` directory.

The `#include "filename"` tells the compiler to look in the current directory from where program is running.

#include directive example

Let's see a simple example of `#include` directive. In this program, we are including `stdio.h` file because `printf()` function is defined in this file.

```
#include<stdio.h>

int main(){

    printf("Hello C");

    return 0;

}
```

Output:

Hello C

#include notes:

Note 1: In `#include` directive, comments are not recognized. So in case of `#include <a//b>`, `a//b` is treated as filename.

Note 2: In `#include` directive, backslash is considered as normal text not escape sequence. So in case of `#include <a\nb>`, `a\nb` is treated as filename.

Note 3: You can use only comment after filename otherwise it will give error.

C #define

The `#define` preprocessor directive is used to define constant or macro substitution. It can use any basic data type.

Syntax:

```
#define token value
```

Let's see an example of `#define` to define a constant.

```
#include <stdio.h>

#define PI 3.14

main() {

    printf("%f",PI);

}
```

Output:

```
3.140000
```

Let's see an example of `#define` to create a macro.

```
#include <stdio.h>

#define MIN(a,b) ((a)<(b)?(a):(b))

void main() {

    printf("Minimum between 10 and 20 is: %d\n", MIN(10,20));
```

```
}
```

Output:

```
Minimum between 10 and 20 is: 10
```

C #undef

The `#undef` preprocessor directive is used to undefine the constant or macro defined by `#define`.

Syntax:

```
#undef token
```

Let's see a simple example to define and undefine a constant.

```
#include <stdio.h>

#define PI 3.14

#undef PI

main() {

    printf("%f",PI);

}
```

Output:

```
Compile Time Error: 'PI' undeclared
```

The `#undef` directive is used to define the preprocessor constant to a limited scope so that you can declare constant again.

Let's see an example where we are defining and undefining number variable. But before being undefined, it was used by square variable.

```
#include <stdio.h>
```

```
#define number 15

int square=number* number;

#undef number

main( ) {

    printf("%d",square);

}
```

Output:

225

C #ifdef

The `#ifdef` preprocessor directive checks if macro is defined by `#define`. If yes, it executes the code otherwise `#else` code is executed, if present.

Syntax:

```
#ifdef MACRO

//code

#endif
```

Syntax with `#else`:

```
#ifdef MACRO

//successful code

#else

//else code

#endif
```

C #ifdef example

Let's see a simple example to use #ifdef preprocessor directive.

```
#include <stdio.h>

#include <conio.h>

#define NOINPUT

void main() {

    int a=0;

    #ifdef NOINPUT

    a=2;

    #else

    printf("Enter a:");

    scanf("%d", &a);

    #endif

    printf("Value of a: %d\n", a);

    getch();

}
```

Output:

Value of a: 2

But, if you don't define NOINPUT, it will ask user to enter a number.

```
#include <stdio.h>

#include <conio.h>

void main() {
```

```

int a=0;

#ifdef NOINPUT

a=2;

#else

printf("Enter a:");

scanf("%d", &a);

#endif

printf("Value of a: %d\n", a);

getch();

}

```

Output:

Enter a:5

Value of a: 5

C #ifndef

The `#ifndef` preprocessor directive checks if macro is not defined by `#define`. If yes, it executes the code otherwise `#else` code is executed, if present.

Syntax:

```

#ifndef MACRO

//code

#endif

```

Syntax with #else:

```
#ifndef MACRO

//successful code

#else

//else code

#endif
```

C #ifndef example

Let's see a simple example to use `#ifndef` preprocessor directive.

```
#include <stdio.h>

#include <conio.h>

#define INPUT

void main() {

    int a=0;

    #ifndef INPUT

        a=2;

    #else

        printf("Enter a:");

        scanf("%d", &a);

    #endif

    printf("Value of a: %d\n", a);

    getch();

}
```

Output:

Enter a:5

Value of a: 5

But, if you don't define INPUT, it will execute the code of #ifndef.

```
#include <stdio.h>

#include <conio.h>

void main() {

    int a=0;

    #ifndef INPUT

    a=2;

    #else

    printf("Enter a:");

    scanf("%d", &a);

    #endif

    printf("Value of a: %d\n", a);

    getch();

}
```

Output:

Value of a: 2

C #if

The #if preprocessor directive evaluates the expression or condition. If condition is true, it executes the code otherwise #elseif or #else or #endif code is executed.

Syntax:

```
#if expression
```

```
//code
```

```
#endif
```

Syntax with #else:

```
#if expression
```

```
//if code
```

```
#else
```

```
//else code
```

```
#endif
```

Syntax with #elif and #else:

```
#if expression
```

```
//if code
```

```
#elif expression
```

```
//elif code
```

```
#else
```

```
//else code
```

```
#endif
```

C #if example

Let's see a simple example to use #if preprocessor directive.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#define NUMBER 0
```

```
void main() {  
  
    #if (NUMBER==0)  
  
        printf("Value of Number is: %d",NUMBER);  
  
    #endif  
  
    getch();  
  
}
```

Output:

Value of Number is: 0

Let's see another example to understand the #if directive clearly.

```
#include <stdio.h>  
  
#include <conio.h>  
  
#define NUMBER 1  
  
void main() {  
  
    clrscr();  
  
    #if (NUMBER==0)  
  
        printf("1 Value of Number is: %d",NUMBER);  
  
    #endif  
  
    #if (NUMBER==1)  
  
        printf("2 Value of Number is: %d",NUMBER);  
  
    #endif  
  
    getch();  
  
}
```

Output:

```
2 Value of Number is: 1
```

C #else

The `#else` preprocessor directive evaluates the expression or condition if condition of `#if` is false. It can be used with `#if`, `#elif`, `#ifdef` and `#ifndef` directives.

Syntax:

```
#if expression
```

```
//if code
```

```
#else
```

```
//else code
```

```
#endif
```

Syntax with #elif:

```
#if expression
```

```
//if code
```

```
#elif expression
```

```
//elif code
```

```
#else
```

```
//else code
```

```
#endif
```

C #else example

Let's see a simple example to use `#else` preprocessor directive.

```

#include <stdio.h>

#include <conio.h>

#define NUMBER 1

void main() {

    #if NUMBER==0

    printf("Value of Number is: %d",NUMBER);

    #else

    print("Value of Number is non-zero");

    #endif

    getch();

}

```

Output:

Value of Number is non-zero

C #error

The `#error` preprocessor directive indicates error. The compiler gives fatal error if `#error` directive is found and skips further compilation process.

C #error example

Let's see a simple example to use `#error` preprocessor directive.

```

#include<stdio.h>

#ifdef __MATH_H

#error First include then compile

```

```

#else

void main(){

    float a;

    a=sqrt(7);

    printf("%f",a);

}

#endif

```

Output:

Compile Time Error: First include then compile

But, if you include math.h, it does not gives error.

```

#include<stdio.h>

#include<math.h>

#ifndef __MATH_H

#error First include then compile

#else

void main(){

    float a;

    a=sqrt(7);

    printf("%f",a);

}

#endif

```

Output:

2.645751

C #pragma

The #pragma preprocessor directive is used to provide additional information to the compiler. The #pragma directive is used by the compiler to offer machine or operating-system feature.

Syntax:

#pragma token

Different compilers can provide different usage of #pragma directive.

The turbo C++ compiler supports following #pragma directives.

#pragma argsused

#pragma exit

#pragma hdrfile

#pragma hdrstop

#pragma inline

#pragma option

#pragma savereg

#pragma startup

#pragma warn

Let's see a simple example to use #pragma preprocessor directive.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void func() ;
```

```
#pragma startup func
```

```
#pragma exit func
```

```
void main(){
```

```
printf("\nI am in main");
```

```
getch();
```

```
}
```

```
void func(){
```

```
printf("\nI am in func");
```

```
getch();
```

```
}
```

Output:

```
I am in func
```

```
I am in main
```

```
I am in func
```

C Command Line

Command Line Arguments in C

The arguments passed from command line are called command line arguments. These arguments are handled by `main()` function.

To support command line argument, you need to change the structure of `main()` function as given below.

```
int main(int argc, char *argv[] )
```

Here, **argc** counts the number of arguments. It counts the file name as the first argument.

The **argv[]** contains the total number of arguments. The first argument is the file name always.

Example

Let's see the example of command line arguments where we are passing one argument with file name.

```
#include <stdio.h>

void main(int argc, char *argv[] ) {

    printf("Program name is: %s\n", argv[0]);

    if(argc < 2){

        printf("No argument passed through command line.\n");

    }

    else{

        printf("First argument is: %s\n", argv[1]);

    }

}
```

Run this program as follows in Linux:

```
./program hello
```

Run this program as follows in Windows from command line:

```
program.exe hello
```

Output:

Program name is: program

First argument is: hello

If you pass many arguments, it will print only one.

```
./program hello c how r u
```

Output:

Program name is: program

First argument is: hello

But if you pass many arguments within double quote, all arguments will be treated as a single argument only.

```
./program "hello c how r u"
```

Output:

Program name is: program

First argument is: hello c how r u

You can write your program to print all the arguments. In this program, we are printing only `argv[1]`, that is why it is printing only one argument.

C Programs

Top 10+ C Programs

C programs are frequently asked in the interview. These programs can be asked from basics, array, string, pointer, linked list, file handling etc. Let's see the list of c programs.

1) Fibonacci Series

Write a c program to print fibonacci series without using recursion and using recursion.

Input: 10

Output: 0 1 1 2 3 5 8 13 21 34

2) Prime number

Write a c program to check prime number.

Input: 44

Output: not prime number

Input: 7

Output: prime number

3) Palindrome number

Write a c program to check palindrome number.

Input: 329

Output: not palindrome number

Input: 12321

Output: palindrome number

4) Factorial

Write a c program to print factorial of a number.

Input: 5

Output: 120

Input: 6

Output: 720

5) Armstrong number

Write a c program to check armstrong number.

Input: 153

Output: armstrong

Input: 22

Output: not armstrong

6) Sum of Digits

Write a c program to print sum of digits.

Input: 234

Output: 9

Input: 12345

Output: 15

7) Reverse Number

Write a c program to reverse given number.

Input: 123

Output: 321

8) Swap two numbers without using third variable

Write a c program to swap two numbers without using third variable.

Input: a=10 b=20

Output: a=20 b=10

9) Print "hello" without using semicolon

Write a c program to print "hello" without using semicolon

10) Assembly Program in C

Write a c program to add two numbers using assembly code.

11) C Program without main() function

Write a c program to print "Hello" without using main() function.

12) Matrix Multiplication

Write a c program to print multiplication of 2 matrices.

Input:

first matrix elements:

1 1 1

2 2 2

3 3 3

second matrix elements

1 1 1

2 2 2

3 3 3

Output:

multiplication of the matrix:

6 6 6

12 12 12

18 18 18

13) Decimal to Binary

Write a c program to convert decimal number to binary.

Input: 5

Output: 101

Input: 20

Output: 10100

14) Alphabet Triangle

Write a c program to print alphabet triangle.

Output:

A

ABA

ABCBA

ABCD CBA

ABCDEDCBA

15) Number Triangle

Write a c program to print number triangle.

Input: 7

Output:

enter the range= 6

1

121

12321

1234321

123454321

12345654321

16) Fibonacci Triangle

Write a c program to generate fibonacci triangle.

Input: 5

Output:

1

1 1

1 1 2

1 1 2 3

1 1 2 3 5

17) Number in Characters

Write a c program to convert number in characters.

Input: 5

Output: five

Input: 203

Output: two zero three

Fibonacci Series in C

Fibonacci Series in C: In case of fibonacci series, *next number is the sum of previous two numbers* for example 0, 1, 1, 2, 3, 5, 8, 13, 21 etc. The first two numbers of fibonacci series are 0 and 1.

There are two ways to write the fibonacci series program:

- Fibonacci Series without recursion
- Fibonacci Series using recursion

Fibonacci Series in C without recursion

Let's see the fibonacci series program in c without recursion.

```
#include<stdio.h>

int main()

{

    int n1=0,n2=1,n3,i,number;

    printf("Enter the number of elements:");

    scanf("%d",&number);

    printf("\n%d %d",n1,n2);//printing 0 and 1

    for(i=2;i<number;++i)//loop starts from 2 because 0 and 1 are already printed

    {

        n3=n1+n2;
```

```

printf(" %d",n3);

n1=n2;

n2=n3;

}

return 0;

}

```

Output:

Enter the number of elements:15

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Fibonacci Series using recursion in C

Let's see the fibonacci series program in c using recursion.

```

#include<stdio.h>

void printFibonacci(int n){

    static int n1=0,n2=1,n3;

    if(n>0){

        n3 = n1 + n2;

        n1 = n2;

        n2 = n3;

        printf("%d ",n3);

        printFibonacci(n-1);

    }

}

```

```

int main(){

    int n;

    printf("Enter the number of elements: ");

    scanf("%d",&n);

    printf("Fibonacci Series: ");

    printf("%d %d ",0,1);

    printFibonacci(n-2);//n-2 because 2 numbers are already printed

    return 0;

}

```

Output:

Enter the number of elements:15

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Prime Number program in C

Prime number in C: **Prime number** is a number that is greater than 1 and divided by 1 or itself. In other words, prime numbers can't be divided by other numbers than itself or 1. For example 2, 3, 5, 7, 11, 13, 17, 19, 23.... are the prime numbers.

Note: Zero (0) and 1 are not considered as prime numbers. Two (2) is the only one even prime number because all the numbers can be divided by 2.

Let's see the prime number program in C. In this c program, we will take an input from the user and check whether the number is prime or not.

```
#include<stdio.h>
```

```
int main(){
```

```
int n,i,m=0,flag=0;

printf("Enter the number to check prime:");

scanf("%d",&n);

m=n/2;

for(i=2;i<=m;i++)

{

    if(n%i==0)

    {

        printf("Number is not prime");

        flag=1;

        break;

    }

}

if(flag==0)

printf("Number is prime");

return 0;

}
```

Output:

Enter the number to check prime:56

Number is not prime

Enter the number to check prime:23

Number is prime

Palindrome program in C

Palindrome number in c: A **palindrome number** is *a number that is same after reverse*.

For example 121, 34543, 343, 131, 48984 are the palindrome numbers.

Palindrome number algorithm

- Get the number from user
- Hold the number in temporary variable
- Reverse the number
- Compare the temporary number with reversed number
- If both numbers are same, print palindrome number
- Else print not palindrome number

Let's see the palindrome program in C. In this c program, we will get an input from the user and check whether number is palindrome or not.

```
#include<stdio.h>

int main()

{

int n,r,sum=0,temp;

printf("enter the number=");

scanf("%d",&n);

temp=n;
```

```

while(n>0)

{

r=n%10;

sum=(sum*10)+r;

n=n/10;

}

if(temp==sum)

printf("palindrome number ");

else

printf("not palindrome");

return 0;

}

```

Output:

enter the number=151

palindrome number

enter the number=5621

not palindrome number

Factorial Program in C

Factorial Program in C: Factorial of n is the *product of all positive descending integers*.

Factorial of n is denoted by $n!$. For example:

$$5! = 5 * 4 * 3 * 2 * 1 = 120$$

$$3! = 3 * 2 * 1 = 6$$

Here, 5! is pronounced as "5 factorial", it is also called "5 bang" or "5 shriek".

The factorial is normally used in Combinations and Permutations (mathematics).

There are many ways to write the factorial program in c language. Let's see the 2 ways to write the factorial program.

- Factorial Program using loop
- Factorial Program using recursion

Factorial Program using loop

Let's see the factorial Program using loop.

```
#include<stdio.h>

int main()

{

    int i,fact=1,number;

    printf("Enter a number: ");

    scanf("%d",&number);

    for(i=1;i<=number;i++){

        fact=fact*i;

    }

    printf("Factorial of %d is: %d",number,fact);

    return 0;

}
```

Output:

Enter a number: 5

Factorial of 5 is: 120

Factorial Program using recursion in C

Let's see the factorial program in c using recursion.

```
#include<stdio.h>

long factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return(n * factorial(n-1));
}

void main()
{
    int number;
    long fact;

    printf("Enter a number: ");

    scanf("%d", &number);

    fact = factorial(number);

    printf("Factorial of %d is %ld\n", number, fact);

    return 0;
```

```
}
```

Output:

Enter a number: 6

Factorial of 5 is: 720

Armstrong Number in C

Before going to write the c program to check whether the number is Armstrong or not, let's understand what is Armstrong number.

Armstrong number is a number that is equal to the sum of cubes of its digits. For example 0, 1, 153, 370, 371 and 407 are the Armstrong numbers.

Let's try to understand why **153** is an Armstrong number.

$$153 = (1*1*1) + (5*5*5) + (3*3*3)$$

where:

$$(1*1*1)=1$$

$$(5*5*5)=125$$

$$(3*3*3)=27$$

So:

$$1+125+27=153$$

Let's try to understand why **371** is an Armstrong number.

$$371 = (3*3*3) + (7*7*7) + (1*1*1)$$

where:

$$(3*3*3)=27$$

$$(7*7*7)=343$$

$$(1*1*1)=1$$

So:

$$27+343+1=371$$

Let's see the c program to check Armstrong Number in C.

```
#include<stdio.h>

int main()

{

int n,r,sum=0,temp;

printf("enter the number=");

scanf("%d",&n);

temp=n;

while(n>0)

{

r=n%10;

sum=sum+(r*r*r);

n=n/10;

}

if(temp==sum)

printf("armstrong number ");

else

printf("not armstrong number");
```

```
return 0;  
  
}
```

Output:

enter the number=153

armstrong number

enter the number=5

not armstrong number

Sum of digits program in C

C program to sum each digit: We can write the sum of digits program in c language by the help of loop and mathematical operation only.

Sum of digits algorithm

To get sum of each digits by c program, use the following algorithm:

- Step 1: Get number by user
- Step 2: Get the modulus/remainder of the number
- Step 3: sum the remainder of the number
- Step 4: Divide the number by 10
- Step 5: Repeat the step 2 while number is greater than 0.

Let's see the sum of digits program in C.

```
#include<stdio.h>
```

```
int main()  
  
{  
  
int n,sum=0,m;  
  
printf("Enter a number:");  
  
scanf("%d",&n);  
  
while(n>0)  
  
{  
  
m=n%10;  
  
sum=sum+m;  
  
n=n/10;  
  
}  
  
printf("Sum is=%d",sum);  
  
return 0;  
  
}
```

Output:

Enter a number:654

Sum is=15

Enter a number:123

Sum is=6

C Program to reverse number

We can reverse a number in c using loop and arithmetic operators. In this program, we are getting number as input from the user and reversing that number. Let's see a simple c example to reverse a given number.

```
#include<stdio.h>

int main()

{

int n, reverse=0, rem;

printf("Enter a number: ");

scanf("%d", &n);

while(n!=0)

{

rem=n%10;

reverse=reverse*10+rem;

n/=10;

}

printf("Reversed Number: %d",reverse);

return 0;

}
```

Output:

Enter a number: 123

Reversed Number: 321

C Program to swap two numbers without third variable

We can swap two numbers without using third variable. There are two common ways to swap two numbers without using third variable:

1. By + and -
2. By * and /

Program 1: Using + and -

Let's see a simple c example to swap two numbers without using third variable.

```
#include<stdio.h>

int main()

{

int a=10, b=20;

printf("Before swap a=%d b=%d",a,b);

a=a+b;//a=30 (10+20)

b=a-b;//b=10 (30-20)

a=a-b;//a=20 (30-10)

printf("\nAfter swap a=%d b=%d",a,b);

return 0;

}
```

Output:

Before swap a=10 b=20

After swap a=20 b=10

Program 2: Using * and /

Let's see another example to swap two numbers using * and /.

```
#include<stdio.h>

#include<stdlib.h>

int main()

{

int a=10, b=20;

printf("Before swap a=%d b=%d",a,b);

a=a*b;//a=200 (10*20)

b=a/b;//b=10 (200/20)

a=a/b;//a=20 (200/10)

system("cls");

printf("\nAfter swap a=%d b=%d",a,b);

return 0;

}
```

Output:

Before swap a=10 b=20

After swap a=20 b=10

C Program to print "hello" without semicolon

We can print "hello" or "hello world" or anything else in C without using semicolon.

There are various ways to do so:

1. Using if
2. Using switch
3. Using loop etc.

Program 1: Using if statement

Let's see a simple c example to print "hello world" using if statement and without using semicolon.

```
#include<stdio.h>

int main()

{

    if(printf("hello world")){}

    return 0;

}
```

Output:

```
hello world
```

Program 2: Using switch statement

Let's see a simple c example to print "hello world" using switch statement and without using semicolon.

```
#include<stdio.h>

int main()

{

    switch(printf("hello world")){}

}
```

```
return 0;

}
```

Output:

```
hello world
```

Program 3: Using while loop

Let's see a simple c example to print "hello world" using while loop and without using semicolon.

```
#include<stdio.h>

int main()

{

    while(!printf("hello world")){}

    return 0;

}
```

Output:

```
hello world
```

Assembly program in C

We can write assembly program code inside c language program. In such case, all the assembly code must be placed inside `asm{}` block.

Let's see a simple assembly program code to add two numbers in c program.

```
#include<stdio.h>

void main() {
```

```

int a = 10, b = 20, c;

asm {

    mov ax,a

    mov bx,b

    add ax,bx

    mov c,ax

}

printf("c= %d",c);

}

```

Output:

c= 30

Note: We have executed this program on TurboC.

C Program without main() function

We can write c program without using main() function. To do so, we need to use #define preprocessor directive.

Let's see a simple program to print "hello" without main() function.

```

#include<stdio.h>

#define start main

void start( ) {

    printf("Hello");

}

```

Output:

Hello

Matrix multiplication in C

Matrix multiplication in C: We can add, subtract, multiply and divide 2 matrices. To do so, we are taking input from the user for row number, column number, first matrix elements and second matrix elements. Then we are performing multiplication on the matrices entered by the user.

In matrix multiplication *first matrix one row element is multiplied by second matrix all column elements.*

Let's try to understand the matrix multiplication of **2*2** and **3*3** matrices by the figure given below:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad B = \begin{pmatrix} 5 & 6 & 7 \\ 8 & 9 & 10 \end{pmatrix}$$

Multiplication of two matrixes:

$$A * B = \begin{pmatrix} 1*5 + 2*8 & 1*6 + 2*9 & 1*7 + 2*10 \\ 3*5 + 4*8 & 3*6 + 4*9 & 3*7 + 4*10 \end{pmatrix}$$

$$A * B = \begin{pmatrix} 21 & 24 & 27 \\ 47 & 54 & 61 \end{pmatrix}$$

JavaTpoint

Let's see the program of matrix multiplication in C.

```

#include<stdio.h>

#include<stdlib.h>

int main(){

int a[10][10],b[10][10],mul[10][10],r,c,i,j,k;

system("cls");

printf("enter the number of row=");

scanf("%d",&r);

printf("enter the number of column=");

scanf("%d",&c);

printf("enter the first matrix element=\n");

for(i=0;i<r;i++)

{

for(j=0;j<c;j++)

{

scanf("%d",&a[i][j]);

}

}

printf("enter the second matrix element=\n");

for(i=0;i<r;i++)

{

for(j=0;j<c;j++)

{

```

```

scanf("%d",&b[i][j]);

}

}

printf("multiply of the matrix=\n");

for(i=0;i<r;i++)

{

for(j=0;j<c;j++)

{

mul[i][j]=0;

for(k=0;k<c;k++)

{

mul[i][j]+=a[i][k]*b[k][j];

}

}

}

//for printing result

for(i=0;i<r;i++)

{

for(j=0;j<c;j++)

{

printf("%d\t",mul[i][j]);

}

}

```

```
printf("\n");  
  
}  
  
return 0;  
  
}
```

Output:

```
enter the number of row=3  
  
enter the number of column=3  
  
enter the first matrix element=  
1 1 1  
  
2 2 2  
  
3 3 3  
  
enter the second matrix element=  
1 1 1  
  
2 2 2  
  
3 3 3  
  
multiply of the matrix=  
  
6 6 6  
  
12 12 12  
  
18 18 18
```

Let's try to understand the matrix multiplication of **3*3** and **3*3** matrices by the figure given below:

$$\text{Matrix 1} \left\{ \begin{array}{ccc} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{array} \right\} \quad \text{Matrix 2} \left\{ \begin{array}{ccc} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{array} \right\}$$

$$\text{Matrix 1} \times \text{Matrix 2} \left\{ \begin{array}{ccc} 1*1+1*2+1*3 & 1*1+1*2+1*3 & 1*1+1*2+1*3 \\ 2*1+2*2+2*3 & 2*1+2*2+2*3 & 2*1+2*2+2*3 \\ 3*1+3*2+3*3 & 3*1+3*2+3*3 & 3*1+3*2+3*3 \end{array} \right\}$$

$$\text{Matrix 1} \times \text{Matrix 2} \left\{ \begin{array}{ccc} 6 & 6 & 6 \\ 12 & 12 & 12 \\ 18 & 18 & 18 \end{array} \right\}$$

JavaTpoint

C Program to convert Decimal to Binary

Decimal to binary in C: We can convert any decimal number (base-10 (0 to 9)) into binary number(base-2 (0 or 1)) by c program.

Decimal Number

Decimal number is a base 10 number because it ranges from 0 to 9, there are total 10 digits between 0 to 9. Any combination of digits is decimal number such as 23, 445, 132, 0, 2 etc.

Binary Number

Binary number is a base 2 number because it is either 0 or 1. Any combination of 0 and 1 is binary number such as 1001, 101, 1111, 101010 etc.

Let's see the some binary numbers for the decimal number.

Decimal	Binary
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010

Decimal to Binary Conversion Algorithm

- Step 1: Divide the number by 2 through % (modulus operator) and store the remainder in array
- Step 2: Divide the number by 2 through / (division operator)

- Step 3: Repeat the step 2 until number is greater than 0

Let's see the c example to convert decimal to binary.

```
#include<stdio.h>

#include<stdlib.h>

int main(){

int a[10],n,i;

system ("cls");

printf("Enter the number to convert: ");

scanf("%d",&n);

for(i=0;n>0;i++)

{

a[i]=n%2;

n=n/2;

}

printf("\nBinary of Given Number is=");

for(i=i-1;i>=0;i--)

{

printf("%d",a[i]);

}

return 0;

}
```

Output:

Enter the number to convert: 5

Binary of Given Number is=101

C Program to print Alphabet Triangle

There are different triangles that can be printed. Triangles can be generated by alphabets or numbers. In this c program, we are going to print alphabet triangles.

Let's see the c example to print alphabet triangle.

```
#include<stdio.h>

#include<stdlib.h>

int main(){

    int ch=65;

    int i,j,k,m;

    system("cls");

    for(i=1;i<=5;i++)

    {

        for(j=5;j>=i;j--)

            printf(" ");

        for(k=1;k<=i;k++)

            printf("%c",ch++);

        ch--;

        for(m=1;m<i;m++)

            printf("%c",--ch);

        printf("\n");
```

```

        ch=65;

    }

    return 0;

}

```

Output:

```

    A

  ABA

ABCBA

ABCDcba

ABCDEDCBA

```

C Program to print Number Triangle

Like alphabet triangle, we can write the c program to print the number triangle. The number triangle can be printed in different ways.

Let's see the c example to print number triangle.

```

#include<stdio.h>

#include<stdlib.h>

int main(){

    int i,j,k,l,n;

    system("cls");

    printf("enter the range=");

    scanf("%d",&n);

    for(i=1;i<=n;i++)

```

```

{

for(j=1;j<=n-i;j++)

{

printf(" ");

}

for(k=1;k<=i;k++)

{

printf("%d",k);

}

for(l=i-1;l>=1;l--)

{

printf("%d",l);

}

printf("\n");

}

return 0;

}

```

Output:

enter the range= 4

1

121

12321

1234321

enter the range= 7

1

121

12321

1234321

123454321

12345654321

1234567654321

C Program to generate Fibonacci Triangle

In this program, we are getting input from the user for the limit for fibonacci triangle, and printing the fibonacci series for the given number of times (limit).

Let's see the c example to generate fibonacci triangle.

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
int main(){
```

```
    int a=0,b=1,i,c,n,j;
```

```
    system("cls");
```

```
    printf("Enter the limit:");
```

```
    scanf("%d",&n);
```

```
    for(i=1;i<=n;i++)
```

```

{

    a=0;

    b=1;

    printf("%d\t",b);

    for(j=1;j<i;j++)

    {

        c=a+b;

        printf("%d\t",c);

        a=b;

        b=c;

    }

    printf("\n");

}

return 0;

}

```

Output:

Enter the limit:9

```

1

1      1

1      1      2

1      1      2      3

1      1      2      3      5

```

1	1	2	3	5	8				
1	1	2	3	5	8	13			
1	1	2	3	5	8	13	21		
1	1	2	3	5	8	13	21	34	

Enter the limit:5

1									
1	1								
1	1	2							
1	1	2	3						
1	1	2	3	5					

C Program to convert Number in Characters

Number in characters conversion: In c language, we can easily convert number in characters by the help of loop and switch case. In this program, we are taking input from the user and iterating this number until it is 0. While iteration, we are dividing it by 10 and the remainder is passed in switch case to get the word for the number.

Let's see the c program to convert number in characters.

```
#include<stdio.h>

#include<stdlib.h>

int main(){

long int n,sum=0,r;

system("cls");

printf("enter the number=");
```

```
scanf("%ld",&n);
```

```
while(n>0)
```

```
{
```

```
    r=n%10;
```

```
    sum=sum*10+r;
```

```
    n=n/10;
```

```
}
```

```
n=sum;
```

```
while(n>0)
```

```
{
```

```
    r=n%10;
```

```
    switch(r)
```

```
    {
```

```
        case 1:
```

```
            printf("one ");
```

```
            break;
```

```
        case 2:
```

```
            printf("two ");
```

```
            break;
```

```
        case 3:
```

```
            printf("three ");
```

```
            break;
```

case 4:

printf("four ");

break;

case 5:

printf("five ");

break;

case 6:

printf("six ");

break;

case 7:

printf("seven ");

break;

case 8:

printf("eight ");

break;

case 9:

printf("nine ");

break;

case 0:

printf("zero ");

break;

default:

```
printf("ttt");  
  
break;  
  
}  
  
n=n/10;  
  
}  
  
return 0;  
  
}
```

Output:

enter the number=4321

four three two one

Hexadecimal to Binary in C

What is hexadecimal?

The hexadecimal is a positional system that represents the numbers in sixteen distinct symbols by using 16 as a base. These distinct symbols, i.e., "0-9" represents the values from zero to nine, and "A-F" represents the values from ten to fifteen.

What is Binary?

The binary is a number which is represented with a base 2 numeral system, and it uses two symbols, i.e., 0 and 1.

Examples

Suppose the hexadecimal number is A12C. Now we calculate the binary value of the hexadecimal number.

Hexadecimal number=A12C

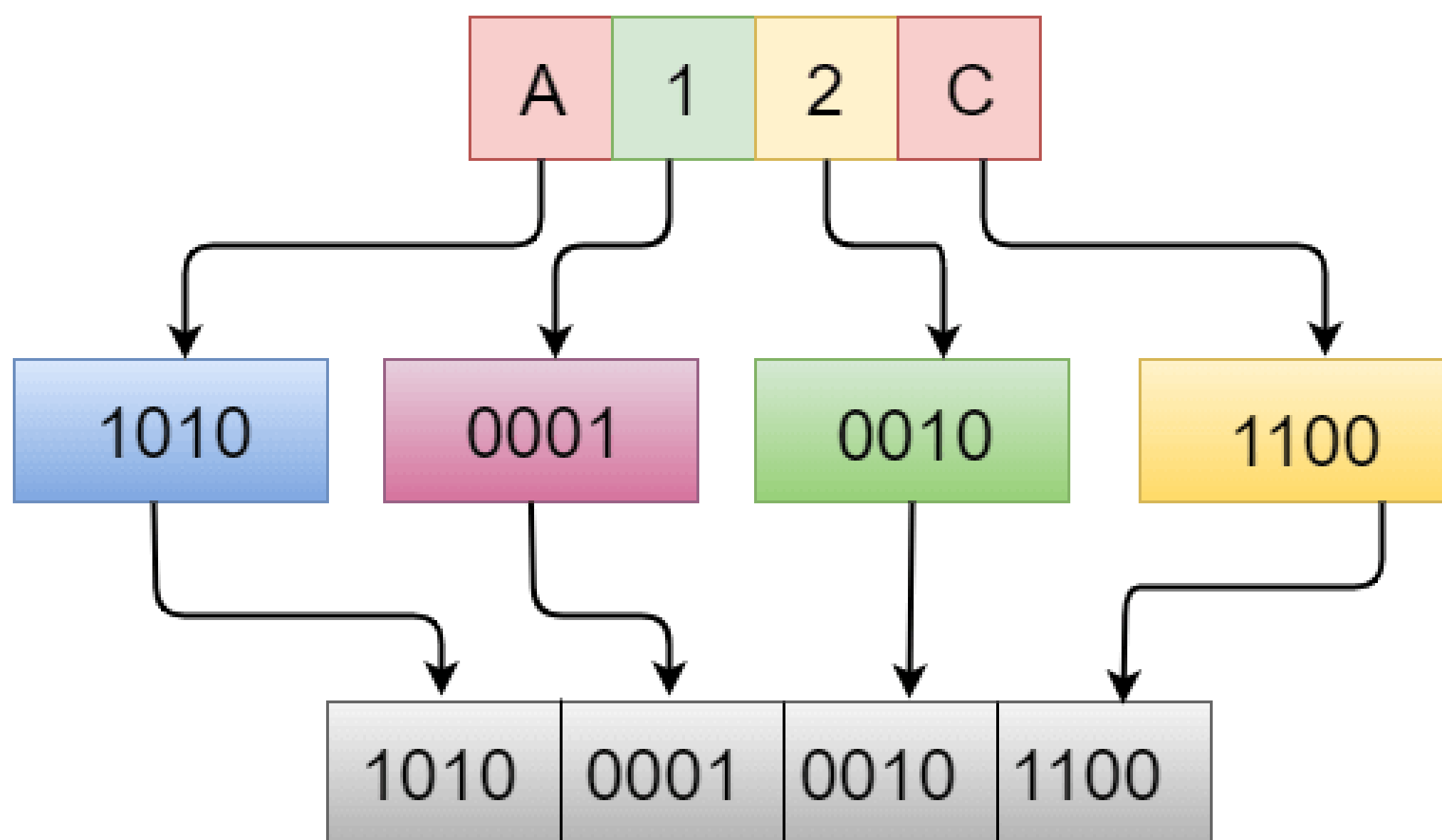
Binary value equivalent to A is 1010

Binary value equivalent to 1 is 0001

Binary value equivalent to 2 is 0010

Binary value equivalent to C is 1100

Therefore, the binary value equivalent to A12C is 1010000100101100.



Note: To convert hexadecimal into a binary number, the binary value of each digit of a hexadecimal number is evaluated and combined to get the binary number of a given hexadecimal number.

Let's understand through an example.

```
#include<stdio.h>
```

```
void hextobin(char b[]);
```

```
void main()
```

```
{
```

```
char hex[]="A12C";

hextobin(hex);

}

void hextobin(char hex[])

{

    int i=0;

    while(hex[i])

    {

        switch(hex[i])

        {

            case '0':

                printf("0000");

                break;

            case '1':

                printf("0001");

                break;

            case '2':

                printf("0010");

                break;

            case '3':

                printf("0011");

                break;
```

```
case '4':  
  
    printf("0100");  
  
    break;  
  
case '5':  
  
    printf("0101");  
  
    break;  
  
case '6':  
  
    printf("0110");  
  
    break;  
  
case '7':  
  
    printf("0111");  
  
    break;  
  
case '8':  
  
    printf("1000");  
  
    break;  
  
case '9':  
  
    printf("1000");  
  
    break;  
  
case 'A':  
  
    printf("1010");  
  
    break;  
  
case 'a':
```

```
printf("1010");
```

```
break;
```

```
case 'B':
```

```
printf("1011");
```

```
break;
```

```
case 'b':
```

```
printf("1011");
```

```
break;
```

```
case 'C':
```

```
printf("1100");
```

```
break;
```

```
case 'c':
```

```
printf("1100");
```

```
break;
```

```
case 'D':
```

```
printf("1101");
```

```
break;
```

```
case 'd':
```

```
printf("1101");
```

```
break;
```

```
case 'E':
```

```
printf("1110");
```

```

        break;

    case 'e':

        printf("1110");

        break;

    case 'F':

        printf("1111");

        break;

    case 'f':

        printf("1111");

        break;

    }

    i++;

}

```

In the above code, we want to calculate the binary value of "A12C". We first store this value in a character array named as `hex[]`, and then we pass this array to a `hextobin()` function. The `hextobin()` will calculate the binary value of the number, it iterates over each element of an array through a while loop, and finds the binary value of each corresponding element through a switch statement.

Output

```
1010000100101100
```

Hexadecimal to Decimal in C

What is Hexadecimal?

The hexadecimal is also known as base-16, it is a number system that uses the 16 symbols to represent a particular value, and these symbols are from (0-9 and A-F).

What is Decimal?

The decimal is the number system that represents both the integer and non-integer numbers. This number system is also known as the base-10 system, and it uses 10 symbols from (0-9) to represent a particular value.

Conversion of Hexadecimal to Binary number

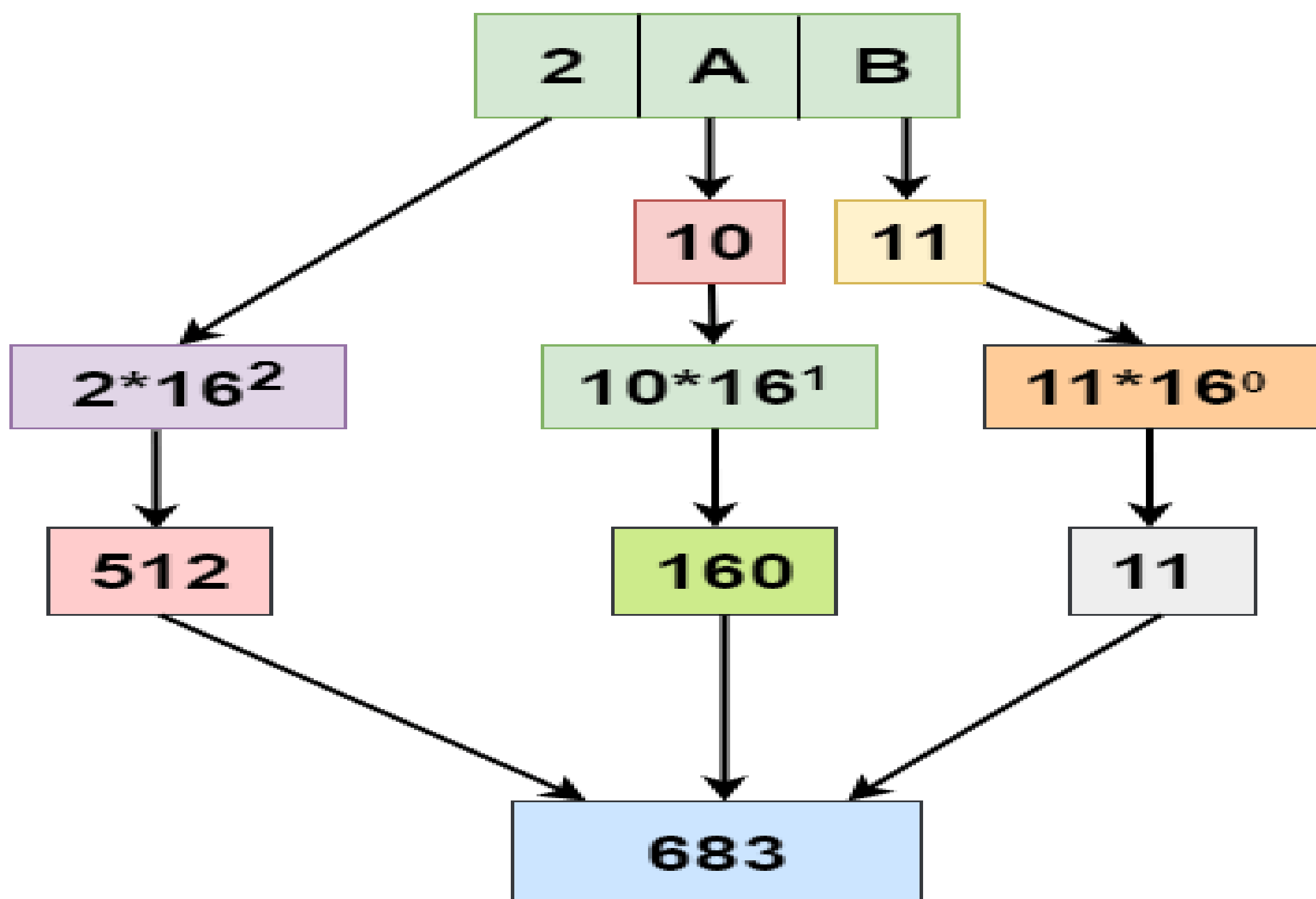
We know that hexadecimal number contains 16 symbols, i.e., 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F where A,B,C,D,E,F represents (10,11,12,13,14,15) values. We can find the decimal number by extracting the digits from a hexadecimal number, and starting from the right-most digit of the number. Each digit of the hexadecimal number is multiplied with the base 16. We will store the result of this multiplication in a particular variable.

Let's understand through an example.

If the hexadecimal number is 5A, then its decimal value would be:

$$\begin{aligned}\text{decimal_value} &= 5 \cdot 16^1 + 10 \cdot 16^0 \\ &= 90\end{aligned}$$

Now, we understand the conversion of hexadecimal number into a decimal number diagrammatically:



In the above diagram, we consider **2AB** as a hexadecimal number and calculated its decimal value.

$$\begin{aligned}
 2AB \text{ (Hexadecimal value)} &= 2 \cdot 16^2 + 10 \cdot 16^1 + 11 \cdot 16^0 \\
 &= 683 \text{ (Decimal value)}
 \end{aligned}$$

Let's understand through the program.

```

#include <stdio.h>

#include <math.h>

#include <string.h>

int main()
{

    char hex[17];           // declaration of character array.

    long long decimal, place;

    int i = 0, val, len;    // variables declaration
  
```

```

decimal = 0;

/* Input hexadecimal number from user */

printf("Enter any hexadecimal number: ");

gets(hex);

/* Find the length of total number of hex digit */

len = strlen(hex);

len--;

/*

* Iterate over each hex digit

*/

while(hex[i]!='\0')

{

    /* To find the decimal representation of hex[i] */

    if(hex[i]>='0' && hex[i]<='9')

    {

        val = hex[i] - 48;

    }

    else if(hex[i]>='a' && hex[i]<='f')

    {

        val = hex[i] - 97 + 10;

    }

    else if(hex[i]>='A' && hex[i]<='F')

```

```

{

    val = hex[i] - 65 + 10;

}

decimal += val * pow(16, len);

```

In the above program, we find the decimal number of a given hexadecimal number. In the above case, we take input from the user and store its value in the `hex` variable. After retrieving the input from the user, we calculate the length of the input and store it in the `len` variable. We have iterated each digit of a hexadecimal number and stores the final result in the `decimal_value` variable

Output

```

Enter any hexadecimal number: 2AB
Hexadecimal number = 2AB
Decimal number = 683

...Program finished with exit code 0
Press ENTER to exit console.

```

Octal to Hexadecimal in C

What is an octal number system?

An octal number system is a number system which is having base-8 where base-8 represents the eight digits from 0 to 7.

What is a Hexadecimal number system?

A hexadecimal number system is a number system which is having base-16 where base-16 represents the sixteen digits, i.e., 0,1,2,3,4,5,6,7,8,9, A, B, C, D, E, F.

How to convert an octal number into a hexadecimal number

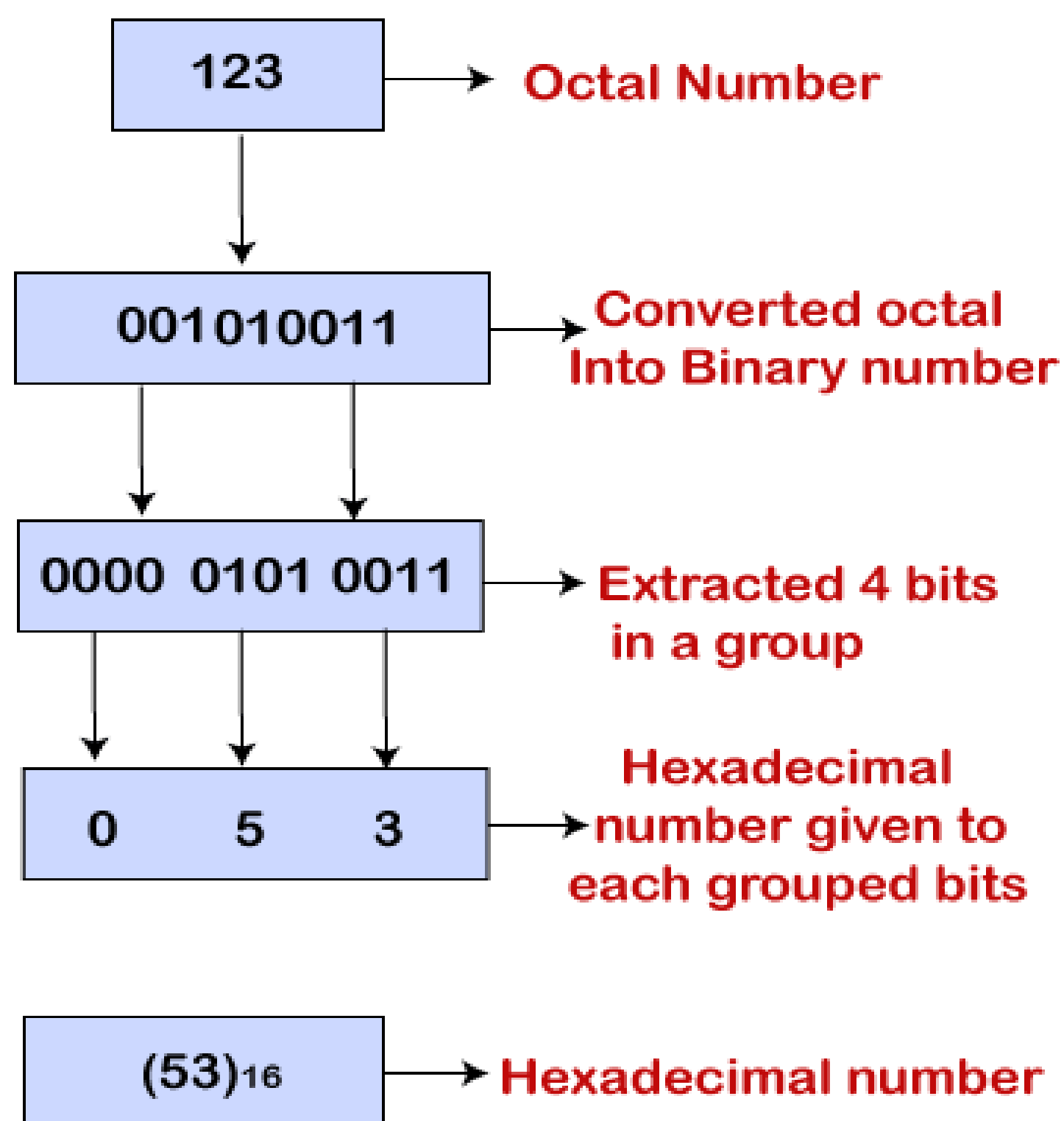
We cannot directly convert an octal number system into a hexadecimal number system.

First, we need to convert the octal number into a binary number system, and then this binary number system will be converted into a hexadecimal number system.

Let's look at the approach.

- Input the octal number.
- An octal number system is converted into a binary number system.
- Extract the 4 bits in a group from the right-side.
- Provide the hexadecimal number to the extracted 4 bits.

For example, the octal number is 123.



Let's create a simple program which converts the octal into a hexadecimal number.

```
#include <stdio.h>

#include<string.h>

int main()

{

    int octaltobinary[]={0,1,10,11,100,101,110,111};

    char hexadecimal[10];

    char hex[10];

    long int binary=0;

    int octal;

    int rem=0;

    int position=1;

    int len=0;

    int k=0;

    printf("Enter a octal number");

    scanf("%d",&octal);

    // Converting octal number into a binary number.

    while(octal!=0)

    {

        rem=octal%10;

        binary=octaltobinary[rem]*position+binary;
```

```

    octal=octal/10;

    position=position*1000;

}

printf("The binary number is : %ld",binary);

// Converting binary number into a hexadecimal number.

while(binary > 0)

{

    rem = binary % 10000;

    switch(rem)

    {

        case 0:

            strcat(hexadecimal, "0");

            break;

        case 1:

            strcat(hexadecimal, "1");

            break;

        case 10:

            strcat(hexadecimal, "2");

            break;

        case 11:

            strcat(hexadecimal, "3");

            break;

```

```
case 100:

    strcat(hexadecimal, "4");

    break;

case 101:

    strcat(hexadecimal, "5");

    break;

case 110:

    strcat(hexadecimal, "6");

    break;

case 111:

    strcat(hexadecimal, "7");

    break;

case 1000:

    strcat(hexadecimal, "8");

    break;

case 1001:

    strcat(hexadecimal, "9");

    break;

case 1010:

    strcat(hexadecimal, "A");

    break;

case 1011:
```

```

        strcat(hexadecimal, "B");

        break;

    case 1100:

        strcat(hexadecimal, "C");

        break;

    case 1101:

        strcat(hexadecimal, "D");

        break;

    case 1110:

        strcat(hexadecimal, "E");

        break;

    case 1111:

        strcat(hexadecimal, "F");

        break;

    }

    len=len+1;

    binary /= 10000;

}

for(int i=len-1;i>=0;i--)

{

    hex[k]=hexadecimal[i];

    k++;

```

```

}

hex[len]='\0';

printf("\nThe hexadecimal number is :");

for(int i=0; hex[i]!='\0';i++)

{

    printf("%c",hex[i]);

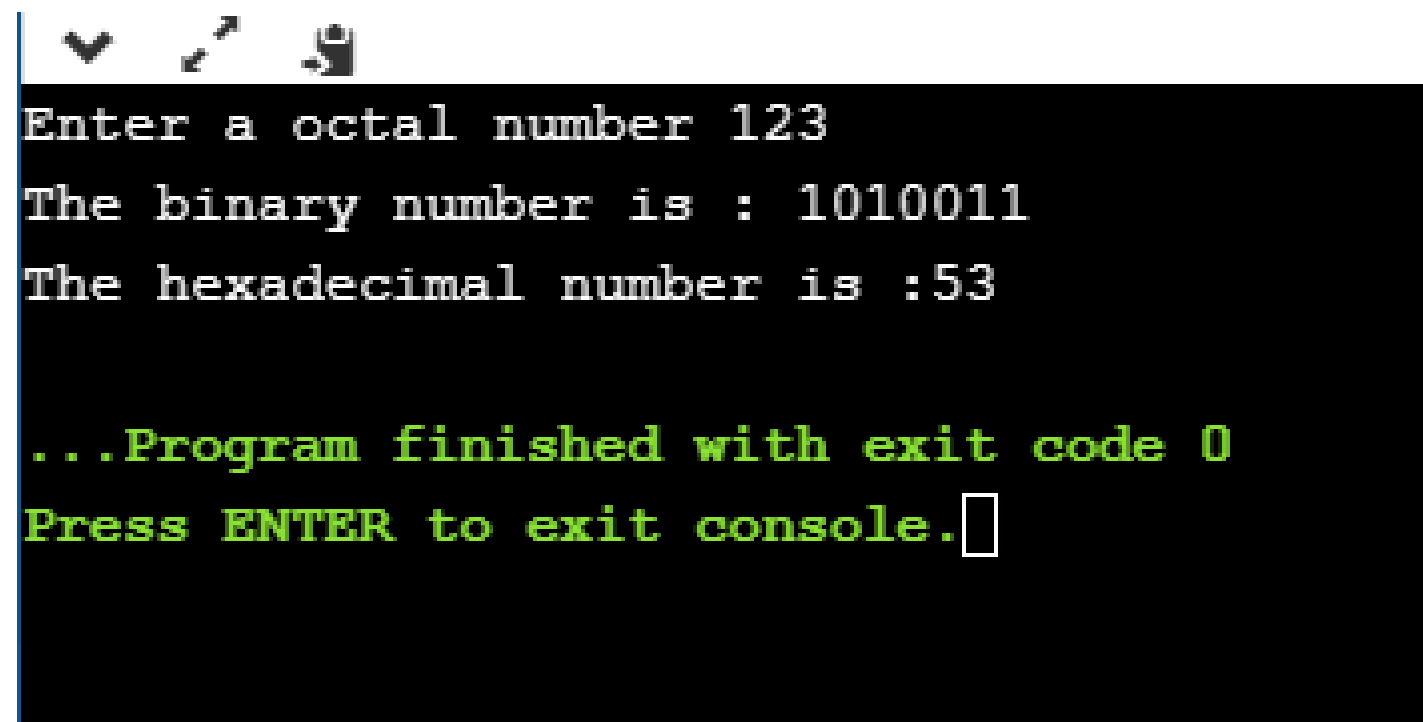
}

return 0;

}

```

Output



```

Enter a octal number 123
The binary number is : 1010011
The hexadecimal number is :53

...Program finished with exit code 0
Press ENTER to exit console.

```

Analysis of the above program

- First, we take the user input, which would be an octal number, and it gets stored in an 'octal' variable.
- After entering the user input, we convert the octal number into binary number. We iterate through the **while(octal!=0)** loop, which executes till the value of octal is not equal to zero. The final value of a binary number will be stored in a 'binary' variable.

- After calculating the binary number, we will calculate the hexadecimal number. We will iterate through the elements of a binary number by using a `while(binary>0)`. On each iteration, four digits from the right-side of a binary number are extracted and replaced with their corresponding hexadecimal number.
- The `hexadecimal[]` variable contains the reverse value of the actual hexadecimal number. In order to get the actual hexadecimal number of an octal number, we define a new variable, i.e., `hex[]`. We will iterate the elements of a `hexadecimal[]` variable from the last, and stores each element in `hex[]`.

Strong number in C

A number can be said as a strong number when the sum of the factorial of the individual digits is equal to the number.

For example, 145 is a strong number.

Let's understand through an example.

Program to check whether the number is strong or not.

```
#include <stdio.h>

int main()
{
    int n;

    int sum=0;

    printf("Enter a number");

    scanf("%d",&n);

    int k=n;
```

```

int r;

while(k!=0)

{

    r=k%10;

    int f=fact(r);

    k=k/10;

    sum=sum+f;

}

if(sum==n)

{

    printf("\nNumber is a strong");

}

else

{

    printf("\nNumber is not a strong");

}

return 0;

}

int fact(int r)

{

    int mul=1;

    for(int i=1;i<=r;i++)

```

```

{

    mul=mul*i;

}

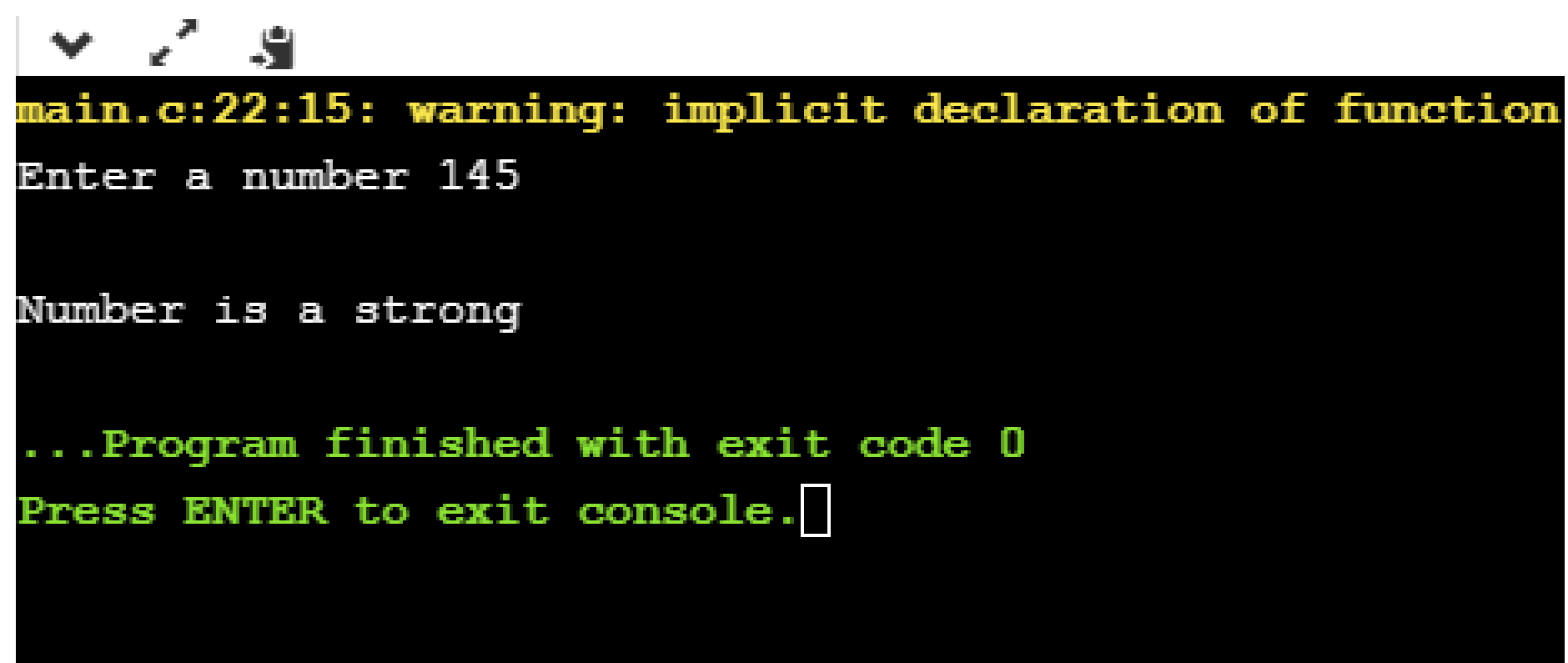
return mul;

}

```

In the above code, the input data is retrieved from the user, and then the data is checked whether the input data is a strong number or not.

Output



```

main.c:22:15: warning: implicit declaration of function
Enter a number 145

Number is a strong

...Program finished with exit code 0
Press ENTER to exit console.

```

Program to print the strong numbers from 1 to n.

```

#include<stdio.h>

int main()

{

    int fact=1,sum=0;

    int n,r;

    printf("Enter the 'n' number");

```

```

scanf("%d",&n);

printf("\n Strong numbers are :");

for(int i=1;i<=n;i++)

{

    int k=i;

    while(k!=0)

    {

        r=k%10;

        fact=factorial(r);

        k=k/10;

        sum=sum+fact;

    }

    if(sum==i){

        printf("%d, ",i);

    }

    sum=0;

}

return 0;

}

int factorial(int f)

{

    int mul=1;

```

```

for(int i=1; i<=f; i++)

{

    mul=mul*i;

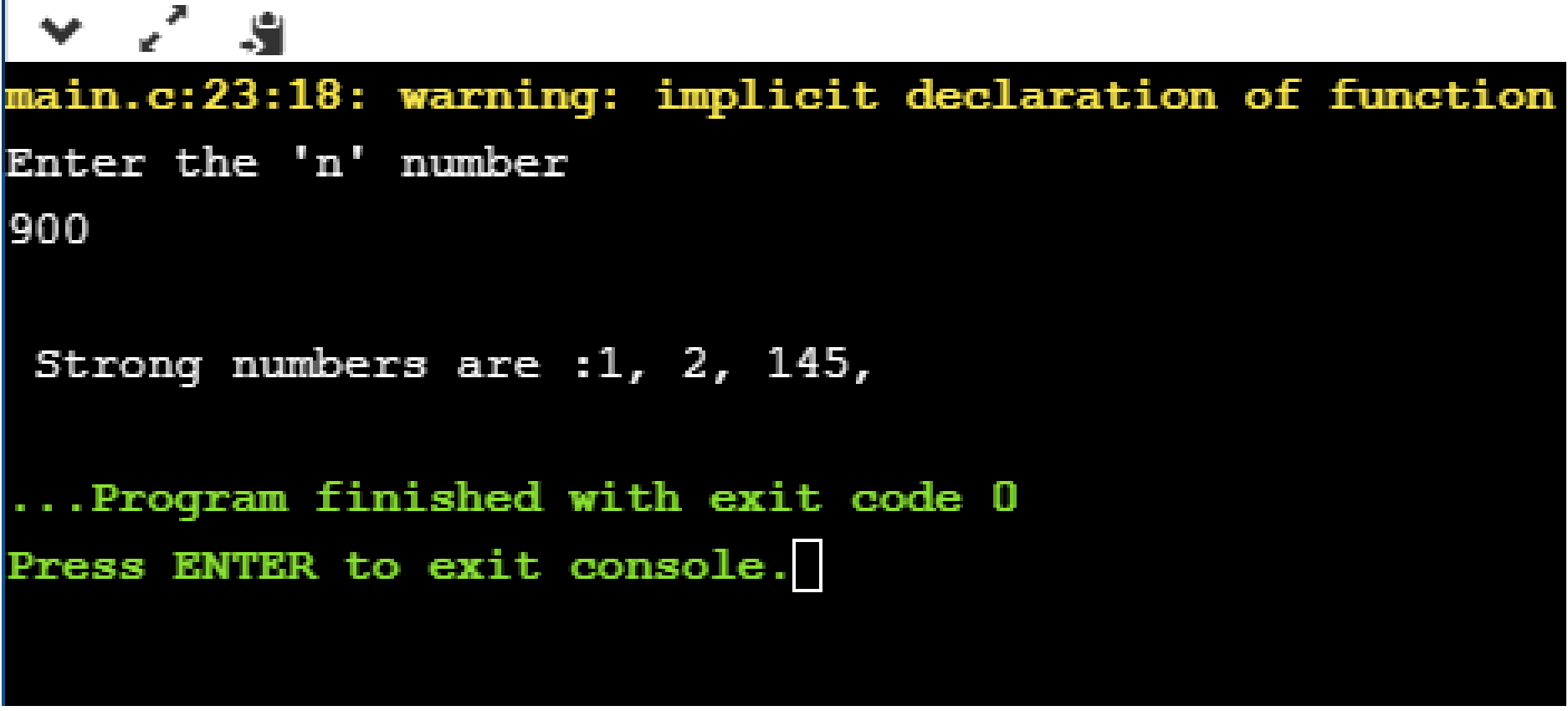
}

return mul;

}

```

Output



```

main.c:23:18: warning: implicit declaration of function
Enter the 'n' number
900

Strong numbers are :1, 2, 145,

...Program finished with exit code 0
Press ENTER to exit console.

```

Program to find strong numbers in a given range.

```

#include<stdio.h>

int main()

{

    int fact=1,sum=0;

    int n1,n2,r;

    printf("Enter the first number");

    scanf("%d",&n1);

    printf("\nEnter the last number");

```

```

scanf("%d",&n2);

printf("\nStrong numbers are :");

for(int i=n1;i<=n2;i++)

{

    int k=i;

    while(k!=0)

    {

        r=k%10;

        fact=factorial(r);

        k=k/10;

        sum=sum+fact;

    }

    if(sum==i){

        printf("%d, ",i);

    }

    sum=0;

}

return 0;

}

int factorial(int f)

{

    int mul=1;

```

```

for(int i=1; i<=f; i++)

{

    mul=mul*i;

}

return mul;

}

```

Output:

```

Enter the first number 1

Enter the last number 300

Strong numbers are :1, 2, 145,

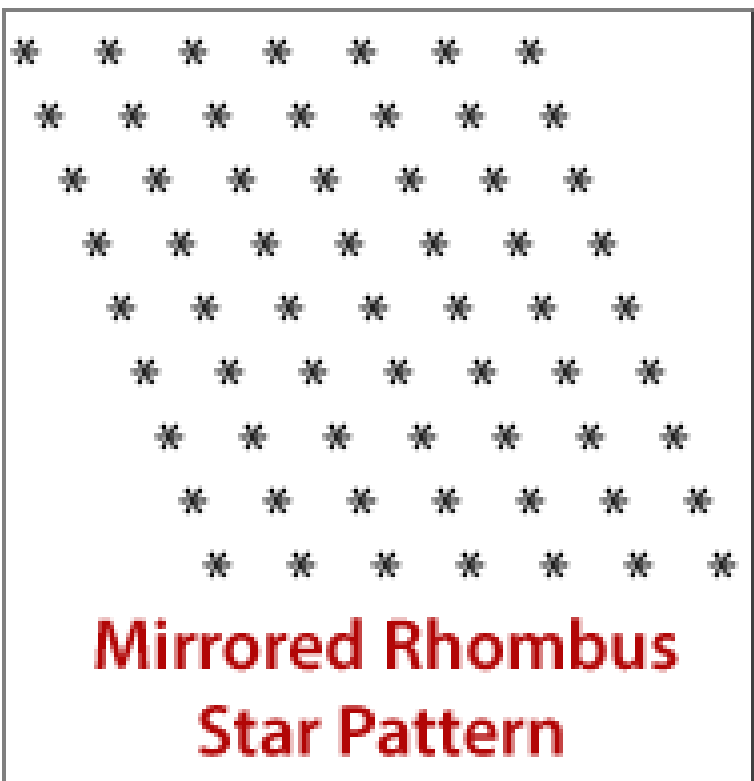
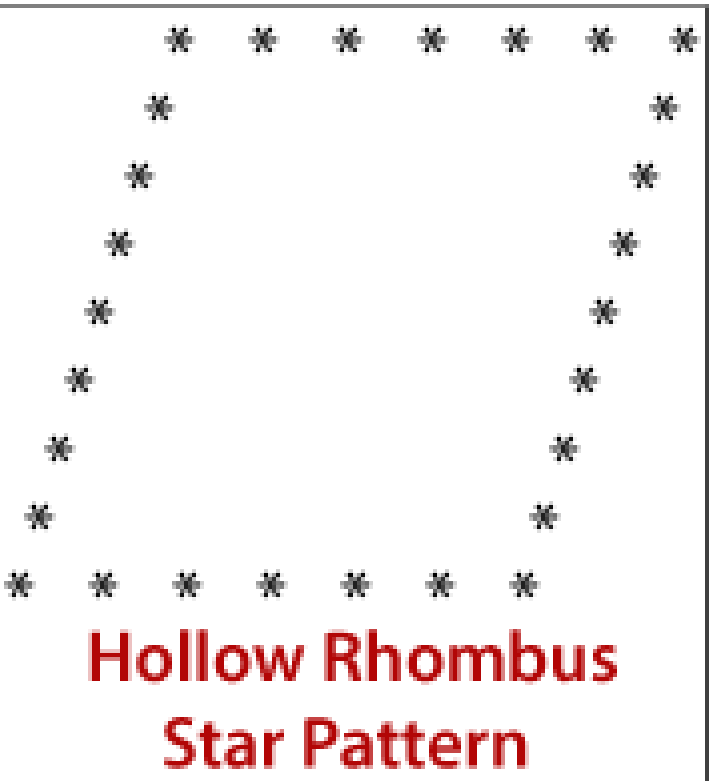
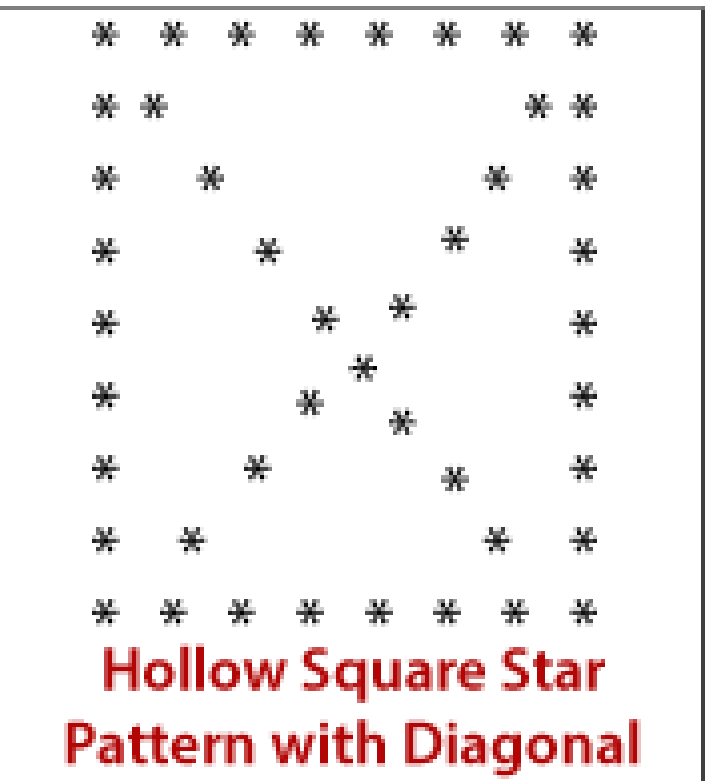
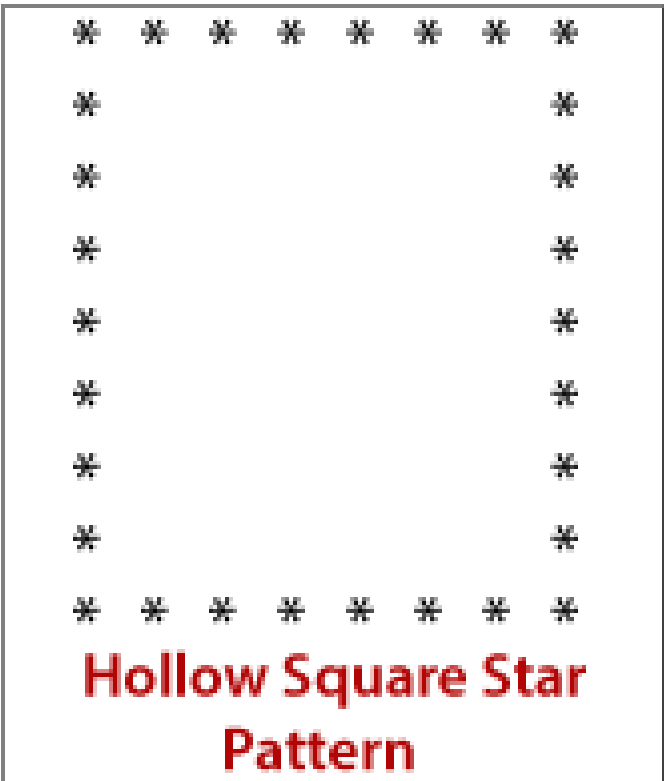
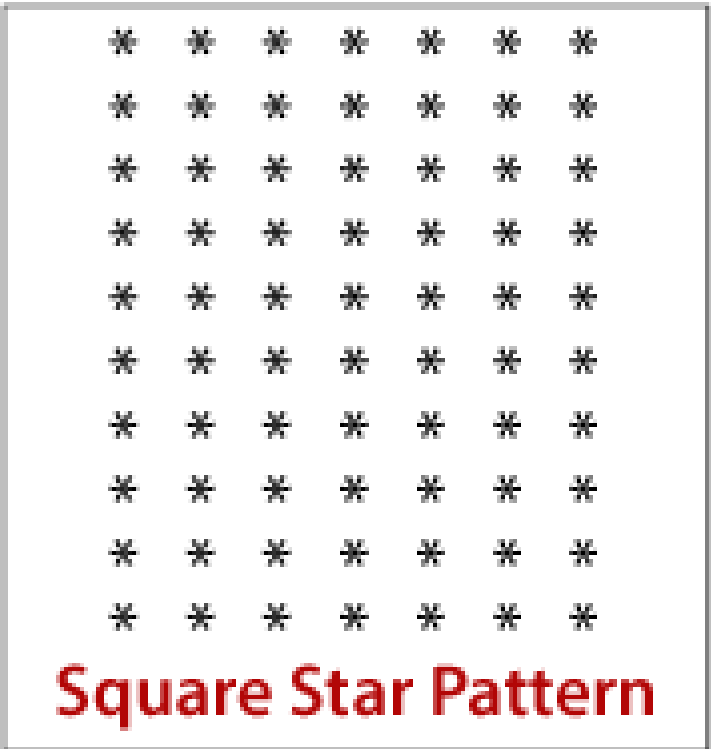
...Program finished with exit code 0
Press ENTER to exit console.

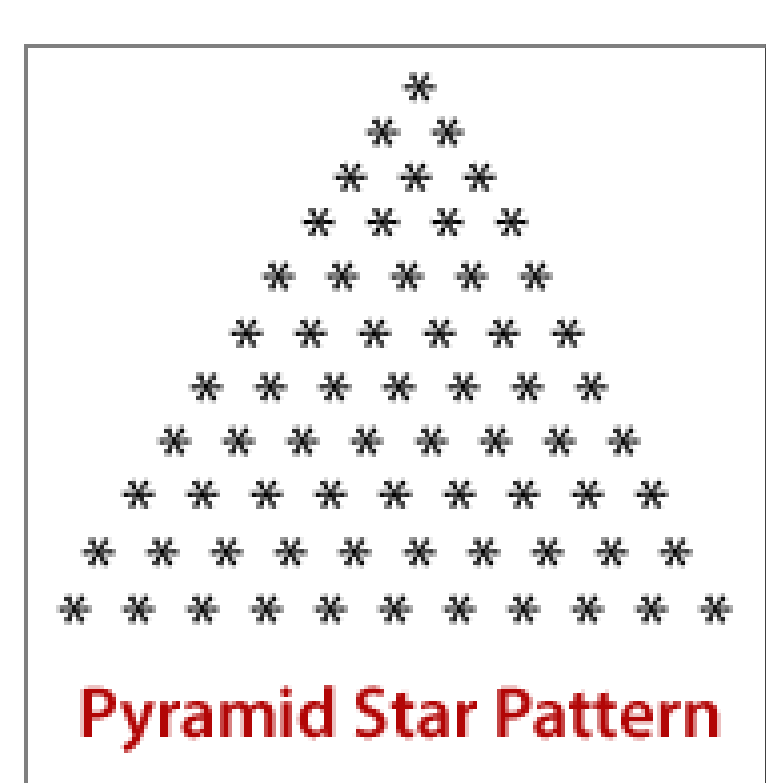
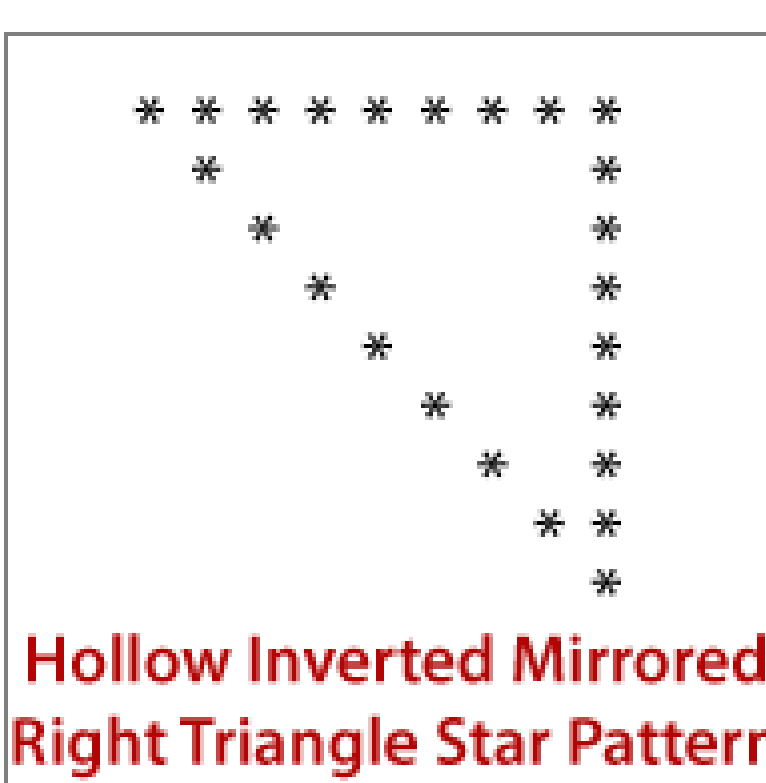
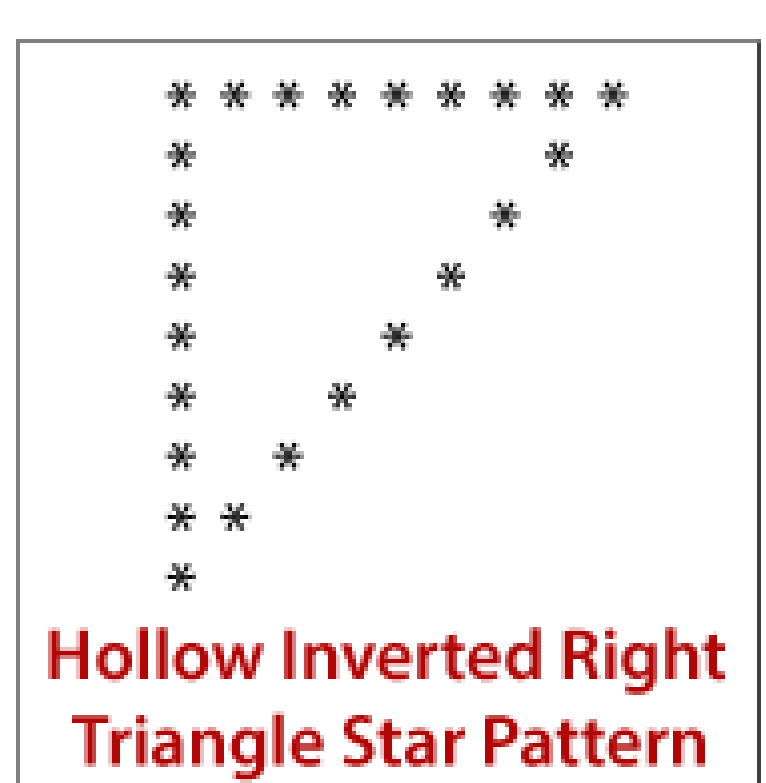
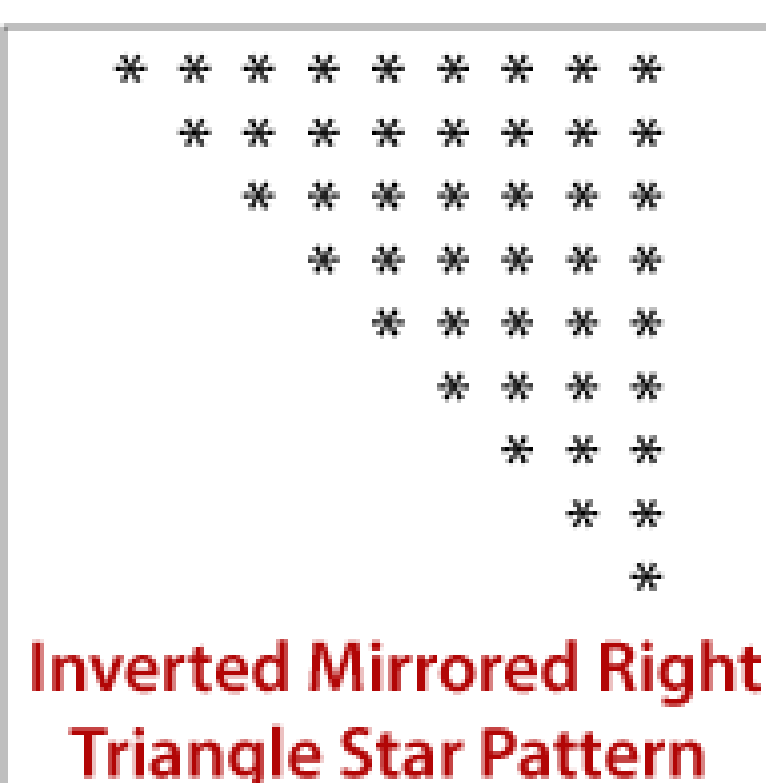
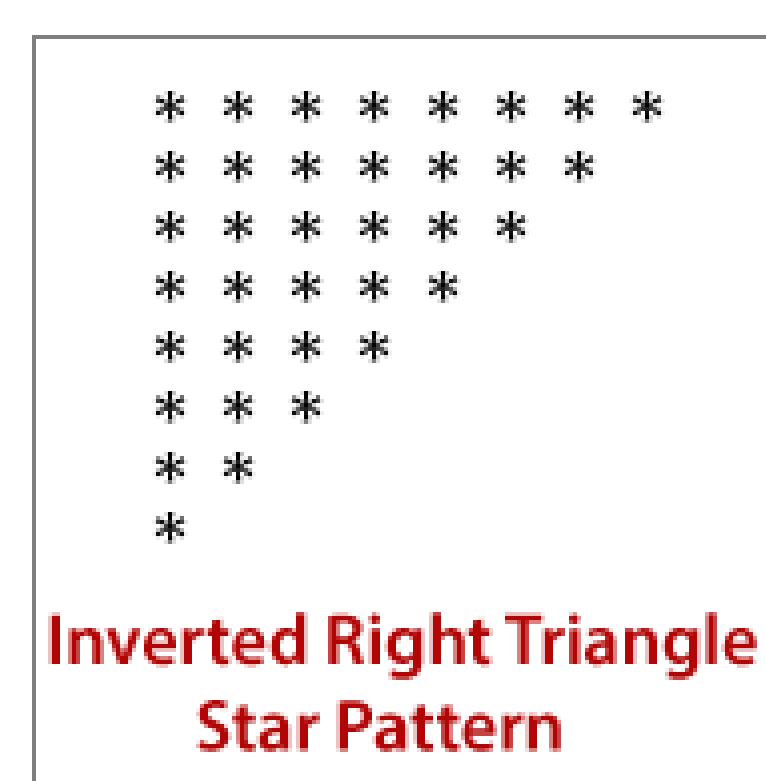
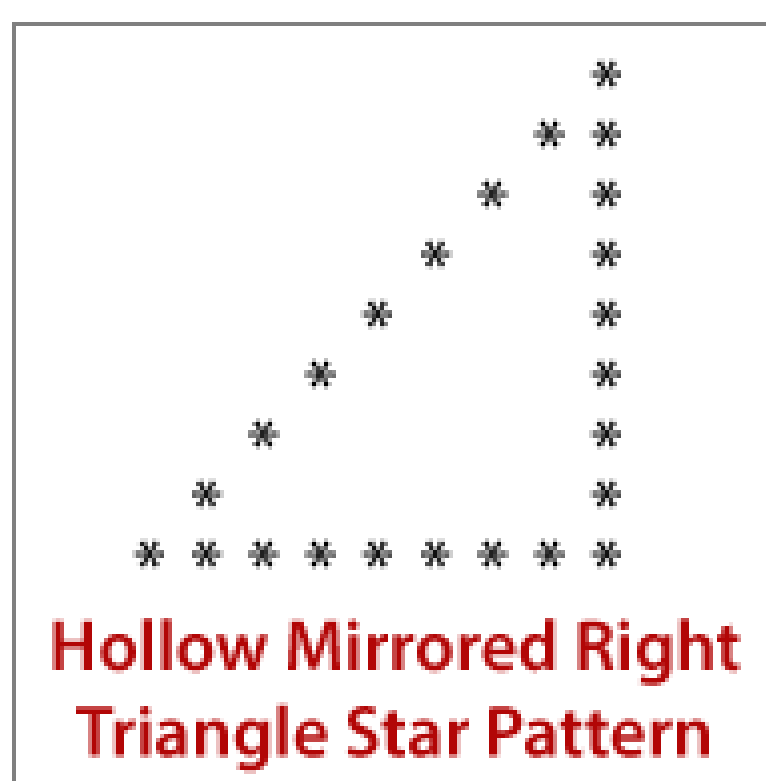
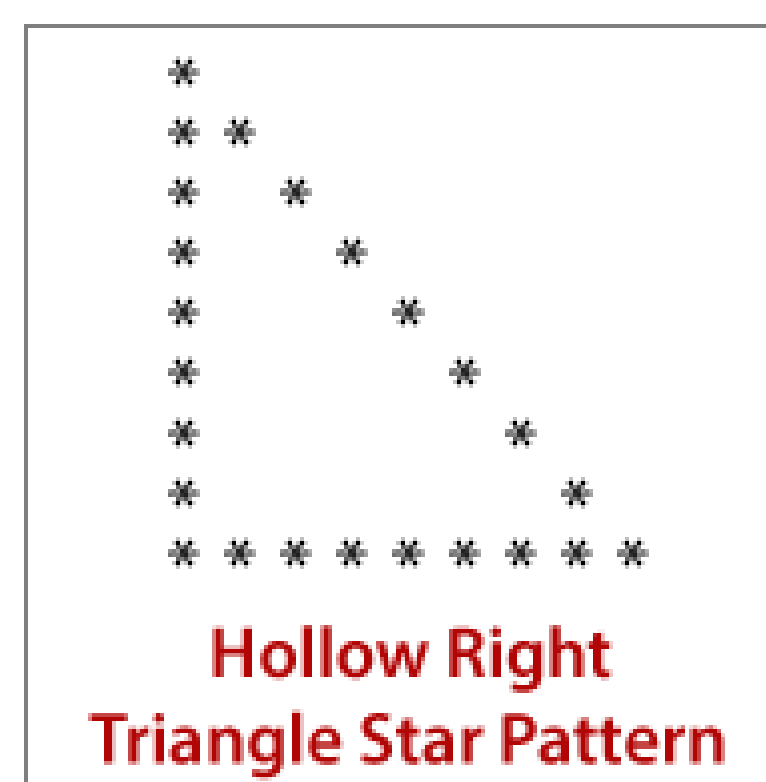
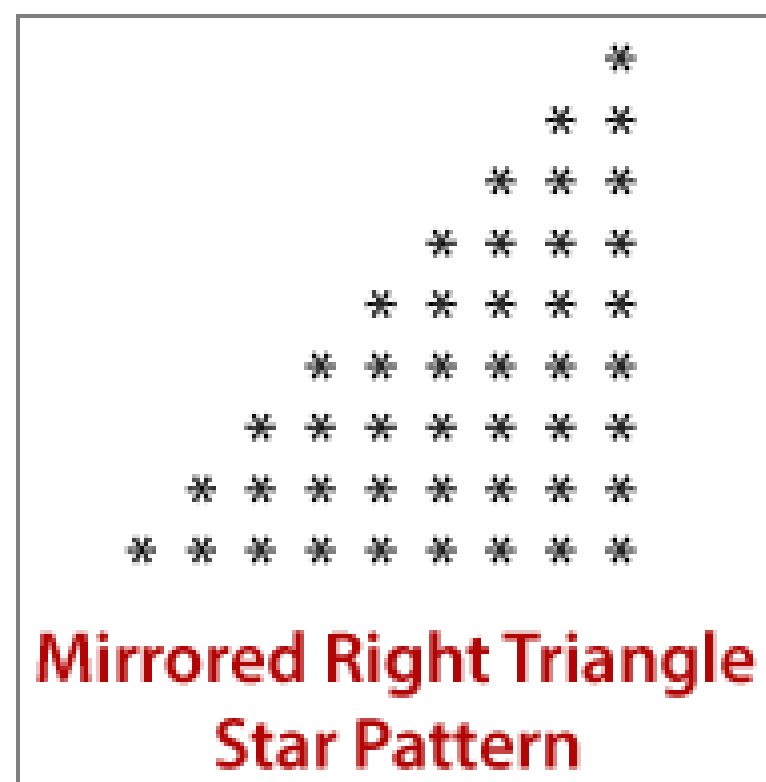
```

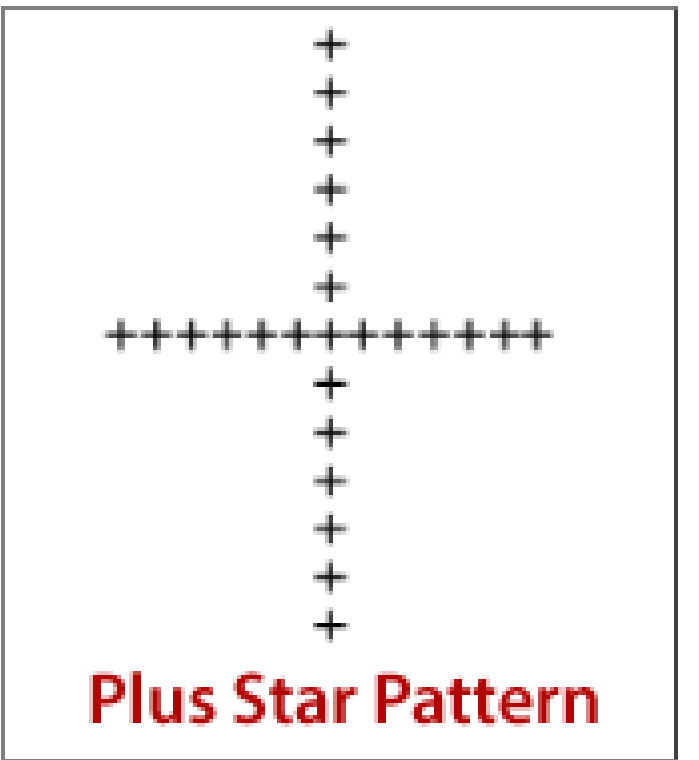
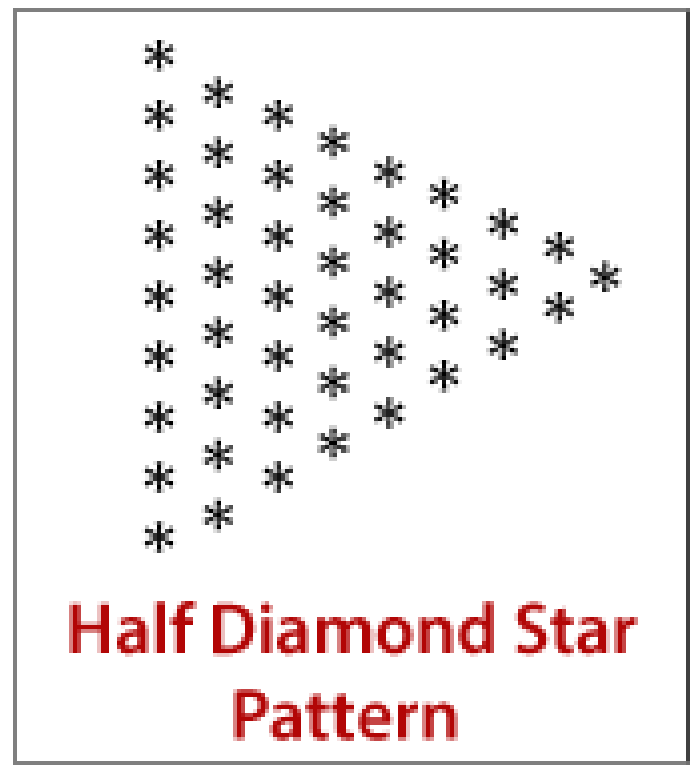
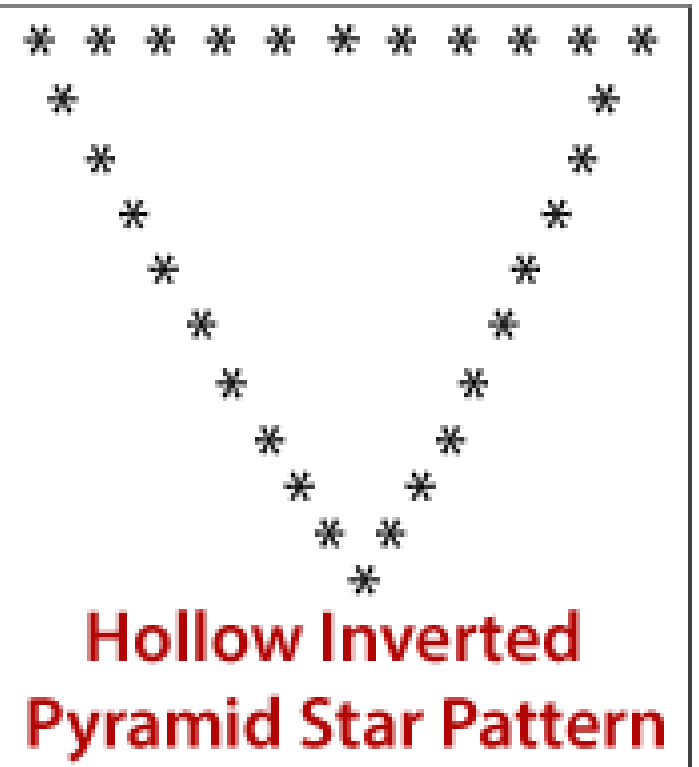
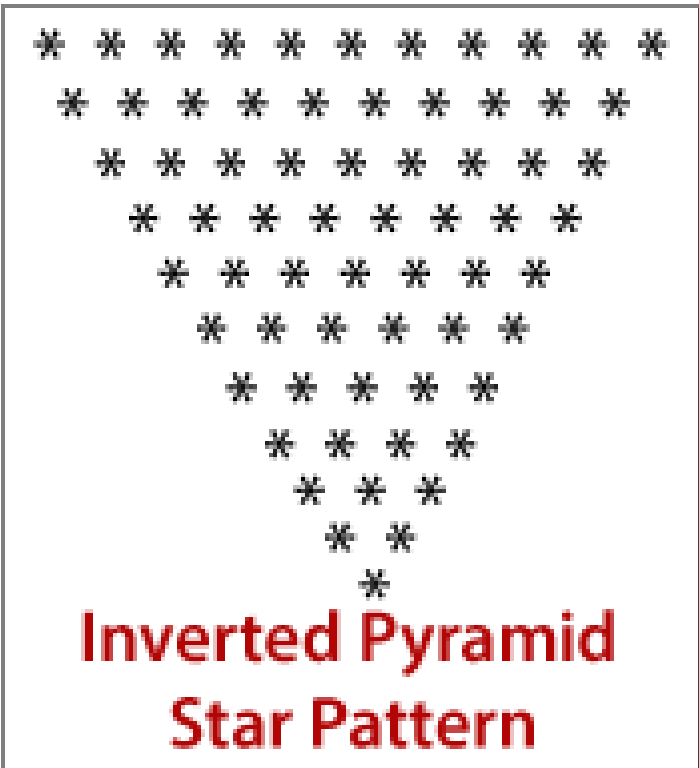
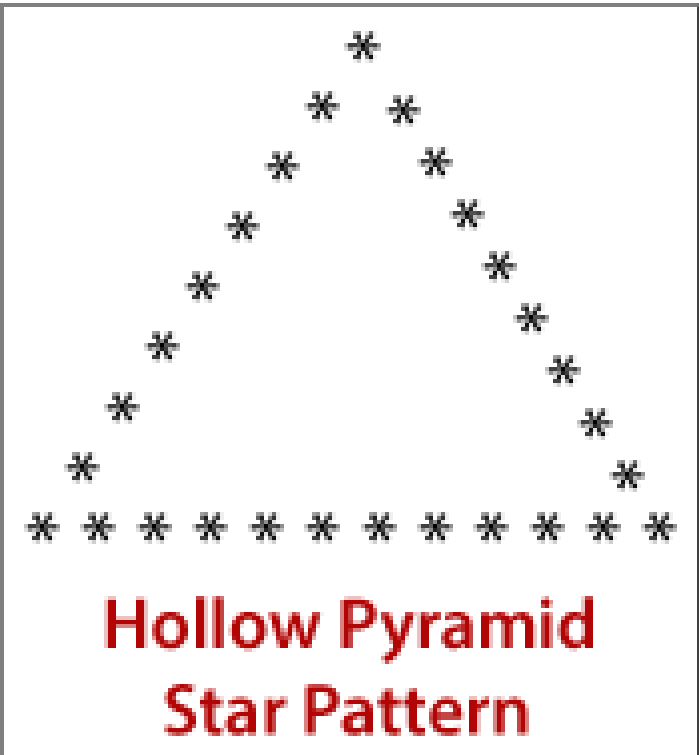
Star Pattern Program in C

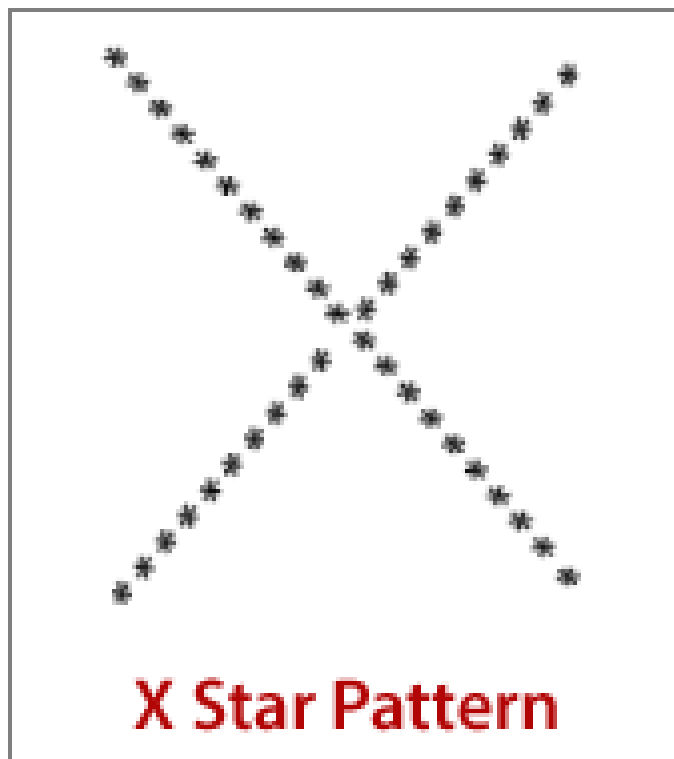
In this topic, we will learn how to create the patterns by using C language. We will create the patterns by using either a '*' star character or some other character. We will create different patterns or a geometrical shape such as triangle, square, etc.

We are going to cover the following patterns. You can view the code of any pattern given below by clicking on the pattern.









Square Star Pattern

The code to create the square star pattern is given below:

```
#include <stdio.h>

int main()
{
    int n;

    printf("Enter the number of rows");

    scanf("%d",&n);

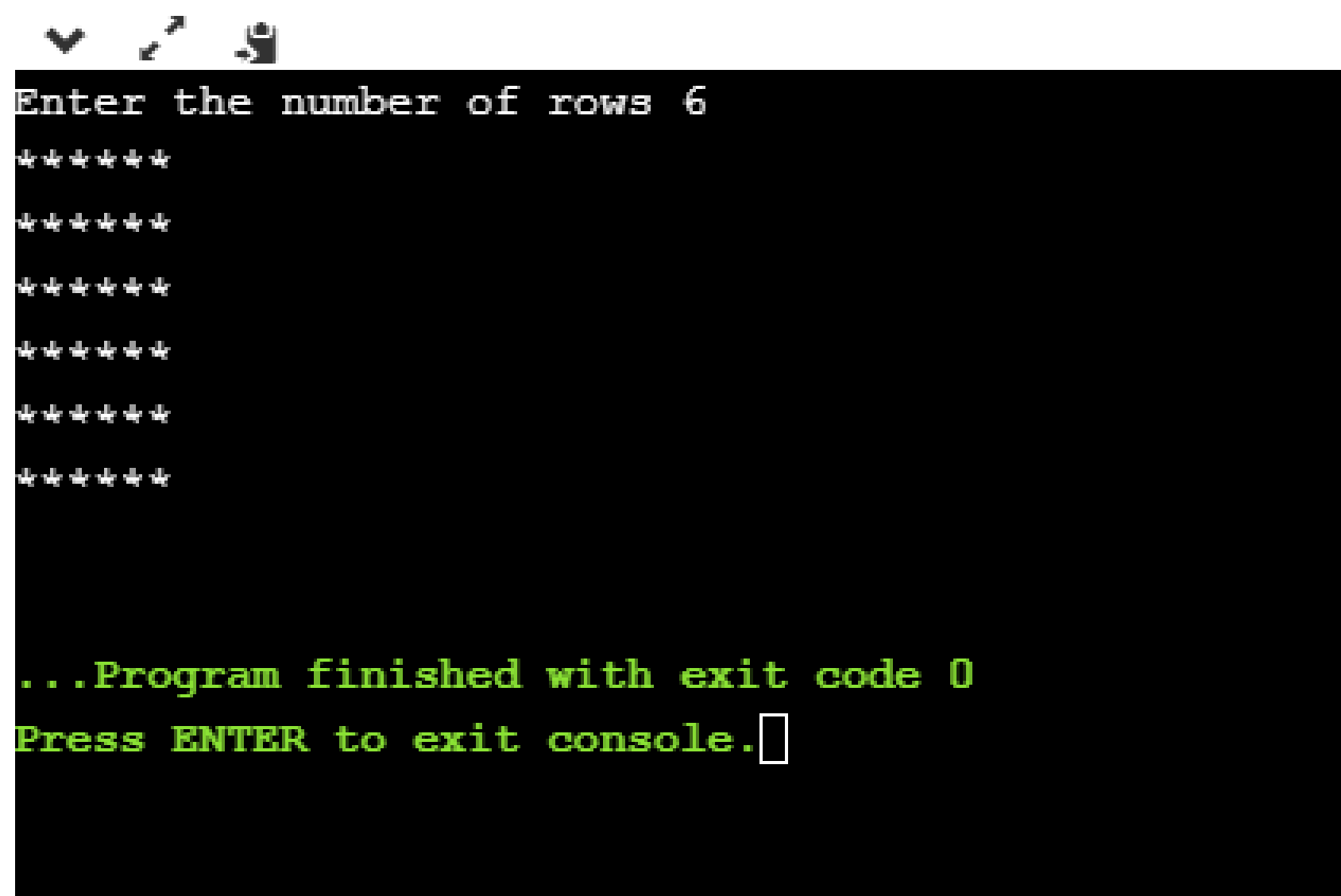
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)
        {
            printf("*");
        }

        printf("\n");
    }
}
```

```
return 0;

}
```

Output



```
Enter the number of rows 6
*****
*****
*****
*****
*****
*****

...Program finished with exit code 0
Press ENTER to exit console.
```

Hollow Square Star Pattern

Now, we create the hollow square star pattern. The source code for this pattern is given below:

```
int main()

{

    int n;

    printf("Enter the number of rows");

    scanf("%d",&n);

    for(int i=1;i<=n;i++)

    {

        for(int j=1;j<=n;j++)
```

```

{

    if(i==1 ||i==n||j==1||j==n)

    {

        printf("*");

    }

    else

        printf(" ");

}

printf("\n");

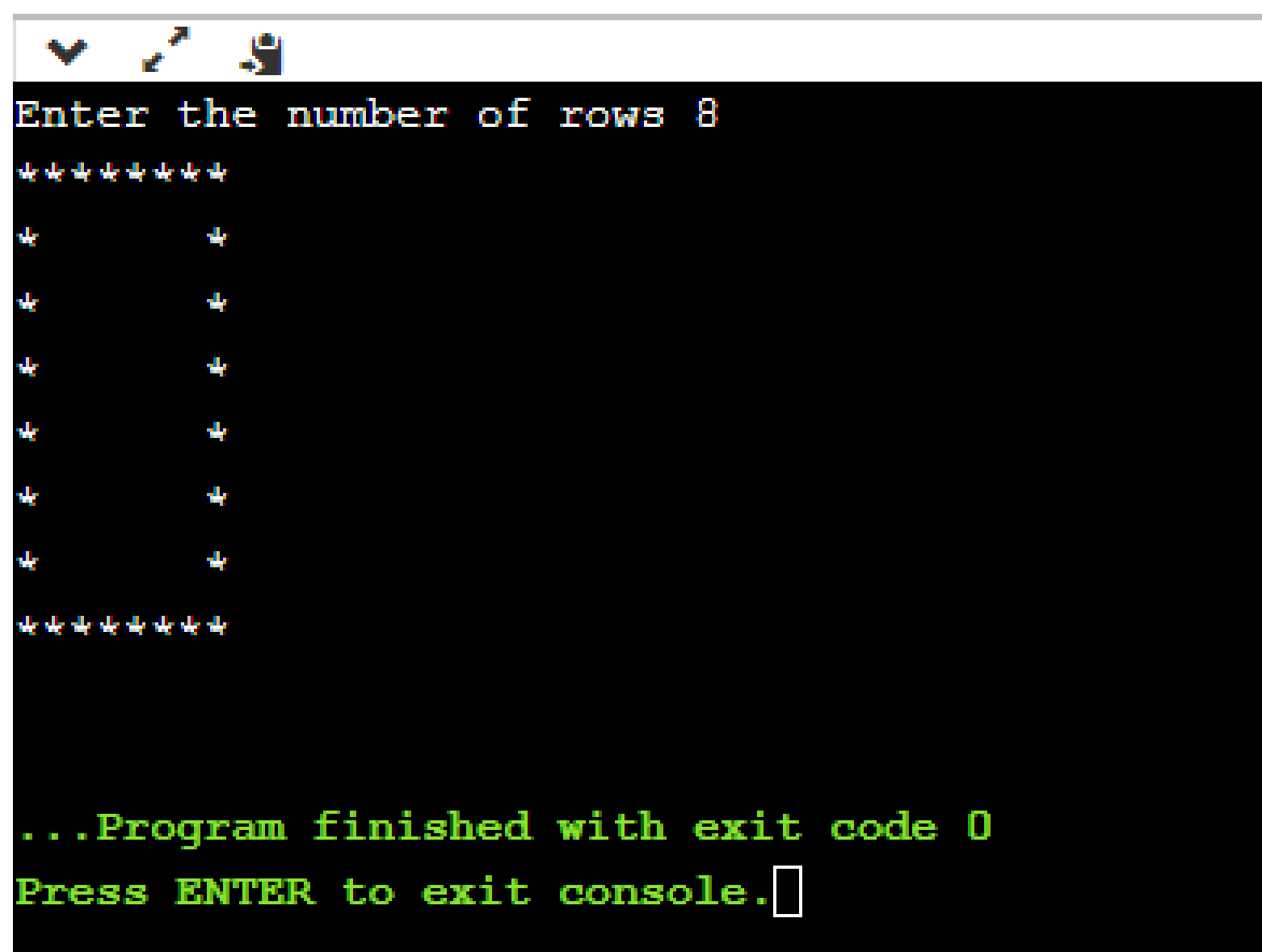
}

return 0;

}

```

Output



```

Enter the number of rows 8
*****
*       *
*       *
*       *
*       *
*       *
*       *
*       *
*****

...Program finished with exit code 0
Press ENTER to exit console.

```

Hollow Square Pattern with Diagonal

The code for the hollow square star pattern is given below:

```
#include <stdio.h>

int main()

{

    int n;

    printf("Enter the number of rows");

    scanf("%d",&n);

    for(int i=1;i<=n;i++)

    {

        for(int j=1;j<=n;j++)

        {

            if(i==1 ||i==n||j==1||j==n-i+1||i==j||j==n)

            {

                printf("*");

            }

            else

            {

                printf(" ");

            }

        }

        printf("\n");

    }
```

```

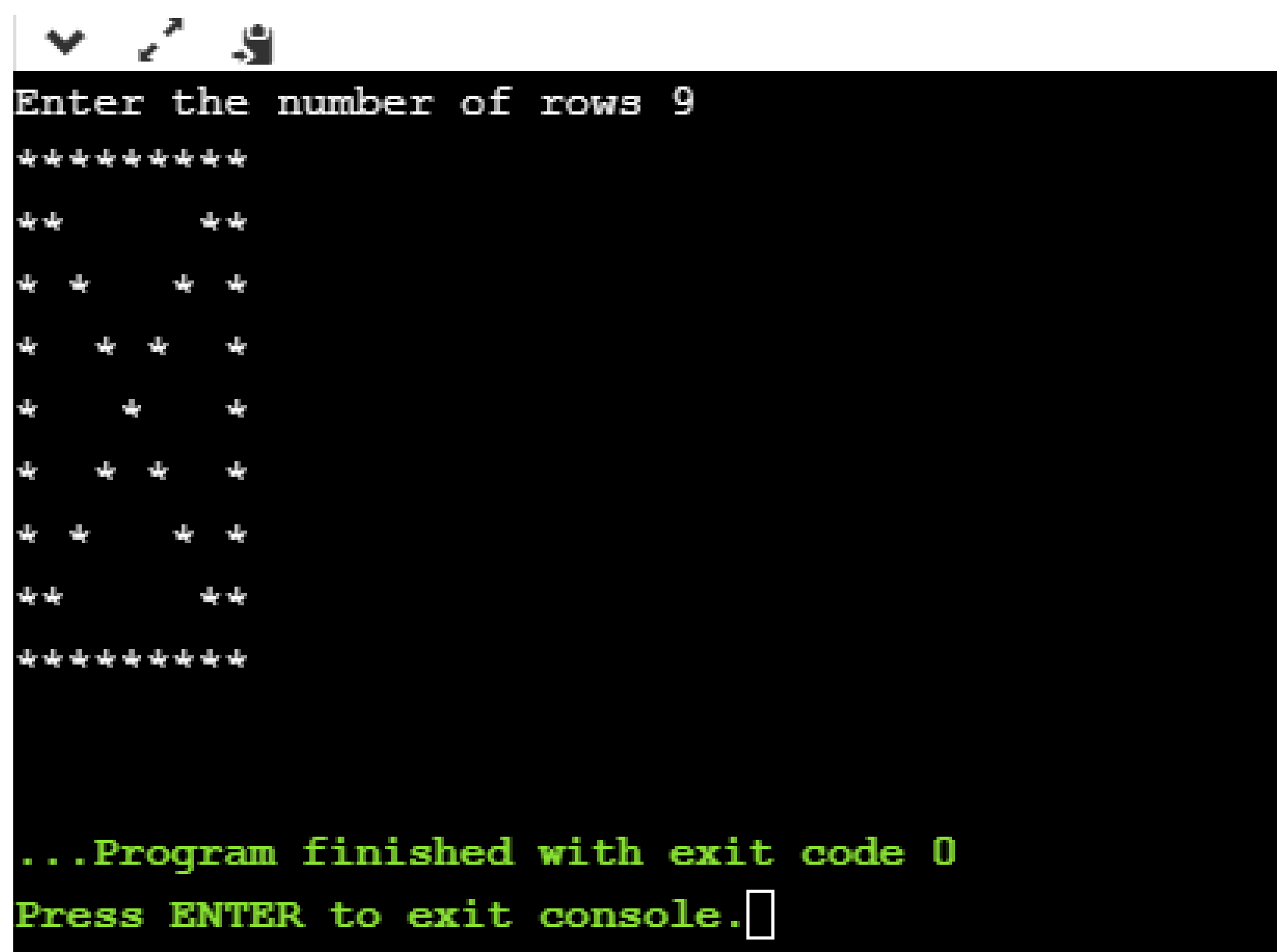
}

return 0;

}

```

Output



```

Enter the number of rows 9
*****
**      **
*  *    *  *
*    *  *    *
*      *      *
*    *  *    *
*  *    *  *
**      **
*****

...Program finished with exit code 0
Press ENTER to exit console.

```

Rhombus Star Pattern

The code for the rhombus star pattern is given below:

```

#include <stdio.h>

int main()

{

    int n;

    printf("Enter the number of rows");

    scanf("%d",&n);

    for(int i=n;i>=1;i--)

    {

```

```

        for(int j=1;j<=i-1;j++)

        {

            printf(" ");

        }

        for(int k=1;k<=n;k++)

        {

            printf("*");

        }

        printf("\n");

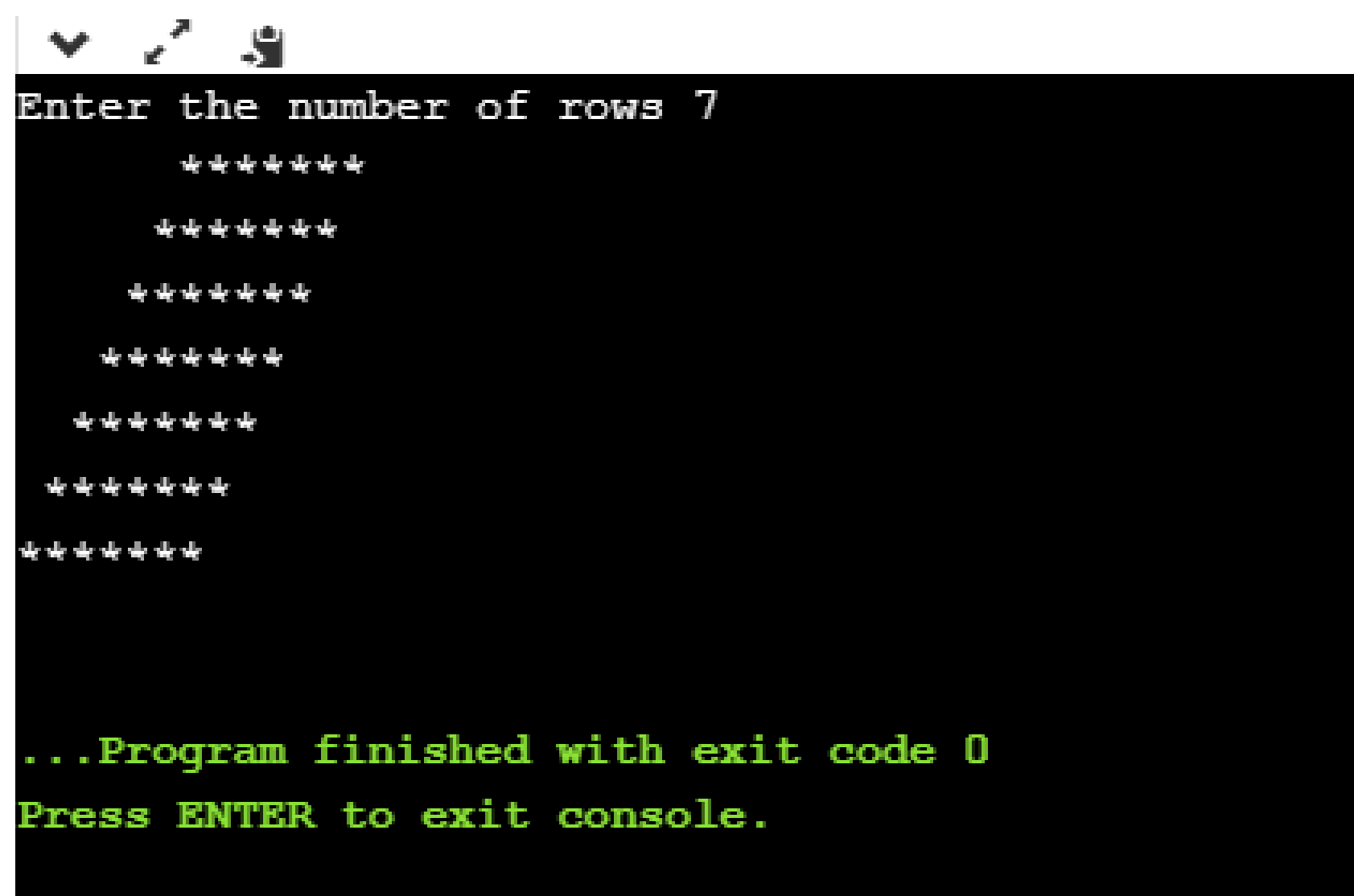
    }

    return 0;

}

```

Output



```

Enter the number of rows 7
      *
    *
  *
*
*
*
*

...Program finished with exit code 0
Press ENTER to exit console.

```

Hollow Rhombus Star Pattern

The code for the hollow rhombus star pattern is given below:

```
#include <stdio.h>

int main()

{

    int n;

    printf("Enter the number of rows");

    scanf("%d",&n);

    for(int i=n;i>=1;i--)

    {

        for(int j=1;j<=i-1;j++)

        {

            printf(" ");

        }

        for(int k=1;k<=n;k++)

        {

            if(i==1 || i==n || k==1 || k==n)

                printf("*");

            else

                printf(" ");

        }

        printf("\n");

    }
```

```

}

return 0;

}

```

Output

```

Enter the number of rows 6

  * * * * *
 *   *
 *   *
 *   *
 *   *
 * * * * *

...Program finished with exit code 0
Press ENTER to exit console.

```

Mirrored Rhombus Star Pattern

The code for the mirrored rhombus star pattern is given below:

```

#include <stdio.h>

int main()
{
    int n;

    printf("Enter the number of rows");

    scanf("%d",&n);

    for(int i=1;i<=n;i++)
    {

```

```

    for(int j=1;j<i;j++)

    {

        printf(" ");

    }

    for(int k=1;k<=n;k++)

    {

        printf("*");

    }

    printf("\n");

}

return 0;

}

```

Output

```

Enter the number of rows 5
*****
 *****
  *****
   *****
    *****

...Program finished with exit code 0
Press ENTER to exit console.

```

Hollow Mirrored Rhombus Star Pattern

The code for the hollow mirrored rhombus star pattern is given below:

```
#include <stdio.h>

int main()

{

    int n;

    printf("Enter the number of rows");

    scanf("%d",&n);

    for(int i=1;i<=n;i++)

    {

        for(int j=1;j<i;j++)

        {

            printf(" ");

        }

        for(int k=1;k<=n;k++)

        {

            if(i==1 || i==n || k==1 || k==n)

                printf("*");

            else

                printf(" ");

        }

        printf("\n");

    }
```

```

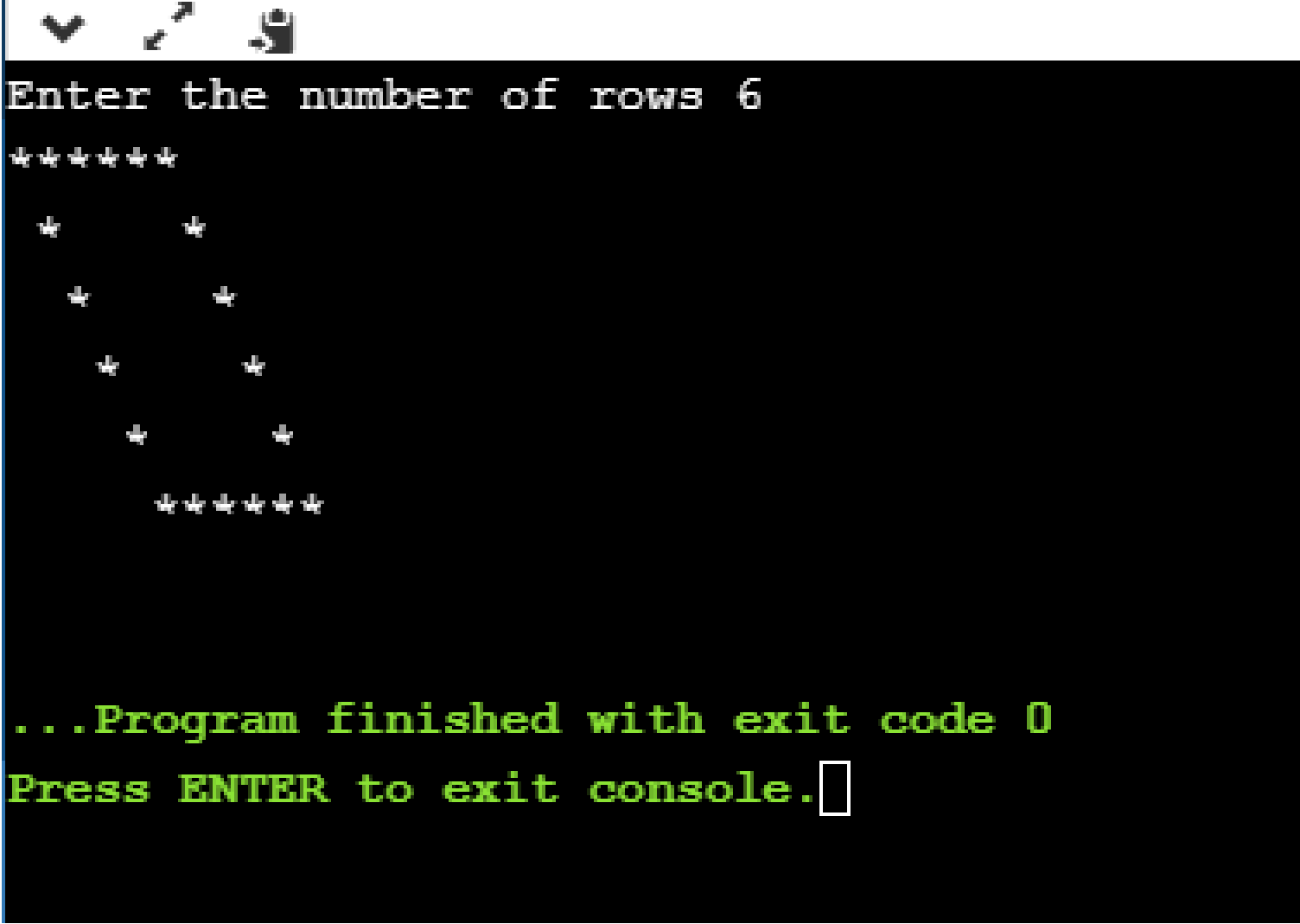
    }

    return 0;

}

```

Output



```

Enter the number of rows 6
*****
 *      *
 *      *
 *      *
 *      *
 *      *
*****

...Program finished with exit code 0
Press ENTER to exit console.

```

Right Triangle Star Pattern

The code for the right triangle star pattern is given below:

```

#include <stdio.h>

int main()

{

    int n;

    printf("Enter the number of rows");

    scanf("%d",&n);

    for(int i=1;i<=n;i++)

    {

```

```

        for(int j=1;j<=i;j++)

        {

            printf(" * ");

        }

        printf("\n");

    }

    return 0;

}

```

Output

```

Enter the number of rows 9
*
* *
* * *
* * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *

...Program finished with exit code 0
Press ENTER to exit console.

```

Mirrored Right Triangle Star Pattern

The code for the mirrored right triangle star pattern is given below:

```

#include <stdio.h>

int main()

```

```

{

    int n,m=1;

    printf("Enter the number of rows");

    scanf("%d",&n);

    for(int i=n;i>=1;i--)

    {

        for(int j=1;j<=i-1;j++)

        {

            printf(" ");

        }

        for(int k=1;k<=m;k++)

        {

            printf("*");

        }

        printf("\n");

        m++;

    }

    return 0;

}

```

Output

```
Enter the number of rows 6
  *
 * *
* * *
* * * *
* * * * *
* * * * *

...Program finished with exit code 0
Press ENTER to exit console.
```

Hollow Right Triangle Star Pattern

The code for the hollow right triangle star pattern is given below:

```
#include <stdio.h>

int main()
{
    int n;

    printf("Enter the number of rows");

    scanf("%d",&n);

    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=i;j++)
        {
            if(j==1 || i==j || i==n )
            {
                printf("*");
            }
            else
            {
                printf(" ");
            }
        }
        printf("\n");
    }
}
```

```

    }

    else

        printf(" ");

    }

    printf("\n");

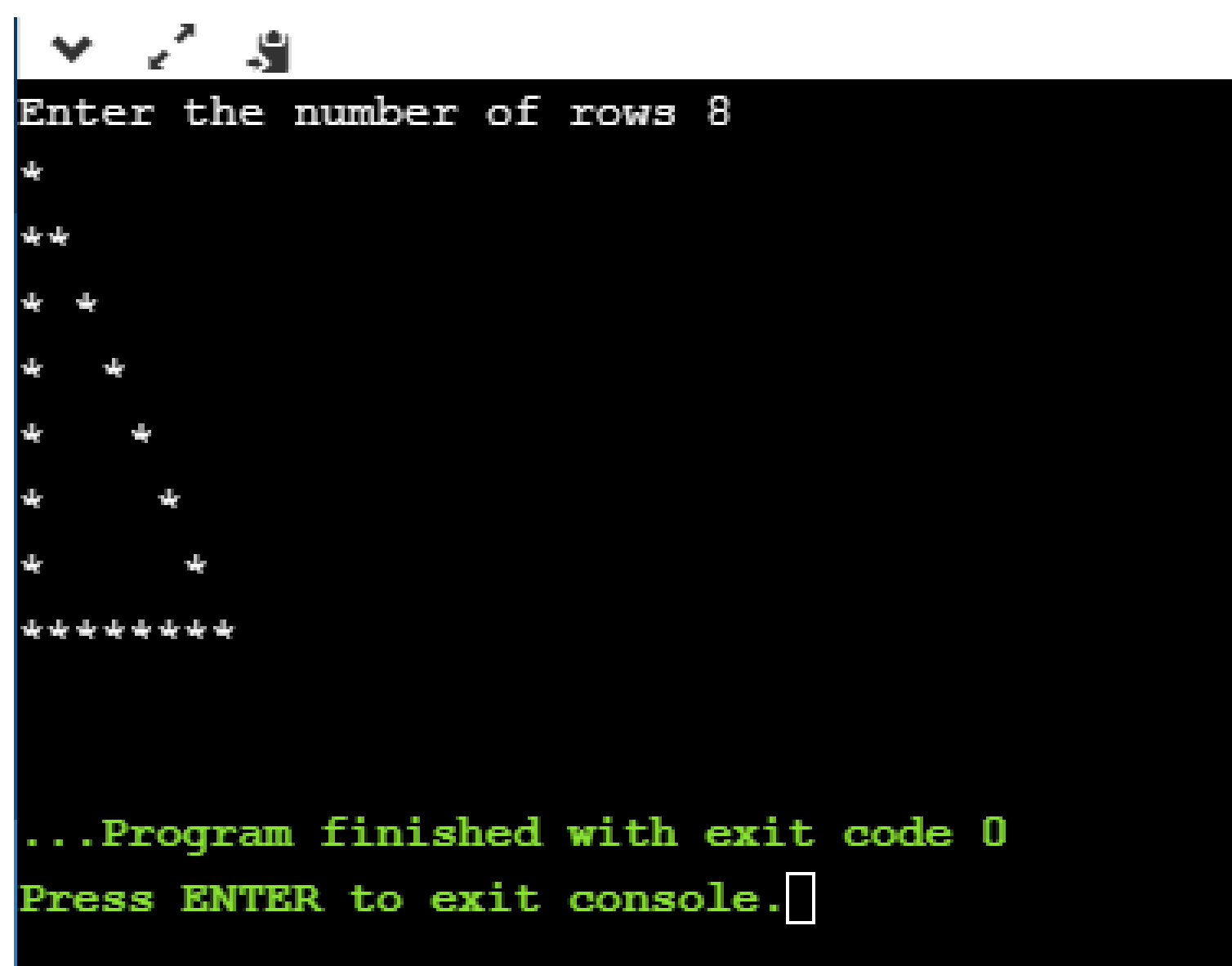
}

return 0;

}

```

Output



```

Enter the number of rows 8
*
**
*  *
*   *
*    *
*     *
*      *
*       *
*****

...Program finished with exit code 0
Press ENTER to exit console.

```

Hollow Mirrored Right Triangle Star Pattern

The code for the hollow mirrored right triangle star pattern is given below:

```

#include <stdio.h>

int main()

{

```

```

int n,m=1;

printf("Enter the number of rows");

scanf("%d",&n);

for(int i=n;i>=1;i--)

{

    for(int j=1;j<=i-1;j++)

    {

        printf(" ");

    }

    for(int k=1;k<=m;k++)

    {

        if(k==1 || k==m || m==n)

            printf("*");

        else

            printf(" ");

    }

    printf("\n");

    m++;

}

return 0;

}

```

Output

```
Enter the number of rows 7
  *
 * *
* * *
* * *
* * *
* * *
* * *
*****

...Program finished with exit code 0
Press ENTER to exit console.
```

Inverted Right Triangle Star Pattern

The code for the inverted right triangle star pattern is given below:

```
#include <stdio.h>

int main()
{
    int n,m=1;

    printf("Enter the number of rows");

    scanf("%d",&n);

    for(int i=n;i>=1;i--)
    {
        for(int j=1;j<=i;j++)
        {
            printf("*");
        }

        printf("\n");
    }
}
```

```

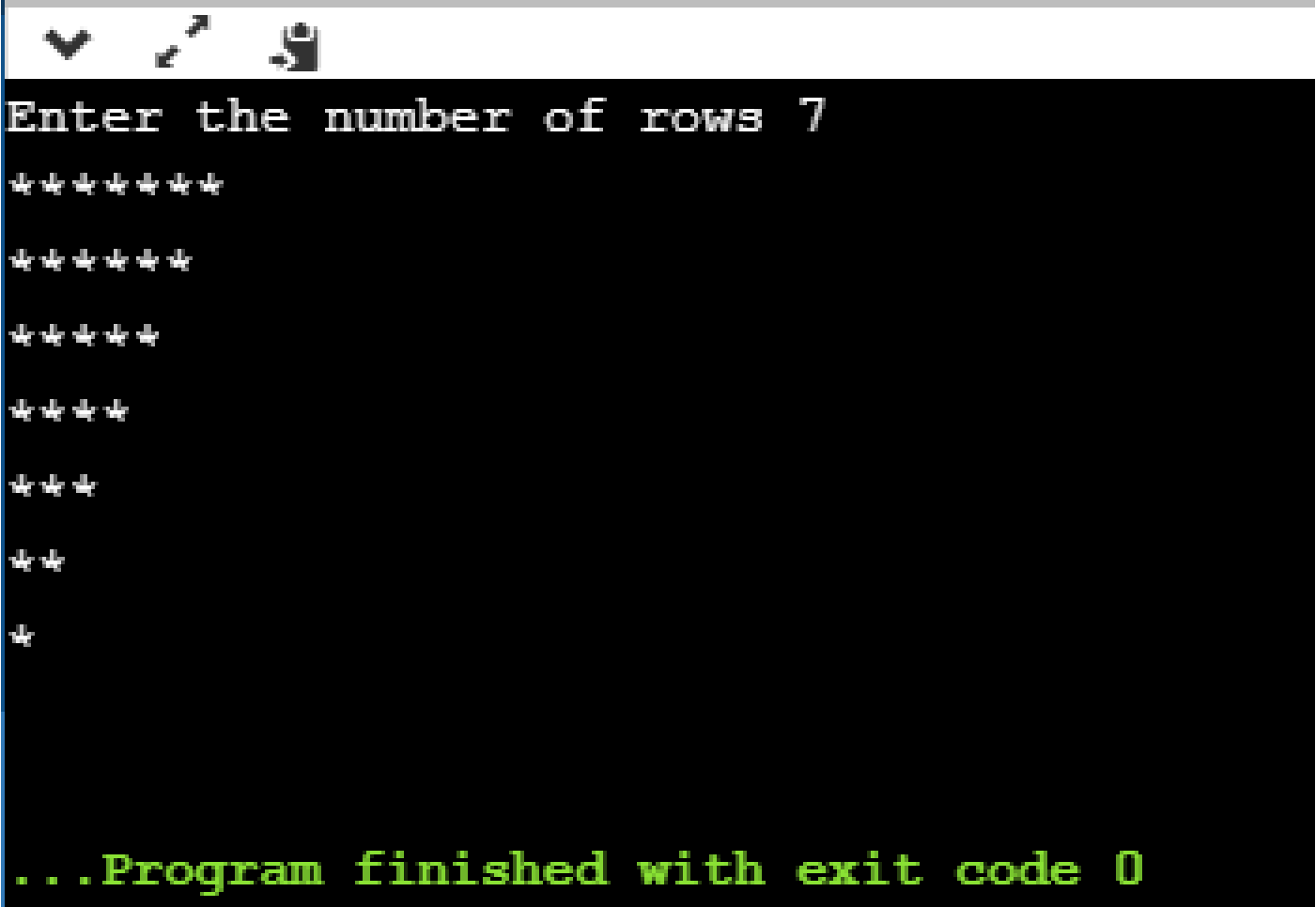
    }

    return 0;

}

```

Output



```

Enter the number of rows 7
*****
*****
*****
****
***
**
*

...Program finished with exit code 0

```

Inverted Mirrored Right Triangle Star Pattern

The code for the inverted mirrored right triangle star pattern is given below:

```

#include <stdio.h>

int main()
{
    int n,m;

    printf("Enter the number of rows");

    scanf("%d",&n);

    m=n;

    for(int i=1;i<=n;i++)
    {

```

```

        for(int j=1;j<i;j++)

        {

            printf(" ");

        }

        for(int k=1;k<=m;k++)

        {

            printf("*");

        }

        m--;

        printf("\n");

    }

    return 0;

}

```

Output

```

Enter the number of rows 8
*****
 *****
  *****
   *****
    *****
     *****
      *****
       *****

...Program finished with exit code 0
Press ENTER to exit console.

```

Hollow Inverted Right Triangle Star Pattern

The code for the hollow inverted right triangle star pattern is given below:

```
#include <stdio.h>

int main()

{

    int n,m=1;

    printf("Enter the number of rows");

    scanf("%d",&n);

    for(int i=n;i>=1;i--)

    {

        for(int j=1;j<=i;j++)

        {

            if(j==1 || j==i || i==n)

                printf("*");

            else

                printf(" ");

        }

        printf("\n");

    }

    return 0;

}
```

Output

```
Enter the number of rows 9
*****
*       *
*     *
*   *
* *
*
**
*

...Program finished with exit code 0
Press ENTER to exit console.
```

Hollow Inverted Mirrored Right Triangle Star Pattern

The code for the hollow inverted mirrored right triangle star pattern is given below:

```
#include <stdio.h>

int main()
{
    int n,m;

    printf("Enter the number of rows");

    scanf("%d",&n);

    m=n;

    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<i;j++)

            printf(" ");
```

```

    }

    for(int k=1;k<=m;k++)

    {

        if(i==1 || k==1 || k==m)

            printf("*");

        else

            printf(" ");

    }

    m--;

    printf("\n");

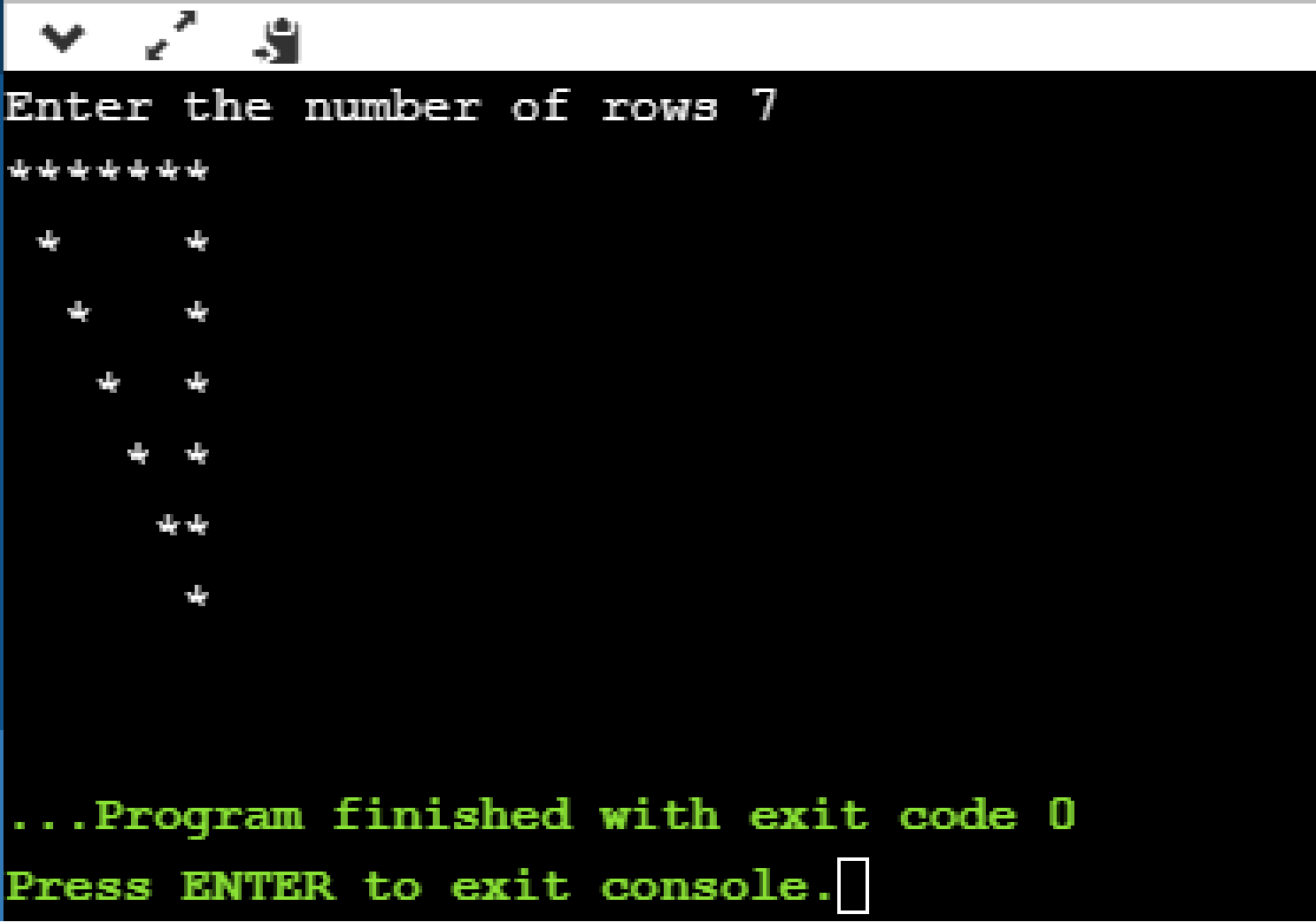
}

return 0;

}

```

Output



```

Enter the number of rows 7
*****
 *   *
  * *
   * *
  * *
   **
    *

...Program finished with exit code 0
Press ENTER to exit console.

```

Pyramid Star Pattern

The code for the pyramid star pattern is given below:

```
#include <stdio.h>

int main()

{

    int n,m;

    printf("Enter the number of rows");

    scanf("%d",&n);

    m=n;

    for(int i=1;i<=n;i++)

    {

        for(int j=1;j<=m-1;j++)

        {

            printf(" ");

        }

        for(int k=1;k<=2*i-1;k++)

        {

            printf("*");

        }

        m--;

        printf("\n");

    }
```

```
return 0;
}
```

Output

```
Enter the number of rows 9  
    *  
   ***  
  *****  
 ****  
*****  
*****  
*****  
*****  
*****  
*****  
*****
```

...Program finished with exit code 0
Press ENTER to exit console.

Hollow Pyramid Star Pattern

The code for the hollow pyramid star pattern is given below:

```
#include <stdio.h>

int main()
{

    int n,m;

    printf("Enter the number of rows");

    scanf("%d",&n);

    m=n;

    for(int i=1;i<=n;i++)
```

```

{

    for(int j=1;j<=m-1;j++)

    {

        printf(" ");

    }

    for(int k=1;k<=2*i-1;k++)

    {

        if(k==1 || k==2*i-1 || i==n)

            printf("*");

        else

            printf(" ");

    }

    m--;

    printf("\n");

}

return 0;

}

```

Output

```
Enter the number of rows 6

      *
     * *
    *  *
   *   *
  *    *
 *     *
*****

...Program finished with exit code 0
Press ENTER to exit console.
```

Inverted Pyramid Star Pattern

The code for the inverted pyramid is given below:

```
#include <stdio.h>

int main()
{
    int n,m=1;

    printf("Enter the number of rows");

    scanf("%d",&n);

    for(int i=n;i>=1;i--)
    {
        for(int j=1;j<m;j++)

        {
            printf(" ");
        }

        for(int k=1;k<=2*i-1;k++)
```

```

{

    printf("*");

}

m++;

printf("\n");

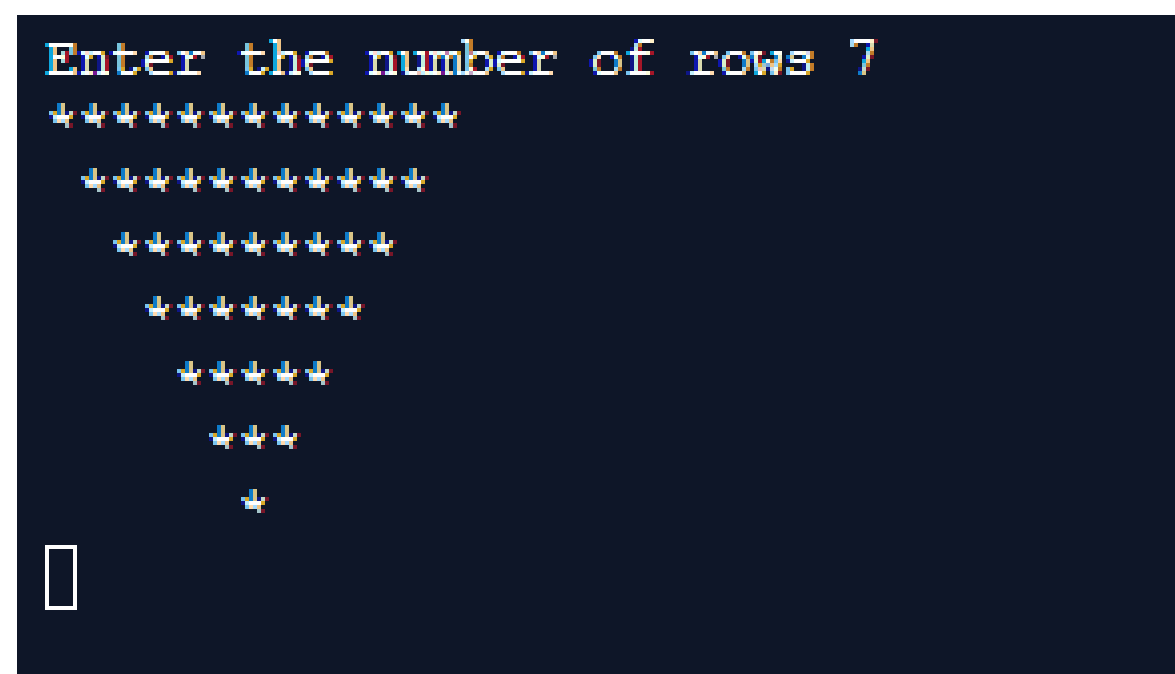
}

return 0;

}

```

Output



```

Enter the number of rows 7
*****
 *   *
  * * 
   ***
  * * 
 *   *
  * * 
   *

```

Hollow Inverted Pyramid Star Pattern

The code for the hollow inverted pyramid star pattern is given below:

```

#include <stdio.h>

int main()

{

    int n,m=1;

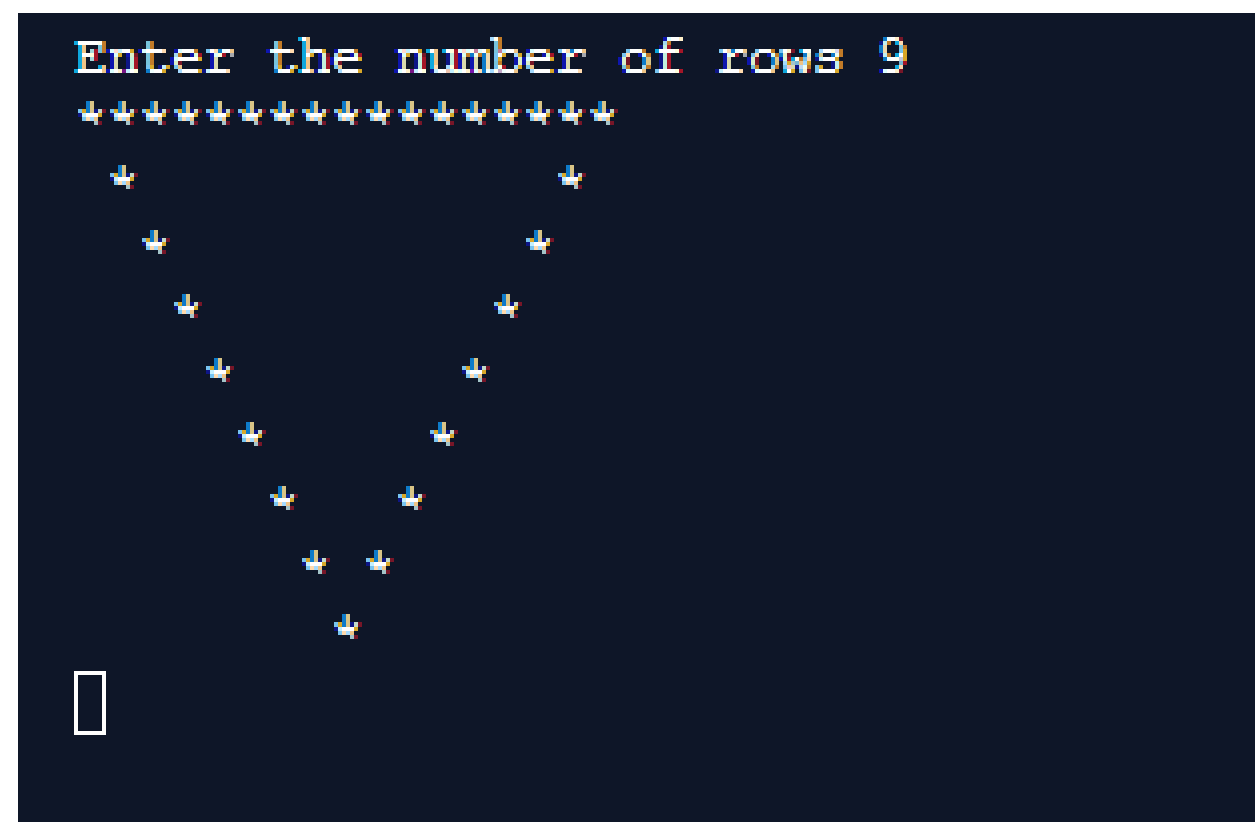
    printf("Enter the number of rows");

    scanf("%d",&n);

```

```
for(int i=n;i>=1;i--)  
  
{  
  
    for(int j=1;j<m;j++)  
  
    {  
  
        printf(" ");  
  
    }  
  
    for(int k=1;k<=2*i-1;k++)  
  
    {  
  
        if(k==1 || k==2*i-1 || i==n)  
  
            printf(" *");  
  
        else  
  
            printf(" ");  
  
    }  
  
    m++;  
  
    printf("\n");  
  
}  
  
return 0;  
  
}
```

Output



Half Diamond Star Pattern

The code for the half diamond star pattern is given below:

```
#include <stdio.h>

int main()

{

    int n,m=1;

    printf("Enter the number of columns");

    scanf("%d",&n);

    for(int i=1;i<=n;i++)

    {

        for(int j=1;j<=i;j++)

        {

            printf("*");

        }

        printf("\n");

    }
```

```

for(int i=n-1;i>=1;i--)

{

    for(int j=1;j<=i;j++)

    {

        printf("*");

    }

    printf("\n");

}

return 0;

}

```

Output

```

Enter the number of columns 7
*
**
***
****
*****
******
*****
****
***
**
*

```

Diamond Star Pattern

The code for the diamond star pattern is given below:

```

#include <stdio.h>

int main(void) {

```

```

int n;

printf("Enter the number of rows\n");

scanf("%d",&n);

int spaces=n-1;

int stars=1;

for(int i=1;i<=n;i++)

{

    for(int j=1;j<=spaces;j++)

    {

        printf(" ");

    }

    for(int k=1;k<=stars;k++)

    {

        printf("*");

    }

    if(spaces>i)

    {

        spaces=spaces-1;

        stars=stars+2;

    }

    if(spaces<i)

    {

```

```
spaces=spaces+1;

stars=stars-2;

}

printf("\n");

}

return 0;}
```

Output

```
➤ ./main
Enter the number of rows
7

  *
 ***
*****
*****
*****
 ***
  *

```

Right Arrow Star Pattern

The code of the right arrow star pattern is given below:

```
#include <stdio.h>

int main(void) {

    int n;

    printf("Enter the number of columns");

    scanf("%d",&n);

    //printing the upper part of the pattern..

    for(int i=0;i<n;i++)
```

```

{

    for(int j=0;j<i;j++)

    {

        printf(" ");

    }

    for(int k=1;k<=n-i;k++)

    {

        printf(" *");

    }

    printf("\n");

}

for(int i=1;i<n;i++)

{

    for(int j=1;j<n-i;j++)

    {

        printf(" ");

    }

    for(int k=1;k<=i+1;k++)

    {

        printf(" *");

    }

    printf("\n");

```

```
}  
  
return 0;  
  
}
```

Output

```

❯ ./main
Enter the number of columns
5
*****
 *   *
  *   *
   *   *
    *   *
     *
    *   *
   *   *
  *   *
 *   *
*****

```

Left Arrow Star Pattern

The code for the Left arrow star pattern is given below:

```
#include <stdio.h>

int main(void) {

    int n;

    printf("Enter the number of columns");

    scanf("%d",&n);

    //printing the upper part of the pattern..

    for(int i=1;i<=n;i++)

    {

        for(int j=1;j<=n-i;j++)

        {
```

```

        printf(" ");

    }

    for(int k=0;k<=n-i;k++)

    {

        printf("*");

    }

    printf("\n");

}

for(int i=1;i<n;i++)

{

    for(int j=1;j<i+1;j++)

    {

        printf(" ");

    }

    for(int k=1;k<=i+1;k++)

    {

        printf("*");

    }

    printf("\n");

}

return 0;

}

```

Output

```
➤ ./main
Enter the number of columns 5
+++++
  ++++
   +++
    ++
     +
    ++
   +++
  ++++
 +++++

```

Plus Star Pattern

The code for the plus star pattern is given below:

```
#include <stdio.h>

int main(void) {

    int n;

    printf("Enter the odd number only");

    scanf("%d", &n);

    for(int i=1;i<=n;i++)

    {

        if(i==((n/2)+1))

        {

            for(int j=1;j<=n;j++)

            {

                printf("+");
```

```

    }

}

else

{

for(int j=1;j<=n/2;j++)

{

    printf(" ");

}

printf("+");

}

printf("\n");

}

return 0;

}

```

Output

```

❏ ./main
Enter the number of rows 13
      +
      +
      +
      +
      +
      +
      +
+++++
      +
      +
      +
      +
      +
      +
      +
❏

```

X Star Pattern

The code for the X star pattern is given below:

```
#include <stdio.h>

int main(void) {

    int n,m;

    printf("Enter the number");

    scanf("%d",&n);

    m=2*n-1;

    for(int i=1;i<=m;i++)

    {

        for(int j=1;j<=m;j++)

        {

            if(i==j || j==(m-i+1))

            {

                printf("*");

            }

            else

            {

                printf(" ");

            }

        }

    }

    printf("\n");
```

```
}
```

```
return 0;
```

```
}
```

Output

